

CHILLISOURCE GAME ENGINE PARTICLE STUDY

By

Angela Nicole Gross

Bachelor of Science, The University of Montana, Missoula, MT, 2014

Thesis

presented in partial fulfillment of the requirements
for the degree of

Master of Science
in Computer Science

The University of Montana
Missoula, MT

Autumn 2016

Approved by:

J.B. Alexander S. Ross Ph.D., Dean
Graduate School

Travis Wheeler Ph.D., Chair
Computer Science

Michael Cassens M.S.
Computer Science

Johnathan Bardsley Ph.D.
Mathematical Sciences

© COPYRIGHT

by

Angela Nicole Gross

2016

All Rights Reserved

Gross, Angela N., M.S., December 2016

Computer Science

ChilliSource Game Engine Particle Study

Chairperson: Travis Wheeler

This is the short summary of why this thesis is awesome and *YOU* really want to read it!

ACKNOWLEDGMENTS

Thanks guys ...

TABLE OF CONTENTS

COPYRIGHT	ii
ABSTRACT	iii
ACKNOWLEDGMENTS	iv
CODE LISTINGS	ix
LIST OF FIGURES	xi
LIST OF TABLES	xvii
CHAPTER 1 INTRODUCTION	1
1.1 Thesis Organization	1
CHAPTER 2 BACKGROUND	2
2.1 Particle Systems	2
2.1.1 Overview	2
2.1.2 Particle Model	3
2.1.2.1 Life Cycle	3
2.1.2.2 Particle Attributes	4
2.1.2.3 Hierarchy	5
2.2 The ChilliSource Game Engine	5
2.2.1 Overview	5
2.2.2 Architecture	6
2.2.3 Life Cycle	7

2.3	ChilliSource Particle Effect Components	10
2.3.1	Overview	10
2.3.2	Architecture	11
2.3.2.1	Overview	11
2.3.2.2	Particle Effect	11
2.3.2.3	Particle Drawable	14
2.3.2.4	Particle Emitter	14
2.3.2.5	Particle Affector	15
2.3.2.6	Particle Array	16
2.3.2.7	Concurrent Particle Data	16
2.3.2.8	Other Important Attributes	17
2.3.3	Life Cycle	18
2.3.3.1	Overview	18
2.3.3.2	Initialization Phase	18
2.3.3.3	Update Phase	18
2.3.3.4	Render Phase	21
2.3.3.5	Clean Up Phase	22
2.3.4	Usage	22
CHAPTER 3 THE CHILLISOURCE AUTOMATED PONG GAME . .		26
3.1	Overview	26
3.2	Architecture	27
3.2.1	Overview	27
3.2.2	CSAPong Application	28
3.2.3	Application Systems	28
3.2.4	Game State	29
3.2.5	State Systems	30

3.3	Gameplay	30
3.3.1	Logistics	30
3.3.2	Examples	31
CHAPTER 4 INSTRUMENTATION		35
4.1	Overview	35
4.2	Metrics System	35
4.2.1	Metadata	35
4.2.2	Metrics	38
4.2.3	Particle Effect Definition	39
4.3	Timing System	40
4.3.1	Visual Studio	40
4.3.2	Shiny	42
4.3.3	Timing Application System	43
4.4	Output and its Evolution	44
4.5	Initial Results	45
CHAPTER 5 CHILLISOURCE PARTICLE EFFECT COMPONENT		
CASE STUDIES		49
5.1	Counting Contention	49
5.2	Using More Than One Mutex	51
5.2.1	Code	51
5.2.2	Results	53
5.2.3	Discussion	56
5.3	Using A Lock Free Data Structure	57
5.3.1	Code	57
5.3.2	Results	60
5.3.3	Discussion	63

5.4	Unexpected Results with Multiple Particle Effects	64
5.4.1	Using More Than One Mutex Results	65
5.4.2	Lock Free Results	67
5.4.3	Discussion	69
 CHAPTER 6 CHILLISOURCE PARTICLE EFFECT COMPONENT		
	OBSERVATIONS	73
6.1	Overview	73
6.2	Copying Data with ConcurrentParticleData	73
6.2.1	Overview	73
6.2.2	Code	73
6.2.3	Results	76
6.2.4	Discussion	81
6.3	Multithreading and Task Scheduling	81
6.3.1	The ChilliSource Task Scheduler	81
6.3.2	Code	83
6.3.2.1	Small Task Type	83
6.3.2.2	Game Logic Task Type	83
6.3.2.3	No Task Scheduled (Not Threaded)	84
6.3.3	Results	84
6.3.4	Discussion	89
6.4	Particles Failing to Visually Emit	90
6.4.1	Emission Failure	90
6.4.2	Emission Failure Source	90
6.4.3	Emission Failure Solution	95
 BIBLIOGRAPHY		101

CODE LISTINGS

1	An example ChilliSource particle definition file for a generic particle effect. Note how the initial and changing-over-time (i.e. "affector") <i>Properties</i> can either be random values or constants.	12
2	The struct that is used to describe the properties of a particle within the <i>Particle Array</i> in <i>Particle Effect Component</i>	16
3	The struct that is used to describe the properties of a particle within the <i>Particle Array</i> in <i>Concurrent Particle Data</i>	17
4	The struct that is used to bundle all information needed by the background task performing the particle update.	19
5	Pseudocode describing how a <i>Particle Effect Component</i> is updated by scheduling a background task.	19
6	Pseudocode describing what CommitParticleData does when it copies its data to the <i>Concurrent Particle Data</i> instance.	21
7	Pseudocode showing how a <i>Particle Effect Component</i> uses its <i>Drawable</i> to render itself.	22
8	Creating a scene that adds a basic <i>Particle Effect Component</i> in ChilliSource.	23
9	The function to create an ambient light with the <i>Basic Entity Factory</i> state system. . .	23
10	Creating a basic particle effect component with ChilliSource.	24
11	An example invocation of the generate particles script.	39
12	Using the pseudocode from Code Listing 5, this shows how <i>Shiny</i> could be used to instrument ParticleUpdateTask	42

13	Using the pseudocode from Code Listing 5, this shows how our custom <i>Timing System</i> could be used to instrument ParticleUpdateTask	43
14	Pseudocode illustrating the changes needed to utilize more than one mutex in the CommitParticleData function.	52
15	Pseudocode illustrating the changes needed to utilize more than one mutex in the Draw function.	52
16	The struct that is used to within the particle array in <i>Concurrent Particle Data</i> during the Lock Free case study.	58
17	Pseudocode illustrating the changes needed to make the <i>particles array</i> a lock free data structure in the CommitParticleData function.	59
18	Pseudocode illustrating the changes needed to make the <i>particles array</i> a lock free data structure in the Draw function.	59
19	Pseudocode describing the important changes needed to remove the <i>ConcurrentParticleData</i> instance from the <i>Particle Effect Component</i>	75
20	Pseudocode describing the important changes needed to remove the <i>ConcurrentParticleData</i> instance from the <i>Particle Drawable</i>	76
21	Pseudocode showing how ChilliSource's <i>Task Scheduler</i> schedules <i>small tasks</i> , <i>game logic tasks</i> , and <i>main thread tasks</i>	81
22	Pseudocode showing how <i>game logic tasks</i> force the main thread to wait until they are finished before allowing the main thread to continue.	83
23	Pseudocode showing how ParticleUpdateTask was scheduled as a <i>small task</i>	83
24	Pseudocode showing how ParticleUpdateTask was scheduled as a <i>game logic task</i>	83
25	Pseudocode showing how ParticleUpdateTask was not scheduled and was simply ran on the main thread.	84

LIST OF FIGURES

Figure 2.1	Particle effect created by William T. Reeves	2
Figure 2.2	Cubes emitting particles	3
Figure 2.3	Basic structure of a ChilliSource application	7
Figure 2.4	An example of the Update life cycle event	9
Figure 2.5	Basic structure of a ChilliSource Particle Effect Component	11
Figure 3.1	A series of screenshots illustrating how the original pong game works.	27
Figure 3.2	A series of screenshots illustrating how the automated pong game works.	28
Figure 3.3	The basic architecture of the automated pong game.	29
Figure 3.4	A series of screenshots showing the automated game with no particles.	32
Figure 3.5	A series of screenshots showing the automated game with a single, giant burst of particles.	33
Figure 3.6	A series of screenshots showing the automated game with a steady stream of particle emissions.	34
Figure 4.1	An example of metadata output by the <i>Metrics System</i>	36
Figure 4.2	An example of the changing particle values during games in CSAPong.	36
Figure 4.3	An example of metrics output by the <i>Metrics System</i> based on the metadata from Figure 4.1.	38
Figure 4.4	Shows the parameters that the generate_particles.py script can use.	39

Figure 4.5	Shows the particle files that were generated by an invocation like in Code Listing 11.	40
Figure 4.6	Shows an example of Visual Studio’s generated call tree for CPU instrumentation when ran using the metadata from Figure 4.1. . .	41
Figure 4.7	Shows the same call tree from Figure 4.6, but sorted and filtered. .	41
Figure 4.8	An example of timing output by the <i>Timing System</i> based on the metadata from Figure 4.1.	44
Figure 4.9	The same example as Figure 4.8, but this series of games did not have the inner loop timed.	44
Figure 4.10	An example of an HDF5 file containing sample metrics, timing, and metadata information.	46
Figure 4.11	A Visual Studio call tree sorted in the same way as Figure 4.7, but from the games based on the metadata in Figure 4.12.	47
Figure 4.12	Metadata of a series of games that demonstrated contention in CommitParticleData	47
Figure 4.13	Timing output based on the metadata in Figure 4.12.	47
Figure 5.1	The results of ”counting the contention” between the background and main threads in <i>Concurrent Particle Data</i>	49
Figure 5.2	The metadata used for the 8 series of automated games for the case studies.	53
Figure 5.3	More Than One Mutex Case Study: A graph showing <i>render ball function calls</i> over <i>total maximum particles</i> for a single particle effect.	54
Figure 5.4	More Than One Mutex Case Study: A graph showing <i>particles actually rendered</i> over <i>total maximum particles</i> for a single particle effect.	54

Figure 5.5	More Than One Mutex Case Study: A graph showing the number of section calls for various sections of code for a single particle effect.	55
Figure 5.6	More Than One Mutex Case Study: A graph showing the time spent in various sections of code for a single particle effect.	55
Figure 5.7	Lock Free Case Study: A graph showing <i>render ball function calls</i> over <i>total maximum particles</i> for a single particle effect.	61
Figure 5.8	Lock Free Case Study: A graph showing <i>particles actually rendered</i> over <i>total maximum particles</i> for a single particle effect.	61
Figure 5.9	Lock Free Case Study: A graph showing the number of section calls for various sections of code for a single particle effect.	62
Figure 5.10	Lock Free Case Study: A graph showing the time spent in various sections of code for a single particle effect.	62
Figure 5.11	The metadata used for the 8 series of automated games for the unexpected results from the case studies.	64
Figure 5.12	More Than One Mutex Case Study: A graph showing <i>render ball function calls</i> over <i>total maximum particles</i> for ten particle effects.	65
Figure 5.13	More Than One Mutex Case Study: A graph showing <i>particles actually rendered</i> over <i>total maximum particles</i> for ten particle effects.	65
Figure 5.14	More Than One Mutex Case Study: A graph showing the number of section calls for various sections of code for ten particle effects. .	66
Figure 5.15	More Than One Mutex Case Study: A graph showing the time spent in various sections of code for ten particle effects.	66
Figure 5.16	Lock Free Case Study: A graph showing <i>render ball function calls</i> over <i>total maximum particles</i> for ten particle effects.	67
Figure 5.17	Lock Free Case Study: A graph showing <i>particles actually rendered</i> over <i>total maximum particles</i> for ten particle effects.	67

Figure 5.18	Lock Free Case Study: A graph showing the number of section calls for various sections of code for ten particle effects.	68
Figure 5.19	Lock Free Case Study: A graph showing the time spent in various sections of code for ten particle effects.	68
Figure 5.20	A summary of the timed sections for the unexpected results for the Lock Free and > 1 Mutex case studies for a single particle effect. .	70
Figure 5.21	A summary of the timed sections for the unexpected results for the Lock Free and > 1 Mutex case studies for ten particle effects. . . .	70
Figure 5.22	The lifetime of a single particle effect in relation to its scheduled background thread and the main thread.	71
Figure 5.23	The lifetime of a many particle effects in relation to their scheduled background threads and the main thread.	71
Figure 6.1	Using Copies Observation: A graph showing <i>render ball function calls</i> over <i>total maximum particles</i> for a single particle effect. . . .	77
Figure 6.2	Using Copies Observation: A graph showing <i>particles actually rendered</i> over <i>total maximum particles</i> for a single particle effect. . . .	77
Figure 6.3	Using Copies Observation: A graph showing <i>render ball function calls</i> over <i>total maximum particles</i> for ten particle effects.	78
Figure 6.4	Using Copies Observation: A graph showing <i>particles actually rendered</i> over <i>total maximum particles</i> for ten particle effects.	78
Figure 6.5	Using Copies Observation: A graph showing the number of section calls for various sections of code for a single particle effect.	79
Figure 6.6	Using Copies Observation: A graph showing the time spent in various sections of code for a single particle effect.	79
Figure 6.7	Using Copies Observation: A graph showing the number of section calls for various sections of code for ten particle effects.	80

Figure 6.8	Using Copies Observation: A graph showing the time spent in various sections of code for ten particle effects.	80
Figure 6.9	Task Scheduling Observation: A graph showing <i>render ball function calls</i> over <i>total maximum particles</i> for a single particle effect. . . .	85
Figure 6.10	Task Scheduling Observation: A graph showing <i>particles actually rendered</i> over <i>total maximum particles</i> for a single particle effect. . .	85
Figure 6.11	Task Scheduling Observation: A graph showing <i>render ball function calls</i> over <i>total maximum particles</i> for ten particle effects.	86
Figure 6.12	Task Scheduling Observation: A graph showing <i>particles actually rendered</i> over <i>total maximum particles</i> for ten particle effects. . . .	86
Figure 6.13	Task Scheduling Observation: A graph showing the number of section calls for various sections of code for a single particle effect. . .	87
Figure 6.14	Task Scheduling Observation: A graph showing the time spent in various sections of code for a single particle effect.	87
Figure 6.15	Task Scheduling Observation: A graph showing the number of section calls for various sections of code for ten particle effects.	88
Figure 6.16	Task Scheduling Observation: A graph showing the time spent in various sections of code for ten particle effects.	88
Figure 6.17	Emission Failure Observation: A graph showing <i>render ball function calls</i> over <i>total maximum particles</i> for a single particle effect. . . .	91
Figure 6.18	Emission Failure Observation: A graph showing <i>particles actually rendered</i> over <i>total maximum particles</i> for a single particle effect. . .	91
Figure 6.19	Emission Failure Observation: A graph showing <i>render ball function calls</i> over <i>total maximum particles</i> for a single particle effect. . . .	92
Figure 6.20	Emission Failure Observation: A graph showing <i>particles actually rendered</i> over <i>total maximum particles</i> for a single particle effect. . .	92

Figure 6.21	Emission Failure Observation: A graph showing the number of section calls for various sections of code for a single particle effect. . .	93
Figure 6.22	Emission Failure Observation: A graph showing the time spent in various sections of code for a single particle effect.	93
Figure 6.23	Emission Failure Observation: A graph showing the number of section calls for various sections of code for ten particle effects. . . .	94
Figure 6.24	Emission Failure Observation: A graph showing the time spent in various sections of code for ten particle effects.	94
Figure 6.25	Emission Failure Solution Observation: A graph showing <i>render ball function calls</i> over <i>total maximum particles</i> for a single particle effect.	97
Figure 6.26	Emission Failure Solution Observation: A graph showing <i>particles actually rendered</i> over <i>total maximum particles</i> for a single particle effect.	97
Figure 6.27	Emission Failure Solution Observation: A graph showing <i>render ball function calls</i> over <i>total maximum particles</i> for a single particle effect.	98
Figure 6.28	Emission Failure Solution Observation: A graph showing <i>particles actually rendered</i> over <i>total maximum particles</i> for a single particle effect.	98
Figure 6.29	Emission Failure Observation: A graph showing the number of section calls for various sections of code for a single particle effect. . .	99
Figure 6.30	Emission Failure Observation: A graph showing the time spent in various sections of code for a single particle effect.	99
Figure 6.31	Emission Failure Observation: A graph showing the number of section calls for various sections of code for ten particle effects. . . .	100
Figure 6.32	Emission Failure Observation: A graph showing the time spent in various sections of code for ten particle effects.	100

LIST OF TABLES

CHAPTER 1 INTRODUCTION

1.1 Thesis Organization

The rest of this thesis is organized as follows:

- **Chapter 2** Review of particle systems and the ChilliSource game engine
- **Chapter 3** Overview of the automated pong game that was created to facilitate case studies
- **Chapter 4** Overview of the systems used to instrument the ChilliSource engine
- **Chapter 5** Discussion of the two case studies performed to explore contention in large data structures
- **Chapter 6** Discussion of three interesting aspects of the ChilliSource particle effect component

CHAPTER 2 BACKGROUND

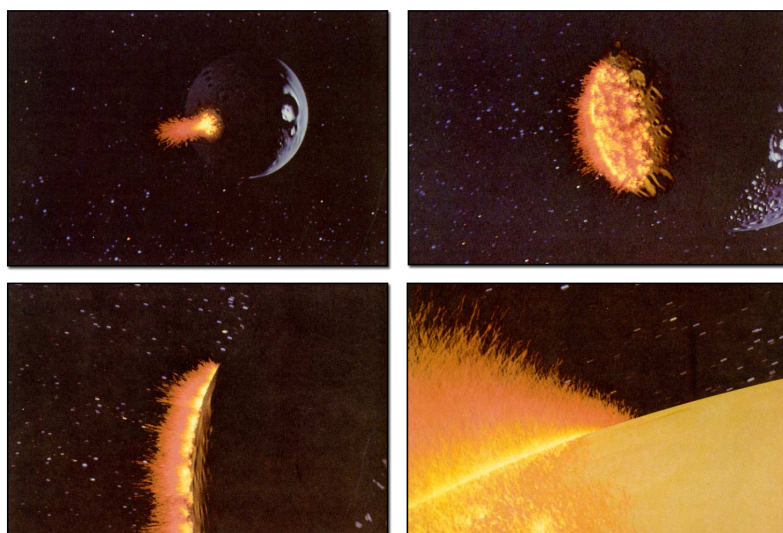


Figure 2.1: The particle effect created by William T. Reeves from the film *Star Trek II: The Wrath of Khan*

2.1 Particle Systems

2.1.1 Overview

Visual representation of "fuzzy objects", or objects that do not have well-defined surfaces, in computer graphics is generally considered a difficult task due to the non-deterministic changes in the fuzzy object's shape and form. When working on the film *Star Trek II: The Wrath of Khan*, a researcher at Lucasfilm Ltd., William T. Reeves, developed what he called a **particle system**, which is "... a collection of many minute particles that together represent a fuzzy object. Over a period of time, particles are generated into a system, move

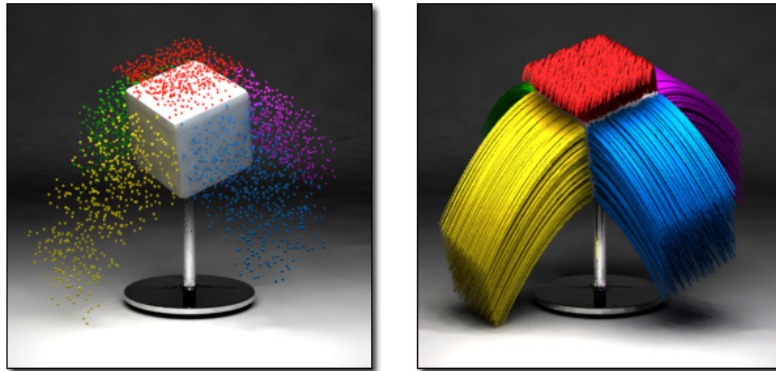


Figure 2.2: Left: A cube emitting a stream of 1000 animated particles from 5 of its 6 faces. Right: A cube emitting a stream of 1000 static particles, or strands, from 5 of its 6 faces

and change from within the system, and die from the system.” [1] Although the concept of modeling these objects as a collection of particles was not novel, Reeves' basic framework for particle systems is still widely used today to create complex visual effects in video games and films.

Fuzzy objects like sparks, clouds, stars, water, and crowds of people can be easily translated and modeled as a particle system, but particle systems can also be used to model objects that contain numerous strands like hair and grass. These objects can be represented by rendering every strand as the entire lifetime of a particle in the system and is then treated as a single strand. The particles in a system that have a distributed lifetime (e.g. sparks) are thought of as *animated* and particles that are rendered all at once (e.g. hair) are thought of as *static*. For the duration of our own study, we have exclusively focused on animated particles.

2.1.2 Particle Model

2.1.2.1 Life Cycle

A particle system has 4 main events that defines its life cycle:

1. Emission (or Generation, according to Reeves)

2. Update
3. Render
4. Death (or Extinction, according to Reeves)

The **Emission** event is the initial process in which particles are generated with their initial values. Often a "generation shape"[1] is used during the emission phase that defines the two or three dimensional space that the particles can originate, or emit, from. Emission can be increased or decreased in frequency using the *emission rate*, and the number of particles that enter the particle system each emission can also be changed using the *particles per emission* variable. After emission, particles in the particle system undergo the **Update** and **Render** events. These events are repeated in perpetuum until the particle dies, or undergoes the **Extinction** phase. The **Update** phase involves iterating over all particles throughout the system and updating their attributes (e.g. position, velocity, etc.). The **Render** phase, on the other hand, is the phase in which all of the particles are actually drawn to the screen.

2.1.2.2 Particle Attributes

As mentioned above, each individual particle in a particle system will have attributes that change over time. Although every particle system is different, the following is what Reeves considered to be the core attributes of a particle:

- Position
- Velocity
- Size
- Color
- Transparency
- Shape
- Lifetime

A particle has a position in two or three dimensional space, and its initial position is determined by the emission process. Other attributes that may also change over time

include its speed, direction, size, color, and transparency. Its lifetime, however, is pre-determined at emission, and it is the attribute that dictates how long the particle "lives", or remains on the screen. In order to decrease uniformity and increase realism, all of these values can vary across the particles in the system. For example, the particles in a given particle system could have a varying lifetime of 5 to 10 seconds so that the particles that shared an emission would not all die simultaneously.

2.1.2.3 Hierarchy

It is often convenient to design a particle system such that it is made up of a hierarchy of child particle systems. This hierarchy "can be used to exert global control on a complicated fuzzy object that is composed of many particle systems." [1] In other words, a fuzzy object could easily be transformed in overall shape, size, color, etc. by sweeping through the descendant particle systems. It also allows the grouping together of particle systems that have different types of particles. For example, a campfire fuzzy object would need smoke, sparks, and the flame itself. All of these components would have distinct particle definitions and would be separate particle systems. A particle system hierarchy, in this case, would allow the smoke, sparks, and flame particle systems to be bundled together as a single fuzzy object.

2.2 The ChilliSource Game Engine

2.2.1 Overview

Chosen for its open source and well-thought out architecture, ChilliSource is the game engine that was used in our study of particle systems. ChilliSource is a free, open source game engine created by ChilliWorks which is a group that is part of United Kingdom's Tag Games. It allows developers to create 2D and 3D content for iOS, Android, Kindle, and Windows operating systems in C++ 11. Some of its key features include networking, shader support, responsive (i.e. adaptive to different resolutions) graphical user interfaces, modular

and extensible lighting and shadows, skinned animation, Facebook connect integration, consistent API, and automatic builds of iOS/Android/Kindle applications. [2]

2.2.2 Architecture

The structure of a ChilliSource application is primarily composition based in that it relies on a hierarchy of modules to define its functionality. The main types of modules that comprise the application are *States*, *Systems*, *Entities* and *Components*. Figure 2.3 demonstrates the architecture of a ChilliSource application using these basic modules.

The *Application* instance initializes and manages *Application Systems*, which are modules that describe functionality used across the entire application. A non-exhaustive list of some application systems include the *Task Manager*, *Texture Provider*, *Render System*, and the *State Manager*. The *State Manager*, as shown in Figure 2.3, manages a stack of *States* and ensures that the state on top of the stack is active.

States can be thought of as different parts of the application such as a main menu, inventory, settings menu, pause screen, and the game scene itself. *States*, like the *Application*, initialize and manage *State Systems*. *State Systems* are similar to *Application Systems* except they describe functionality that only pertains to a single state and its lifetime is limited to that state. An important state system, the *Scene*, manage *Entities* that form that scene. The base *State* also creates and manages the *Canvas*, *Gesture*, and "User" state systems. The *Canvas* renders the user interface, the *Gesture* state system helps receive input events like pinching or tapping, and "User" state systems are custom modules built by the developer to describe functionality for the state.

Entities are essentially objects within the game that are defined by both (a) a transform describing their rotation, size, and position and (b) *Components*, which are modules that describe the entity's functionality. *Entities* can also contain a hierarchy of entities in which, much like in particle systems, their transforms and life cycle events are linked by the hierarchy. Some examples of *Components* are AI Controllers, Health, Damage, and Physics.

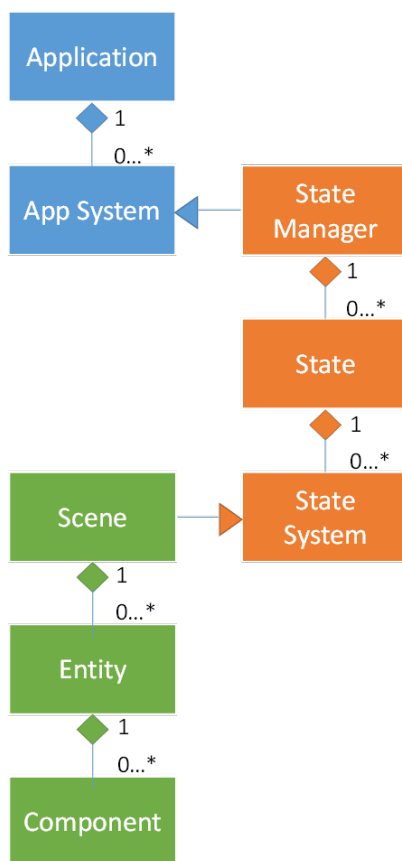


Figure 2.3: The basic structure of a ChilliSource application

ChilliSource’s architecture favors composition over inheritance. For example, if we wanted to create enemy and player *Entities*, we may be tempted to create a base Human *Entity* class that had health related methods and attributes. In order to truly adhere to the spirit of ChilliSource’s design, however, we would create a *Component* called something like **HealthComponent** that could be attached to any single *Entity*. In this way, ChilliSource lends itself to a great deal of code reusability and flexibility.

2.2.3 Life Cycle

The majority of game engines follow a life cycle very similar to the one for particle systems that Reeves developed, which is:

1. Initialize
2. Update
3. Draw
4. Stop

The **Initialize** step usually involves loading textures, sounds, and anything else that the engine needs to do before starting the main game loop. The **Update** step is the heart of game, and is usually driven by some kind of state machine. Some examples of typical states include Menu, Game, Settings, and Exit. The **Draw** step is also part of the main game loop and, just as it sounds, this is where everything is drawn to the screen. Typically, there is some kind of renderer that handles all of the low-level calls to the graphics card (e.g. OpenGL, DirectX, etc.). And, optionally, there is the **Stop** step in which the game has been stopped and all assets and resources will be released. This may or may not be included as a separate state in the **Update** phase, but it is good to logically separate the two events.

ChilliSource, as you may expect, adheres to the basic "Initialize, Update, Draw, and Stop" phases. However, there are a great deal of other events that are perpetuated from the *Application* instance. The following is a list of the main life cycle events that all modules receive:

1. Init
 - This initializes the application and begins the **Update** event.
2. Update
 - This updates the application, its systems, and the state manager's states.
3. Render
 - This renders objects and user interface elements to the screen and is the main draw loop of the application.
4. Suspend

- This will notify active state(s) to pause and it will suspend all systems in reverse order.
5. Resume
 - This resumes the application from a suspended state.
 6. Background
 - This occurs when the application is not in view any longer.
 7. Foreground
 - This occurs when the application is in view again.
 8. Destroy
 - This cleans up the application and releases all systems in reverse order.

All of these events are kicked off from the *Application* instance. The flow of events looks something like *Application* → *Application Systems (State Manager)* → *States* → *State Systems (Scene)* → *Entities* → *Components*. Figure 2.4 shows what this flow looks like for the **Update** life cycle event using pseudocode. Note, however, that each event means something slightly different depending on which module it is called in (e.g. **Update**

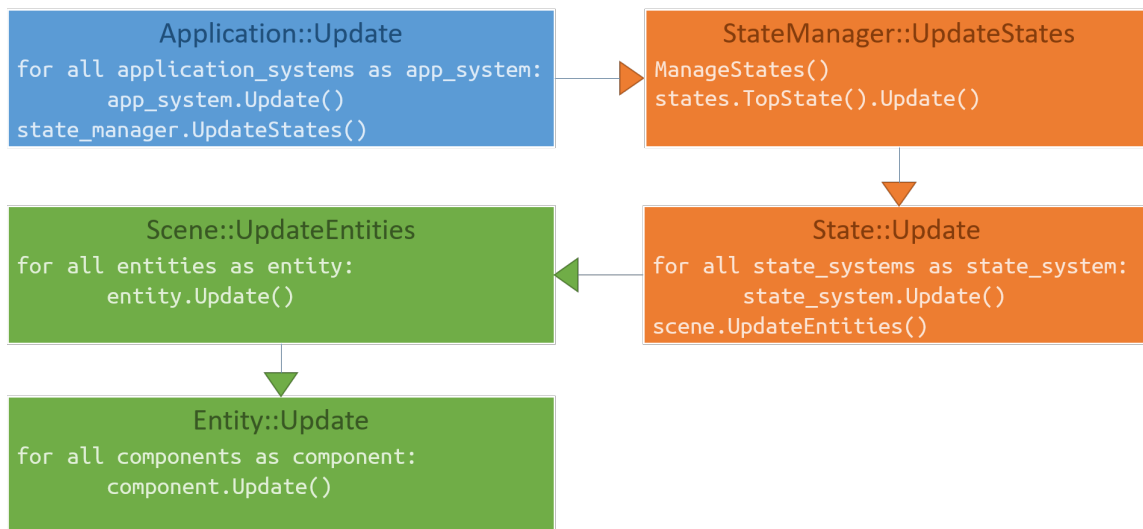


Figure 2.4: An example of the **Update** life cycle event being perpetuated from the *Application* all the way down to a *Component*

in the *Application* means something different than the **Update** in a *State*). Additionally, there are a number of other events that are not discussed here that only have meaning for specific modules (e.g. **OnAddedToEntity** for the *Component* module). See [3] for more information.

Although these are the main events that are automatically pushed by the application, the game developer can create custom events that can be subscribed to by entities, components, and systems. An *Event* can open, close, and notify *Connections*, where a *Connection* allows its user to listen for *Events*. Some examples of custom events are entity collisions, player death, cut scene trigger, game lost, game won, etc.

2.3 ChilliSource Particle Effect Components

2.3.1 Overview

A special type of component in ChilliSource is the *Particle Effect Component*. It inherits from the *Render Component* class¹, which is, just as it sounds, a component that is rendered to the screen. *Render Components* have methods for visibility, culling, casting shadows, and usage of its transformation matrix and sort predicates. They also have virtual methods that child classes can define for materials, transparency, rendering, and bounding boxes. For the purpose of our study, however, the exact details of how a *Render Component* works is outside of our scope. It is only necessary to know that the *Particle Effect Component* is not only updated by its parent entity but is also rendered by the *Application*'s renderer.

¹It's important to note here that the version of ChilliSource that we are using is 1.6.0 and, at the time of writing this paper, it is currently at version 2.0.0. In version 2.0.0, the rendering system is completely overhauled and *Particle Effect Components* are *Volume Components* and not *Render Components*. This new system was not studied and will not be discussed in depth here.

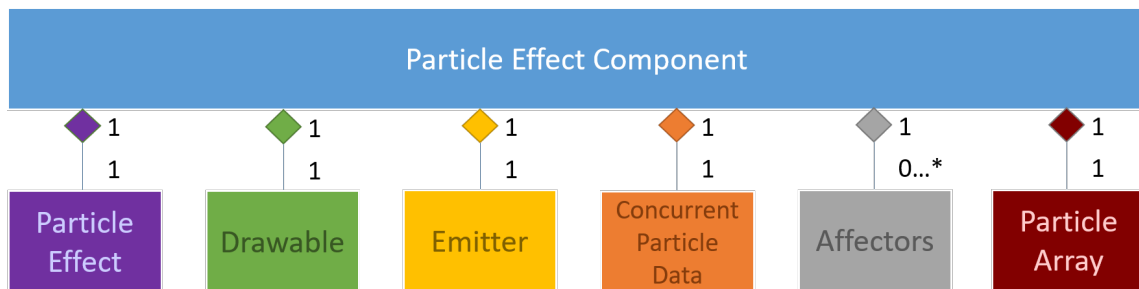


Figure 2.5: The basic structure of a ChilliSource Particle Effect Component

2.3.2 Architecture

2.3.2.1 Overview

The structure of a *Particle Effect Component* in ChilliSource is fairly complex. Its composition includes a number of objects, primitives, and data structures, but we can boil it down to 6 parts:

- A *Particle Effect* object
- A *Particle Drawable* object
- An *Particle Emitter* object
- A list of *Particle Affectors* (optional)
- A *Concurrent Particle Data* object
- An array of *Particle* structs

2.3.2.2 Particle Effect

The *Particle Effect* object contains all of the *Properties* and *Definitions* for the effect. A *Property* defines a value that may be more complex than just a single value. For example, a *Property* could be defined as a random value selected within a certain range, it may be a value that changes over the lifetime of the particle effect, or it could simply be a constant primitive value. *Definitions*, on the other hand, describe a number of properties that should be used to create an object. The *Properties* and *Definitions* that the *Particle Effect* contains

include:

- The particle effect's duration
- The total maximum number of particles used in the effect
- The space in which the particle effect is simulated (either "local" or "world")
- The lifetime property of a new particle generated in the effect
- The initial scale property of a new particle generated in the effect
- The initial rotation property of a new particle generated in the effect
- The initial color property of a new particle generated in the effect
- The initial speed property of a new particle generated in the effect
- The initial angular velocity property of a new particle generated in the effect
- The particle effect's drawable definition
- The particle effect's emitter definition
- The particle effect's affector definitions

These *Properties* and *Definitions* are used to create instances and values that will be used by the *Particle Effect Component*. Thus, the contents of the *Particle Effect* instance completely determines the behavior and appearance of the *Particle Effect Component*. In order to create these intricate *Particle Effects*, we must utilize a JSON particle definition file like the one shown in Code Listing 1. Given the length and verbosity of this file, it is easy to see that defining a particle effect by hand in C++ would quickly become messy and difficult to maintain.

Code Listing 1: An example ChilliSource particle definition file for a generic particle effect. Note how the initial and changing-over-time (i.e. "affector") *Properties* can either be random values or constants.

```

1  {
2      "Duration": "1.0",
3      "MaxParticles": "10000",
4      "SimulationSpace": "World",
5      "LifetimeProperty": {
6          "Type": "RandomConstant",
7          "LowerValue": "0.8",

```

```

8     "UpperValue": "1.2"
9 },
10 "InitialScaleProperty": {
11     "Type": "RandomConstant",
12     "LowerValue": "0.5 0.5",
13     "UpperValue": "1.2 1.2"
14 },
15 "InitialRotationProperty": "0.0",
16 "InitialColourProperty": "0.0 0.0 0.0 0.0",
17 "InitialSpeedProperty": "0.5",
18 "InitialAngularVelocityProperty": "0.0",
19 "Drawable": {
20     "Type": "StaticBillboard",
21     "MaterialPath": "Particle/Particle.csmaterial",
22     "AtlasPath": "TextureAtlases/Particle/Particle.csatlas",
23     "ImageIds": "Generic",
24     "Size": "1.0 1.0",
25     "SizePolicy": "FitMaintainingAspect"
26 },
27 "Emitter": {
28     "Type": "Cone",
29     "EmissionMode": "Stream",
30     "EmitFromType": "Inside",
31     "EmitDirectionType": "AwayFromBase",
32     "EmissionRateProperty": "100.0f",
33     "ParticlesPerEmissionProperty": "10000",
34     "EmissionChanceProperty": "1.0",
35     "RadiusProperty": "0.5",
36     "AngleProperty": "0.8"
37 }
38 "Affectors": [
39     {
40         "Type": "ColourOverLifetime",
41         "TargetColourProperty": "0.0 0.0 0.0 0.0",
42         "IntermediateColours": [
43             {
44                 "ColourProperty": {
45                     "Type": "RandomConstant",
46                     "LowerValue": "0.9 0.4 0.12 0.0",
47                     "UpperValue": "1.0 0.25 0.0 0.5"
48                 },
49                 "TimeProperty": "0.15"
50             },
51             {
52                 "ColourProperty": {
53                     "Type": "RandomConstant",
54                     "LowerValue": "0.9 0.4 0.12 0.0",
55                     "UpperValue": "1.0 0.25 0.0 0.5"
56                 },
57                 "TimeProperty": "0.75"
58             }
59         ]
60     },
61     {
62         "Type": "ScaleOverLifetime",

```

```

63     "ScaleProperty": {
64         "Type": "RandomConstant",
65         "LowerValue": "0.75 1.5",
66         "UpperValue": "1.0 2.0"
67     }
68 }
69 ]
70 }

```

2.3.2.3 Particle Drawable

The *Drawable* object is responsible for rendering all of the particles that belong to its *Particle Effect Component*. A *Static Billboard Particle Drawable*², which is a *Drawable* that renders each and every individual particle as camera facing sprites, is used to accomplish this. The *Drawable* will always render particles on the main game thread.

2.3.2.4 Particle Emitter

The *Emitter* object is used to spawn new particles during **Emission**. The *Emitter* gets to decide the direction, location, and moment that a new particle emits. Unlike the rendering that a *Drawable* does, the *Emitter* emits particles as a background task. There are 5 different kinds of *Emitters*, and their differences lie in the "generation", or emission, shape:

- *Point Particle Emitter*
 - This emitter spawns particles at the parent entity's position with a random direction.
- *Circle Particle Emitter*
 - This emitter spawns particles either within a circle or on the circle's perimeter. It can be set to emit particles with a random direction or a direction away from the center of the circle.

²As of version 1.6.0, the only *Drawable* type available to us is the *Static Billboard*, but the current architecture suggests that there could be different types of *Drawables* in future versions.

- *Sphere Particle Emitter*
 - Similar to the *Circle Particle Emitter*, this emitter spawns particles either within a sphere or on the sphere’s surface. It can be set to emit particles with a random direction or a direction away from the center of the sphere.
- *Cone Particle Emitter*
 - This emitter spawns particles either within a cone or on the cone’s surface. It can be set to emit particles with a random direction or a direction away from the base of the cone.
- *Cone 2D Particle Emitter*
 - This emitter spawns particles either within a 2D cone, on the 2D cone’s edges, or at the 2D cone’s base. It can be set to emit particles with a random direction or a direction away from the base of the 2D cone.

2.3.2.5 Particle Affector

A *Particle Affector* object applies an effect to all particles that change over time. A *Particle Effect Component* can utilize zero or more *Affectors*, and, like the *Emitter*, *Affectors* update particles in a background thread. There are 4 different kinds of *Affectors* that can be applied:

- *Scale Over Lifetime Particle Affector*
 - This affector changes the size of the particles over their lifetime by using a calculated scale factor. This factor will be used to help interpolate the particle’s size across updates.
- *Color Over Lifetime Particle Affector*
 - This affector changes the color of particles over their lifetime by using its initial and target colors to generate interpolated colors across updates.
- *Acceleration Lifetime Particle Affector*
 - This affector changes the speed of particles over their lifetime.

- *Angular Acceleration Lifetime Particle Affector*
 - This affector changes the angular velocity of particles over their lifetime.

2.3.2.6 Particle Array

The *Particle Array* is a data structure that holds all of the particles that comprise the *Particle Effect Component*, and it is used by the *Drawable*, *Emitter*, *Affector*, and *Concurrent Particle Data* instances. Each particle is represented by a *Particle* struct, as shown in Code Listing 2. We can observe that the information provided by this struct mirrors the particle attributes that Reeves used in his particle systems.

Code Listing 2: The struct that is used to describe the properties of a particle within the *Particle Array* in *Particle Effect Component*.

```

1  struct Particle final
2  {
3      bool m_isActive = false;
4      f32 m_lifetime = 0.0f;
5      f32 m_energy = 0.0f;
6      Vector3 m_position;
7      Vector2 m_scale = Vector2::k_one;
8      f32 m_rotation = 0.0f;
9      Colour m_colour = Colour::k_white;
10     Vector3 m_velocity;
11     f32 m_angularVelocity = 0.0f;
12 };

```

2.3.2.7 Concurrent Particle Data

The *Concurrent Particle Data* object is a container for data that is shared by the background and main threads. *Concurrent Particle Data* is written to by the same background thread that emits and affects particles, and it is read by the *Particle Effect Component's Drawable* in the main thread. It contains, essentially, copies of important attributes that are shared by these threads. This includes an array of particles, indices of particles that were just emitted, and the bounds of the particle effect. Thus, all thread synchronization

and locking occurs in *Concurrent Particle Data* and not in the *Particle Effect Component* itself.

It is important to note that *Concurrent Particle Data*'s array of particles uses a different struct than the one used in *Particle Effect Component*. As we can see in Code Listing 3, the struct contains a subset of the information that is provided by *Particle Effect Component*'s *Particle* struct. This is intentional, as we only want to copy the information that will be used by the *Drawable* later.

Code Listing 3: The struct that is used to describe the properties of a particle within the *Particle Array* in *Concurrent Particle Data*.

```

1  struct Particle final
2  {
3      bool m_isActive = false;
4      Vector3 m_position;
5      Vector2 m_scale = Vector2::k_zero;
6      f32 m_rotation = 0.0f;
7      Colour m_colour = Colour::k_white;
8  };

```

2.3.2.8 Other Important Attributes

There are a number of other important attributes that are not the 6 parts discussed above. These include:

- Bounding Box Objects
- Playback Enumerators and Floats
- Custom Events

The *Bounding Box Objects* are containers that describe the effect's shape and are used to aide in the rendering of the effect. The *Playback Enumerators*, *Playback Type* and *Playback State*, help describe the whether or not the particle effect loops and the current life cycle phase of the particle effect, respectively. The *Playback Floats* keep track of how long the particle effect has been playing. The two *Custom Events* that the effect uses are fired off

when the particle effect fully completes (i.e. no more particles are left alive) and when a particle effect finishes emitting.

2.3.3 Life Cycle

2.3.3.1 Overview

The life cycle of a *Particle Effect Component* is very similar to the one that was developed by Reeves, and it can be described as 4 main phases:

1. Initialization
2. Update
3. Render
4. Clean Up

2.3.3.2 Initialization Phase

During the **Initialization** phase, the *Particle Effect Component* creates the *Drawable*, *Emitter*, *Concurrent Particle Data*, and, if specified by the *Particle Effect* instance, its *Affectors*. It also creates the *Particle Array* and initializes the *Bounding Shapes*.

2.3.3.3 Update Phase

The **Update** phase uses 4 states to characterize the current playback state of the particle effect, which are **Starting**, **Playing**, **Stopping**, and **Not Playing**. During the **Starting** state, particles are initialized to be inactive before it updates the particle effect for the first time and enters the **Playing** state. The **Playing** state is when particles are rendered and updated. If, during the **Playing** state, the particle effect may enter a **Stopping** state if the playback timer exceeds the effect's duration. However, it may go back to a **Playing** state if active particles still remain within the *Particle Array*. When the particle effect's duration time has elapsed and there are no more active particles, the particle effect finally enters the **Not Playing** state and the particle effect no longer does anything. The bulk of a particle

effect's time is spent during the **Playing** state, and that is the state of the **Update** phase that we will be focusing on throughout our study and will be discussing at length.

Code Listing 4: The struct that is used to bundle all information needed by the background task performing the particle update.

```

1  struct ParticleUpdateDesc final
2  {
3      ParticleEffectCSPtr m_particleEffect;
4      ParticleEmitterSPtr m_particleEmitter;
5      std::vector<ParticleAffectorSPtr> m_particleAffectors;
6      std::shared_ptr<dynamic_array<Particle>> m_particleArray;
7      ConcurrentParticleDataSPtr m_concurrentParticleData;
8      f32 m_playbackTime = 0.0f;
9      f32 m_deltaTime = 0.0f;
10     Vector3 m_entityPosition;
11     Vector3 m_entityScale;
12     Quaternion m_entityOrientation;
13     bool m_interpolateEmission = false;
14 };

```

When the particle effect is updating, it prepares a struct containing pertinent information to hand over to a background task. This struct contains pointers to the *Particle Effect*, *Emitter*, *Affector(s)*, *Particle Array*, *Concurrent Particle Data*. It additionally holds some other miscellaneous information, such as the effect's parent entity's position, scale, and orientation. The struct in its entirety is shown in Code Listing 4. After it prepares the struct, it uses the *Application Task Scheduler* application system to schedule a background task. This process is shown using pseudocode in lines 1 - 11 in Code Listing 5.

As mentioned before in Section 2.3.2, the **Particle Update Task** updates particles from the *Particle Array*, applies *Affectors* to the particles, emits particles using the *Emitter*, calculates its new *Bounding Shapes*, and it updates *Concurrent Particle Data* with the new information. Lines 13 - 41 demonstrate these actions in Code Listing 5.

Code Listing 5: Pseudocode describing how a *Particle Effect Component* is updated by scheduling a background task.

```

1 ParticleEffectComponent::OnUpdate()
2 {
3     // Get a struct containing copies (and pointers) to pass to the task
4     copiedAtts = CopyUpdateAttributes();
5
6     // Schedule a task to update the particle in the background
7     ApplicationTaskScheduler.ScheduleTask
8     ({
9         ParticleUpdateTask(copiedAtts);
10    });
11 }
12
13 ParticleUpdateTask(copiedAtts)
14 {
15     // Update particles if they are active
16     for(particle in copiedAtts.particleArray)
17     {
18         if(particle.isActive)
19             particle.UpdateValues();
20     }
21
22     // Apply affectors
23     for(affector in copiedAtts.particleAffectors)
24     {
25         affector.AffectParticles();
26     }
27
28     // Try to emit
29     newParticleIndicesArray = copiedAtts.particleEmitter.EmitParticles();
30
31     // Update bounding shapes
32     AABB, Sphere = CalculateBoundingShapes();
33
34     // Copy this updated data to the concurrent particle data object
35     copiedAtts.particleConcurrentParticleData.CommitParticleData
36         (particleArray,
37          newParticleIndicesArray,
38          AABB, Sphere);
39 }

```

Code Listing 6 illustrates how the function, **CommitParticleData**, copies data from the background task to the *Concurrent Particle Data* instance. The function first copies all of the particles in the *Particle Array*, and only a subset of information that is related to drawing particles is copied per particle. It then appends the indices of the particles that were just newly emitted to its member array, *newParticleIndices*. At the end, it copies the *Bounding Objects*. During all of this, the *Concurrent Particle Data* instance is locked by

locking a member variable mutex.

Code Listing 6: Pseudocode describing what **CommitParticleData** does when it copies its data to the *Concurrent Particle Data* instance.

```

1 ConcurrentParticleData::CommitParticleData(particleArray,
2                                           newParticleIndicesArray,
3                                           AABB, Sphere)
4 {
5     // Lock the mutex for the scope of the whole function
6     using lock(this.mutex)
7     {
8         // Copy over all particles to the member particle array
9         for(i = 0 to particleArray.size())
10        {
11            this.particleArray[i] = particleArray[i].copySubset();
12        }
13
14        // Append the new indices to the member new indices array
15        this.newParticleIndices.append( newParticleIndices.copy() );
16
17        // Copy the bounding information to the member variables
18        this.AABB = AABB.copy();
19        this.Sphere = Sphere.copy();
20    }
21 }

```

2.3.3.4 Render Phase

Unlike the **Update** phase, the **Render** phase is not directly called by the *Particle Effect Component*. Instead, it is called by the *Renderer* application system. The *Renderer* retrieves the active scene from the *State Manager*, filters entities added to the scene that have *Render Components*, and attempts to render them.

The **Render** function uses the *Drawable* instance to draw the *Particle Effect Component*. The *Drawable*, in turn, uses *Concurrent Particle Data* to read the information needed about the particle effect in order to render it. The **Draw** function within the *Drawable* first locks the *Concurrent Particle Data* instance using a **Lock** function, and then proceeds to work with the copied data in *Concurrent Particle Data*. It uses the *newParticleIndices* to register newly emitted particles to the *Static Billboard* that is used for rendering, and continues on

to iterate over the copied array of particles in order to draw each particle that is currently active. When it is finished, it unlocks the *Concurrent Particle Data* instance.

Code Listing 7: Pseudocode showing how a *Particle Effect Component* uses its *Drawable* to render itself.

```

1 ParticleEffectComponent::Render(camera)
2 {
3     // Use the Particle Drawable to draw particles to the screen
4     if(this.playbackState is playing or this.playbackState is stopping)
5         this.drawable->Draw(camera);
6 }
7
8 Drawable::Draw(camera)
9 {
10    // Lock the Concurrent Particle Data object
11    this.concurrentParticleData.Lock();
12
13    // Activate the newly emitted particles in the static billboard
14    for(particleIndex in this.concurrentParticleData.GetNewParticleIndices())
15    {
16        ActivateParticleInBillboard(particleIndex);
17    }
18
19    // Iterate through all of the particles and draw each one
20    DrawParticles(this.concurrentParticleData.GetParticleArray(), camera);
21
22    // Unlock the Concurrent particle Data object
23    this.concurrentParticleData.Unlock();
24 }
```

2.3.3.5 Clean Up Phase

Of all phases, this is one is by far the simplest. It releases all resources and clears allocated arrays. Specifically, it cleans up the *Particle Array*, the *Concurrent Particle Data* instance, the *Drawable* instance, the *Emitter* instance, and the array of *Affectors*.

2.3.4 Usage

Creating a basic scene that uses a *Particle Effect Component* in *ChilliSource* is fairly simple. As Code Listing 8 illustrates, it requires that we first create a camera along with a light source and add them both to the current *State's Scene*. Although a state system,

Basic Entity Factory, abstracts some of this code, Code Listing 9³ demonstrates that utilizing ChilliSource’s API to create an entity like an ambient light is still easy to use and understand. We did not need to do this, but we also used the *Basic Entity Factory* to create a room in our scene.

Code Listing 8: Creating a scene that adds a basic *Particle Effect Component* in ChilliSource.

```

1  void State::OnInit()
2  {
3      // Retrieve the resource pool application system to create a particle effect
4      // Set the state's current scene's clear color to black
5      GetScene()->SetClearColour(CS::Colour::k_black);
6
7      // Retrieve a state system that will help create basic entities like
8      // cameras, rooms, and lights.
9      auto basicEntityFactory = CS::Application::Get()
10         ->GetSystem<Common::BasicEntityFactory>();
11
12     // Create a basic room and add it to the scene
13     CS::EntitySPtr room = basicEntityFactory->CreateRoom();
14     room->GetTransform().SetPosition(0.0f, 10.0f, 0.0f);
15     GetScene()->Add(room);
16
17     // Create a third person camera and add it to the scene
18     auto camera = basicEntityFactory
19         ->CreateThirdPersonCamera(room, CS::Vector3(0.0f, -9.0f, 0.0f));
20     GetScene()->Add(std::move(camera));
21
22     // Create an ambient light and add it to the scene
23     CS::EntitySPtr ambientLight = basicEntityFactory
24         ->CreateAmbientLight
25         (CS::Colour(0.65f, 0.65f, 0.65f, 1.0f));
26     GetScene()->Add(ambientLight);
27
28     // Create a particle effect entity and add it to the scene
29     auto particleEntity = CreateParticleEffect();
30     GetScene()->Add(std::move(particleEntity));
31 }

```

Code Listing 9: The function to create an ambient light with the *Basic Entity Factory* state system.

³This is the only time within this paper that we will not use pseudocode. Since we are demonstrating how to use and create a *Particle Effect Component*, it seemed appropriate to show the code verbatim.

```

1 CS::EntityUPtr BasicEntityFactory::CreateAmbientLight
2                                     (const CS::Colour& in_colour)
3 {
4     // Use the application system, Render Component Factory, to create an
5     // ambient light.
6     auto ambientLightComponent = m_renderComponentFactory
7                                     ->CreateAmbientLightComponent(in_colour);
8
9     // Create an entity, and attach the ambient light component to it.
10    auto entity = CS::Entity::Create();
11    entity->AddComponent(ambientLightComponent);
12
13    // Return the entity
14    return entity;
15 }

```

Once we've added a room, lighting, and a camera to our scene, we create a particle effect entity using a function called **CreateParticleEffect**. The exact code inside of this function is shown in Code Listing 10. Inside of this function, we use the main *Application* instance to generate a *Particle Effect* from a *csparticle* file, and it accomplishes this by using a *Resource Pool* application system. Once we have a *Particle Effect*, we can use a *Render Component Factory* application system to create a *Particle Effect Component* using that *Particle Effect*. Lastly, we must add the *Particle Effect Component* to an entity and add that entity to the current state's scene.

Code Listing 10: Creating a basic particle effect component with ChilliSource.

```

1 CS::EntityUPtr State::CreateParticleEffect()
2 {
3     // Retrieve the singleton application instance through a static method
4     auto app = CS::Application::Get();
5
6     // Retrieve the resource pool application system to create a particle effect
7     // from a JSON csparticle file
8     auto resourcePool = app->GetResourcePool();
9     auto particleEffect = resourcePool
10                             ->LoadResource<CS::ParticleEffect>
11                             (CS::StorageLocation::k_package,
12                             "Particle/generic.csparticle");
13
14     // Using a render component factory application system and the particle
15     // effect, create a particle effect component
16     auto renderComponentFactory = app->GetSystem<CS::RenderComponentFactory>();

```

```

17     auto particleComponent = renderComponentFactory
18         ->CreateParticleEffectComponent(particleEffect);
19
20     // Set the particle component's playback to looping
21     particleComponent
22         ->SetPlaybackType(CS::ParticleEffectComponent::PlaybackType::k_looping);
23
24     // Create an entity, and add the particle component to the entity
25     auto particleEntity = CS::Entity::Create();
26     particleEntity->AddComponent(std::move(particleComponent));
27
28     return particleEntity;
29 }

```

Compared to Reeves' particle systems, we can observe here that a *Particle Effect Component* is not composed of a hierarchy of child *Particle Effect Components*. As mentioned in Section 2.2.2, however, *Entities* are made up of *Components* and child *Entities*. Thus, ChilliSource's architecture allows the developer to create parent and child *Particle Effect Components* by nesting *Entities*.

CHAPTER 3 THE CHILLISOURCE AUTOMATED PONG GAME

3.1 Overview

The ChilliSource Automated Pong Game [4] (i.e. CSAPong) is the application that we used to aid in our study of ChilliSource’s *Particle Effect Components*. CSAPong is a derivative of CSPong [5], a simple pong game created with ChilliSource that was developed by the ChilliWorks team. CSPong was built to illustrate as much of the engine as possible without creating a complex game, and the three main states attempt to reflect this goal. These states include the *Splash State*, the *Main Menu State*, and the *Game State*. These states help show the engine’s life cycle events, how to use the UI app system, how to make custom state systems, how to utilize custom events, how to use the *State Manager*, and, most importantly, how you can design entities and components to add enemy AI, player interaction, physics, collision detection, and so on. All three of these states are shown in Figure 3.1.

Unlike CSPong, CSAPong only uses a single state, the *Game State*. The main application instance restarts the *Game State* a number of times, and it automatically quits the application when it’s finished. CSAPong does not have enemy AI or player interaction (i.e. enemy and player paddles), but the left and right walls do collide with the wall and keep track of the ”score”. Additionally, CSAPong attaches a *Particle Effect Component* to the ball that will emit particles as the ball moves around the arena. Figure 3.2 shows a simple example of CSAPong in action.

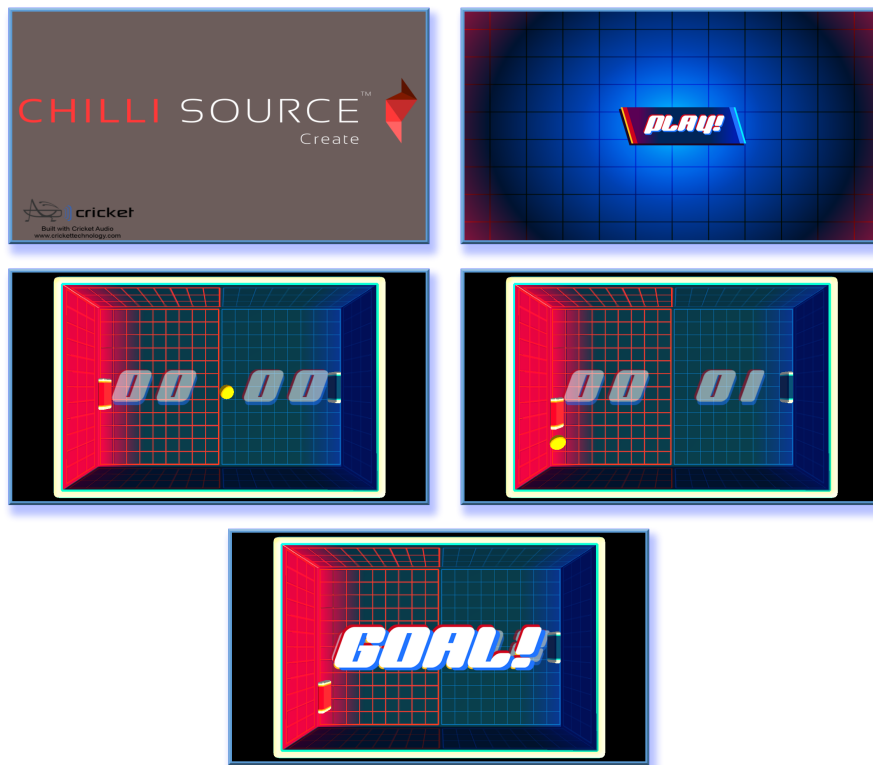


Figure 3.1: The above screenshots illustrate the original pong game. The screenshots were taken from the beginning of the application to the first goal. The top-left screenshot is from the *Splash State*, the top-right screenshot is from the *Main Menu State*, and the other three screenshots are from the *Game State*.

3.2 Architecture

3.2.1 Overview

Although it is not necessary to go over every single class that makes up CSAPong, it is essential to understand the general architecture of it. As shown in Figure 3.3, we can broadly describe the structure having two main parts, the ChilliSource engine¹ and the CSAPong application itself. Further, CSAPong can be defined as having *Application Systems*, a *Game State*, and the *Game State's State Systems*.

¹Obviously, we know that the entirety of the ChilliSource engine is part of the CSAPong application, but we will include it in our discussion of CSAPong's architecture due to the custom application systems that we created within ChilliSource specifically for CSAPong.

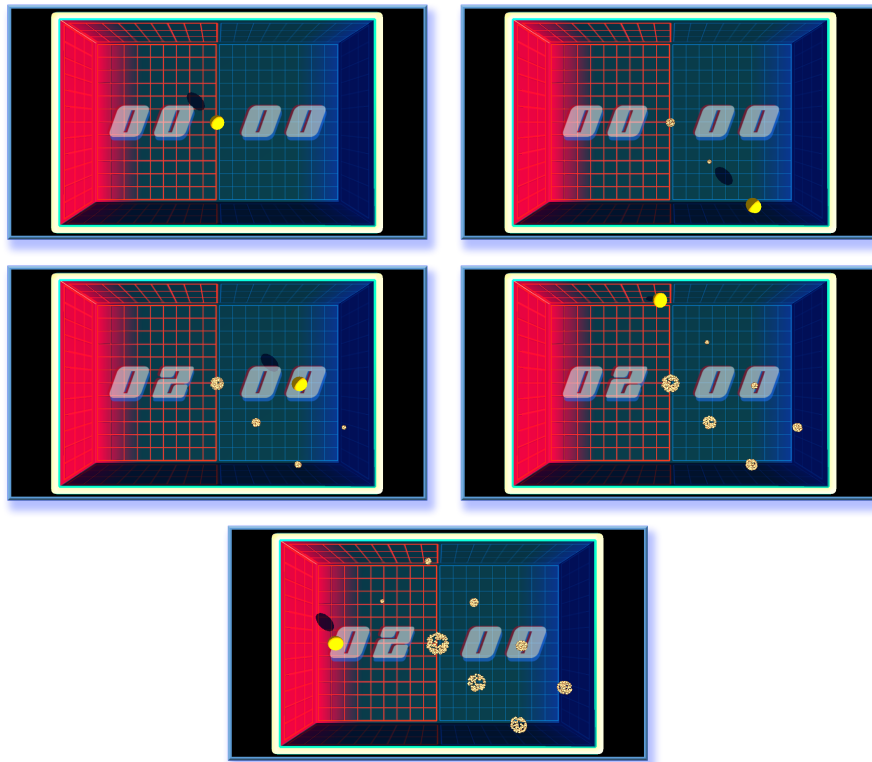


Figure 3.2: The above screenshots illustrate the automated pong game. The screenshots were taken 1 second apart with the ball starting at the beginning of the arena. Due to a small bug, note that the score increments by 2 when the ball collides with the opposing "team's" wall.

3.2.2 CSAPong Application

The *CSAPong Application* instance creates all application systems that will have a lifetime that lasts throughout the whole application. This includes *Particle Effect Component Factory* and *Profiling* application systems, where the former is defined within CSAPong and the latter is defined within ChilliSource itself. It also handles pushing the initial *Game State*, restarting the *Game State*, and quitting the application.

3.2.3 Application Systems

Although not explicitly part of CSAPong, the *Profiling Application Systems* are integral to our studies. They are custom-made application systems that help keep track of engine-

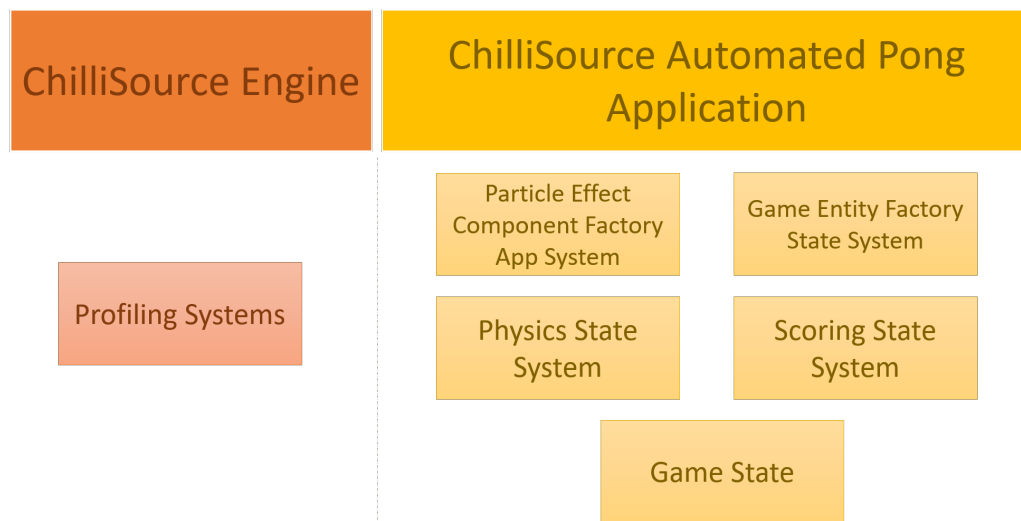


Figure 3.3: The basic architecture of the automated pong game.

specific metrics such as particles per emission, particles actually emitted, particles actually rendered, time spent in **ParticleUpdateTask**, time spent in **CommitParticleData**, and so forth. The *Profiling Application Systems* will be discussed more in detail in Chapter 4.

The *Particle Effect Component Factory Application System* generate *Particle Effect Components* and attach them to entities. It allows the user to hand it an entity, a csparticle file path, the number of desired *Particle Effect Component(s)* to add, and it will add the desired *Particle Effect Component(s)* to the entity. This application system is used to generate different kinds of particle effects as CSAPong runs.

3.2.4 Game State

The *Game State* manages the game’s life cycle, its entities and components, and its user interface. Similar to the main application instance, the *Game State* creates all state systems that will have the same lifetime as the state. It uses the *Game Entity Factory State System* on initialization to create a camera, some lighting, the arena, and the ball. The *Game Entity Factory*, in turn, uses the *Particle Effect Component Factory* to attach particle effects to the ball during that ball’s creation. It also signals the *Profiling Application Systems* to start

gathering metrics once the *Game State* is fully initialized and has started the game.

3.2.5 State Systems

The *Physics State System* handles the movement of dynamic bodies (such as the ball) and collision checking. Being that ChilliSource is a composition-based system, this behavior is described with components that are attached to entities. Thus, if we want the ball to be able to move, we attach a special type of component to the ball, a *Dynamic Body Component*, that is used by the *Physics State System*. Additionally, if we want collisions to occur between dynamic bodies and the walls, we must add static body components to the four wall entities within the arena.

The *Scoring State System* uses the collision detection in the *Physics State System* to trigger points being added to each "opposing team". As we can see in Figure 3.3, however, there is a small bug in which 2 points are awarded each time the ball collides with the wall. The bug did not interfere with our study of the *Particle Effect Component*, and so time was not taken to fix it.

3.3 Gameplay

3.3.1 Logistics

The gameplay can be described as a series of "games" where the ball appears begins at the center of the arena, accelerates toward the bottom right corner, bounces to the right-most wall, bounces to the center of the top wall, and heads toward the center of the left-most wall. Although the game can continue for longer than this, the majority of our studies set the "game length", or *run time*, to 5 seconds, and so this is the behavior that we saw the most often. This ball movement was specifically chosen to minimize the number of particles that would overlap each other during a game so we could challenge the system to render as many particles as it could. If the particles did overlap, then those particles would be culled, or not drawn, and thus the engine would not be working as hard.

Another variable that we can change is the number of "games", or the *maximum run number*, that will be "played". The game has the ability to change particle effect types during execution, and so the maximum run number is the number of games that are ran *for each* particle effect type that we want to use. For example, if we had 3 different particle effect types and the maximum run number was set to 3, then we would expect to see 9 games play in total.

When the "game" is over, the *CSAPong Application* instance destroys the current *Game State* and creates a new *Game State* with the same kind of particle effect type until the maximum run number is reached. The application will then either create a new *Game State* with the next particle effect type or it will quit the application if there is not another particle effect type to "play". A series of files within a timestamped folder will be placed in the application's save data once the application has finally quit. These files contain the instrumentation information, and we will learn more about them in Chapter 4.

3.3.2 Examples

Particle effects, as we found in the previous chapter, are considerably complicated and have many variables that can be changed. Although we changed many of these variables during our studies, we can generalize the different scenarios we encountered as the following:

- No particles emitting during the game
- A single, giant burst of particles emitting during the game
- A steady stream of particle emissions during the game

The following figures show screenshots of these three different scenarios.

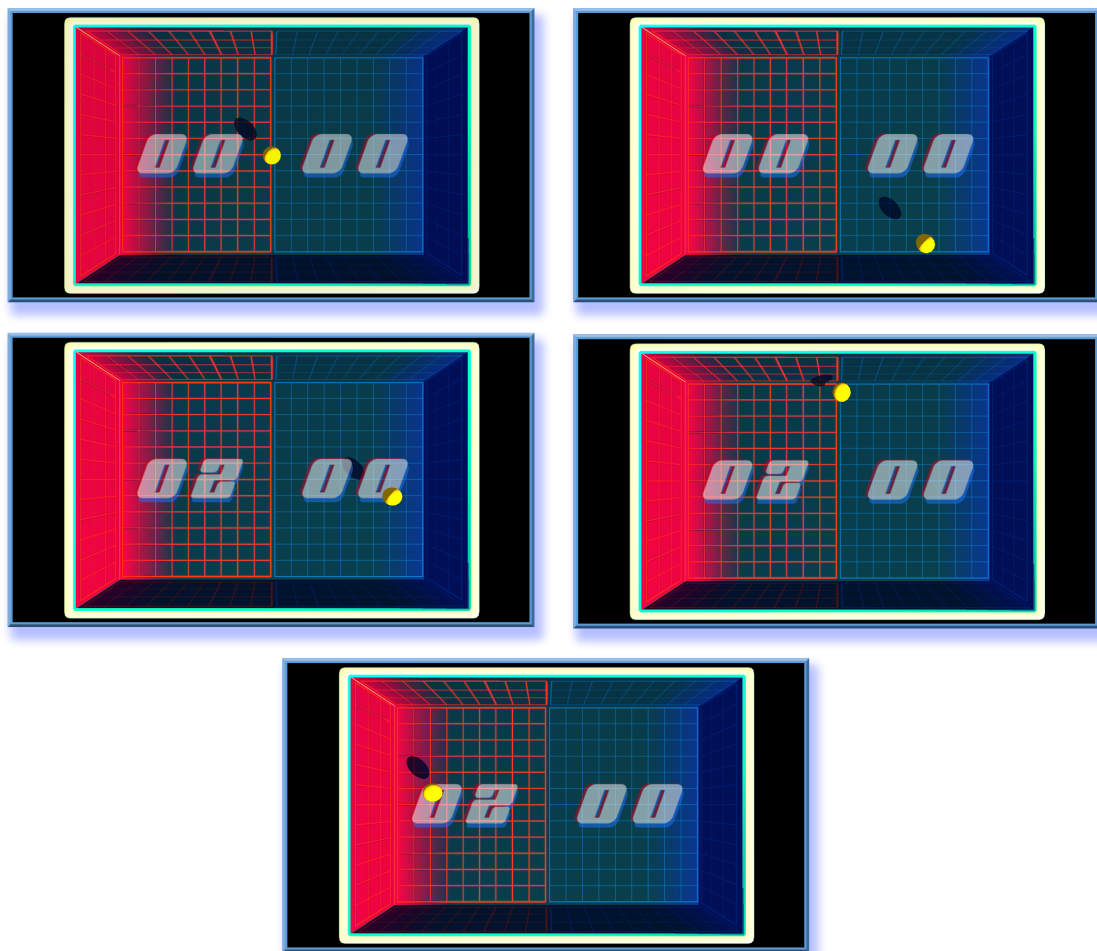


Figure 3.4: A series of screenshots showing the automated game with no particles. The screenshots were taken 1 second apart with the ball starting at the beginning of the arena.

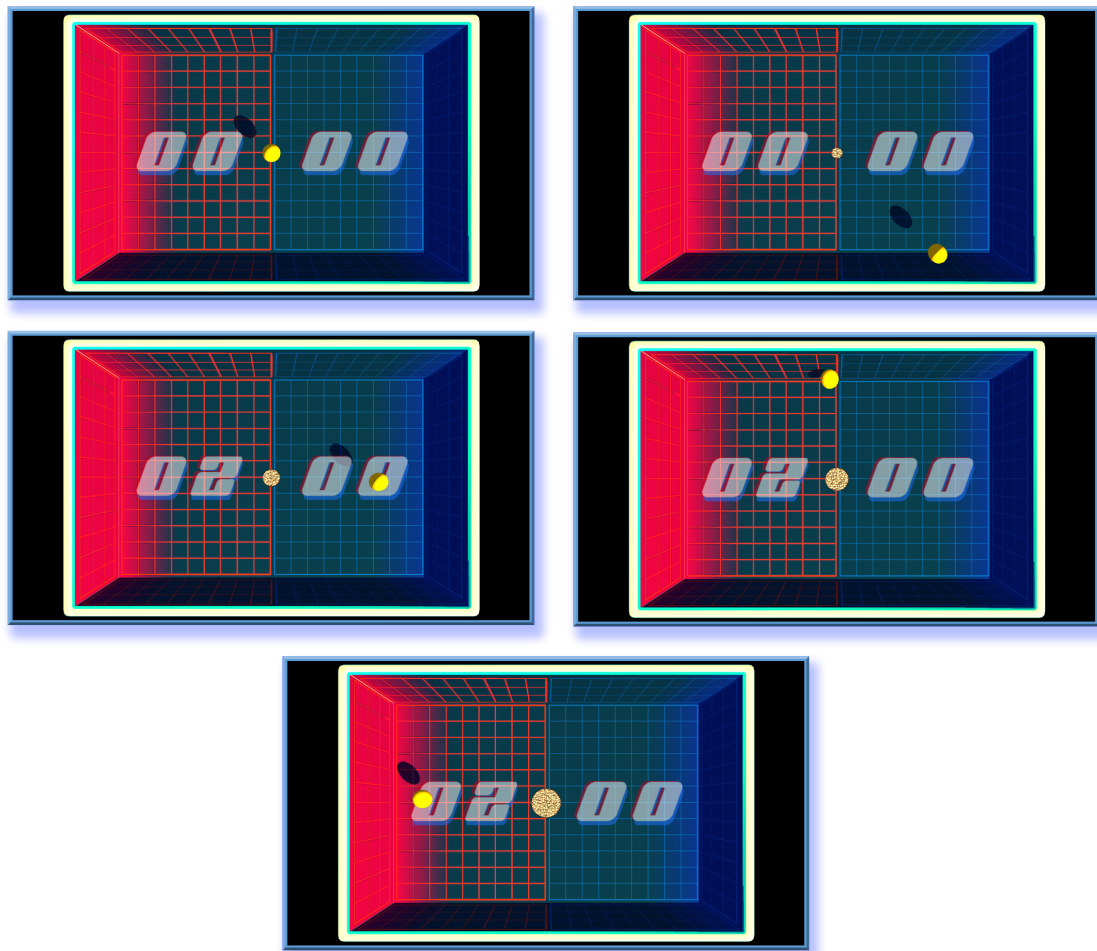


Figure 3.5: A series of screenshots showing the automated game with a single, giant burst of particles. The screenshots were taken 1 second apart with the ball starting at the beginning of the arena.

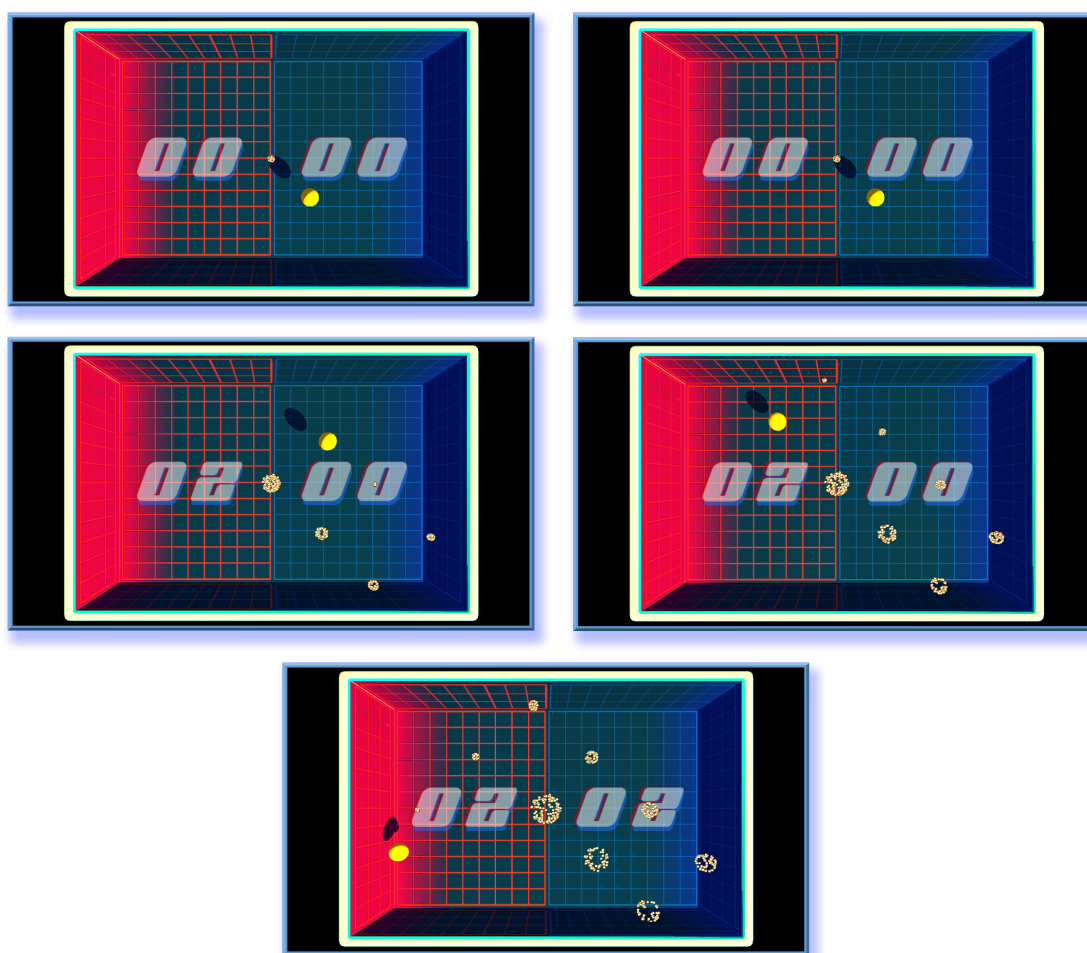


Figure 3.6: A series of screenshots showing the automated game with a steady stream of particle emissions. The screenshots were taken 1 second apart with the ball starting at the beginning of the arena.

CHAPTER 4 INSTRUMENTATION

4.1 Overview

In order to numerically quantify what occurs during games that are ran by CSAPong, we instrumented different parts of the engine. We examined various metrics (e.g. particles actually emitted), and we also timed how long various sections of code took to run. These two use cases were separated into two application systems that resided within the engine itself, the *Metrics* and *Timing* systems. As mentioned before, these systems will save a series of files containing the gathered information that will go into a timestamped folder within CSAPong’s save data file system. These files fueled the direction of our studies due to their ability to answer questions about the engine such as ”What function in *Particle Effect Component* takes the longest to run?””

4.2 Metrics System

4.2.1 Metadata

Although not structurally complicated, the *Metrics System* keeps track of a great deal of primitives that describe the series of games ran by CSAPong. The following is a list of the tracked primitives that remained constant throughout the games:

- ***Are particles looping***
 - Whether or not the particle effect component is looping. This means that the particle effect will continue even after the duration has elapsed.
- ***Number of particle effects***
 - The number of particle effect components that were attached to the ball entity.

attribute	value
are particles looping	TRUE
num particle effects	1
min particles	500
max particles	2000
particles step	500
is total max particles changing	TRUE
is particles per emission changing	TRUE
particles per emission	0
total maximum particles	0
particles per emission step	50
total maximum particles step	500
total num runs	2
duration per run	5

Figure 4.1 An example of metadata output by the *Metrics System*.

CSAPong Game Num	Particles Per Emission	Total Maximum Particles	Particles
1	50	500	500
2	100	1000	1000
3	150	1500	1500
4	200	2000	2000

Figure 4.2 An example of the changing particle values during games in CSAPong.

- ***Is total maximum particles changing***
 - Whether or not *total maximum particles* is changing across different particle effect types.
- ***Total maximum particles***
 - If *is total maximum particles changing* is false, then this will be the constant value of *total maximum particles* across particle effect types.
- ***Is particles per emission changing***
 - Whether or not *particles per emission* is changing across different particle effect types.
- ***Particles per emission***
 - If *is particles per emission changing* is false, then this will be the constant value of *particles per emission* across particle effect types.
- ***Minimum particles***
 - The minimum value for the changing values (i.e. *total maximum particles* and/or *particles per emission*).
- ***Maximum particles***
 - The maximum value for the changing values (i.e. *total maximum particles* and/or *particles per emission*).
- ***Particles step***

- The step value for the changing values (i.e. *total maximum particles* and/or *particles per emission*). This is used in conjunction with the minimum and maximum particles values to determine how many particle effects to create and use during CSAPong’s execution.
- ***Particles per emission step***
 - If *is particles per emission changing* is true, then this will dictate the step value for *particles per emission* across particle effect types. This is generally a fraction of *particles step*.
- ***Total maximum particles step***
 - If *is total maximum particles changing* is true, then this will dictate the step value for *total maximum particles* across particle effect types. This is generally a fraction of *particles step*.
- ***Total number of runs***
 - The number of games to run per particle effect type.
- ***Duration per run***
 - The amount of time in seconds that a single game lasts.

Let us examine Figure 4.1 in order to make the distinction clear between the different kinds of particle variables. The *minimum particles* is 500, the *maximum particles* is 2000, and the *particles step* is 500. This tells us right away that there will be $2000/500 = 4$ kinds of particle effects that will be created and played¹. This will be true regardless of the other variables’ values. From the figure, we also know that *particles per emission* and *total maximum particles* are changing across particle effects and they will not be constant values. Thus, their ”step” values will tell us how much each variable will be added to as we advance through the 4 different kinds of particle effects. We can see that the *total maximum particles step* is 500, or 100% of the *particles step*. Additionally, we can see that the *particles per emission step* is 50, or 10% of the *particles step*. Figure 4.2 shows how all of these particle variables change as CSAPong executes. We should note that, since we have 4 different kinds of particle effects and 2 *total number of runs*, CSAPong would play a total of $4 \times 2 = 8$ times in this example.

¹Note that we do not subtract 500 from 2000 since the range is inclusive.

4.2.2 Metrics

The *Metrics System* keeps track of three *engine metrics*, but it also manages six *system metrics*. Engine metrics are metrics that originate from the engine, e.g. counting the number of times that each particle was drawn. System metrics are metrics that come from the metrics system, e.g. the current game run number. In other words, system metrics come from metadata values, but they are printed with the engine metrics in order to give them context. The following list shows all of the different engine and system metrics that are tracked:

- *Engine Metrics*
 - *All effects particles actually emitted*
 - * This retrieves the size of the *New Particle Indices* array after each emission and adds it to this metric.
 - *All effects particles actually rendered*
 - * This is incremented every time a particle is rendered in the *Drawable* instance.
 - *Render ball called*
 - * This is incremented every time the ball is rendered.
- *System Metrics*
 - *Run number*
 - *Per effect total max particles*
 - *Per effect particles per emission*

run num	per effect total max particles	per effect particles per emission	all effects total max particles	all effects particles per emission	all effects particles actually emitted	all effects particles actually rendered	render ball called
0	500	50	500	50	500	49250	340
1	500	50	500	50	500	34250	302
0	1000	100	1000	100	1000	69400	300
1	1000	100	1000	100	1000	68500	302
0	1500	150	1500	150	1500	102300	300
1	1500	150	1500	150	1500	101700	302
0	2000	200	2000	200	2000	135600	300
1	2000	200	2000	200	2000	135000	302

Figure 4.3: An example of metrics output by the *Metrics System* based on the metadata from Figure 4.1.

- *All effects total max particles*
- *All effects particles per emission*

4.2.3 Particle Effect Definition

As we mentioned in Chapter 2, *Particle Effects* that are passed into *Particle Effect Components* cannot be created programmatically. They must be created with JSON particle definition files. If we wanted to create tens or hundreds of particle effects to test in CSAPong, then we would have to make just as many particle definition files. This is clearly something that we do not want to do by hand, and so a script, **generate_particles.py**, was created to automate this process. An example invocation of this script is shown in Code Listing 11, and that invocation will generate the particle definition files shown in Figure 4.5. Note that these generated particle effects are the same ones that were used in Figure 4.1.

```
$ python generate_particles.py -h
usage: generate_particles.py [-h] -changing {PPE,TMP,Both} [-min MIN]
                             [-max MAX] [-step STEP] [-constant CONSTANT]
                             [-tmpStep TMPSTEP] [-ppeStep PPESTEP] [-dir DIR]

Generate Particle JSON Files

optional arguments:
  -h, --help            show this help message and exit
  -changing {PPE,TMP,Both}
                        Either the particles per emission (PPE) or total max
                        particles (TMP) will be changing from min to max.
  -min MIN              The minimum particles- either PPE or TMP. (non-
                        negative number, default: 0)
  -max MAX              The maximum particles- either PPE or TMP. (non-
                        negative number, default: 1000)
  -step STEP            The step between min and max particles- either PPE or
                        TMP. (default: 100)
  -constant CONSTANT    The constant number of particles, either PPE or TMP
                        but it will be opposite of arg "changing". If both are
                        changing, then this has no effect. (default: 10000)
  -tmpStep TMPSTEP      The percent that TMP will step by if it is changing.
                        (default: 1.0)
  -ppeStep PPESTEP      The percent that PPE will step by if it is changing.
                        (default: 1.0)
  -dir DIR              The output directory name (default is "Generated" and
                        it will create it for you)
```

Figure 4.4: Shows the parameters that the **generate_particles.py** script can use.

Code Listing 11: An example invocation of the generate particles script.

```

1 | python generate_particles.py
2 | > -changing Both
3 | > -min 500 -max 2000 -step 500
4 | > -tmpStep 1.0 -ppeStep 0.1;

```

Name	Date modified	Type	Size
 500_particles.csparticle	11/2/2016 9:39 PM	CSPARTICLE File	2 KB
 1000_particles.csparticle	11/2/2016 9:39 PM	CSPARTICLE File	2 KB
 1500_particles.csparticle	11/2/2016 9:39 PM	CSPARTICLE File	2 KB
 2000_particles.csparticle	11/2/2016 9:39 PM	CSPARTICLE File	2 KB

Figure 4.5: Shows the particle files that were generated by an invocation like in Code Listing 11.

4.3 Timing System

4.3.1 Visual Studio

ChilliSource generates a Visual Studio solution file for the developer to use when we create a project, and so we utilized Visual Studio as an IDE during development on a Windows machine. Visual Studio boasts a wide array of profiling tools which include (but is certainly not limited to) CPU sampling and function instrumentation. The CPU sampling was considerably helpful during the initial stages of our studies. The inclusive (includes samples from all functions calls within it) and exclusive (only includes samples from itself) sampling pointed us in the direction of hot spots within the *Particle Effect Component* to investigate. The call trees that the sampling produced (see figures Figure 4.6 and Figure 4.7) also solidified our understanding on how the engine worked. Although this information is helpful, the CPU sampling does not provide concrete times. The function instrumentation that Visual Studio provides should have filled that gap, but we did not use it simply because we could not get it to reliably and smoothly work. It would slow down the game considerably during execution, and then would take a great deal of time to process after execution. Even

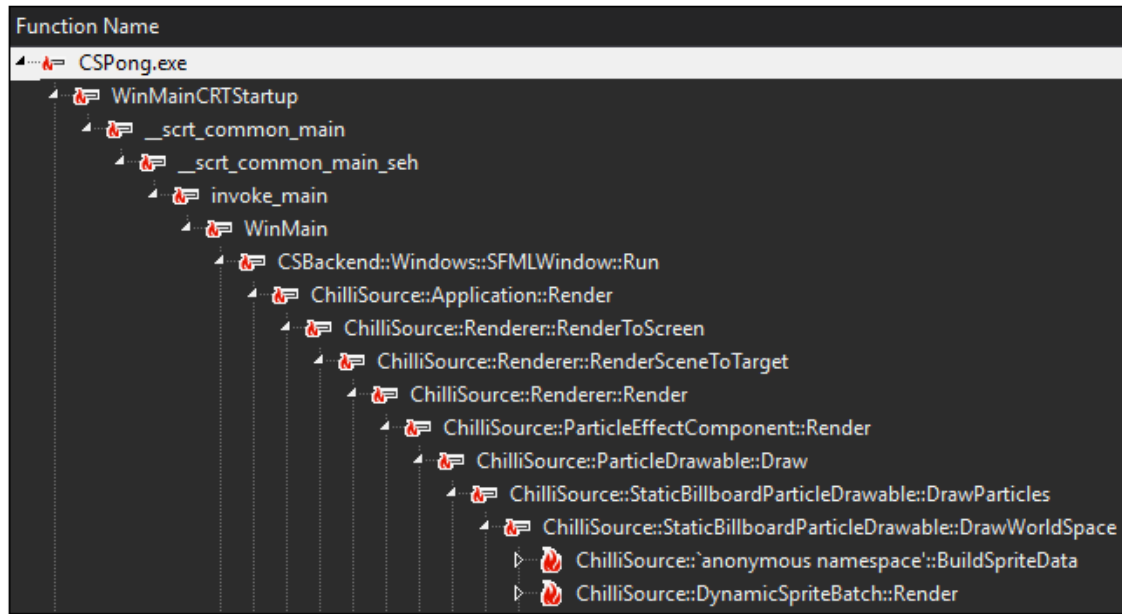


Figure 4.6 Shows an example of Visual Studio's generated call tree for CPU instrumentation when ran using the metadata from Figure 4.1.

Level	Function Name	Inclusive Samples %	Exclusive Samples %
8	ChilliSource::Renderer::RenderSceneToTarget	65.38	0.01
9	ChilliSource::Renderer::Render	45.64	0
10	ChilliSource::ParticleEffectComponent::Render	29.28	0
11	ChilliSource::ParticleDrawable::Draw	29.26	0
12	ChilliSource::StaticBillboardParticleDrawable::DrawParticles	27.42	0
13	ChilliSource::StaticBillboardParticleDrawable::DrawWorldSpace	27.35	0.11
12	ChilliSource::TaskPool::ProcessTasks	19.32	0
13	ChilliSource::TaskPool::PerformTask	19.32	0
20	ChilliSource::`anonymous namespace`::ParticleUpdateTask	19.04	0.12
9	ChilliSource::Renderer::RenderShadowMap	13.02	0
10	ChilliSource::Renderer::RenderShadowMap	12.87	0.02
14	ChilliSource::`anonymous namespace`::BuildSpriteData	11.4	0.29
14	ChilliSource::DynamicSpriteBatch::Render	10.63	0.09
10	ChilliSource::StaticMeshComponent::Render	7.12	0
11	ChilliSource::Mesh::Render	7.11	0
21	ChilliSource::ColourOverLifetimeParticleAffector::AffectParticles	6.8	0.18
12	ChilliSource::SubMesh::Render	6.8	0

Figure 4.7 Shows the same call tree from Figure 4.6, but sorted and filtered. The level is ≥ 8 , the function name begins with "ChilliSource::", the inclusive samples % is $\geq 6\%$, and it is sorted by the inclusive samples % in descending order. These parameters were used to ensure that the results were low-level ChilliSource function calls with a high inclusive sample percent.

if it did work, it did not have the flexibility to instrument custom sections, and it also could not be used to instrument the game on other platforms such as iOS or Android.

4.3.2 Shiny

Seeking a way to instrument functions and custom blocks of code across all platforms, a third party library called Shiny was used. Shiny is an older C/C++/Lua intrusive profiler created by Aidin Abedi with "very very low overhead" [6]. It allows us to insert macros within functions and code blocks, and it outputs a call tree very similar to Visual Studio's with how much time the named code blocks and function calls took. Although Shiny's creator asserted that Shiny is "amazingly simple to use and flexible", it took about a week to integrate Shiny within CSAPong and ChilliSource in such a way that it worked on Windows, iOS, and Android. The majority of the encountered problems had more to do with compilation errors and flags, however, and the actual API of Shiny is simple and flexible, as shown in Code Listing 12. Regrettably, Shiny has two major flaws. First, it does not reliably run if the program is multi-threaded (and ChilliSource is multi-threaded). Second, the output is not easily parsed since it is not separated by delimiters, and we required parsable output to quickly format, collate, and examine results during our studies.

Code Listing 12: Using the pseudocode from Code Listing 5, this shows how *Shiny* could be used to instrument **ParticleUpdateTask**.

```

1 ParticleUpdateTask(copiedAtts)
2 {
3     // Use Shiny macro to profile the entire block
4     PROFILE_SHARED_BLOCK(ParticleUpdateTask);
5
6     // Use Shiny macros to profile outside and inside of the
7     // particle iteration loop
8     PROFILE_SHARED_BEGIN(ParticleIter_OuterLoop);
9     for (particle in copiedAtts.particleArray)
10    {
11        PROFILE_SHARED_BEGIN(ParticleIter_InnerLoop);
12        if (particle.isActive)
13            particle.UpdateValues();
14        PROFILE_END();
15    }

```

```

16 |     PROFILE_END ();
17 |
18 |     // ... and so on
19 | }

```

4.3.3 Timing Application System

The *Timing System* was borne from dissatisfaction with the Visual Studio and Shiny function profilers. This system reliably instruments the multi-threaded engine across all platforms and it returns results that are easily parsed. It uses the built-in *Performance Timer* along with its own **StartTimer** and **StopTimer** static methods in order to time code blocks in a similar fashion to Shiny, as shown in Code Listing 13. To achieve this, it uses three hash tables² with the following key-value pairs:

- *Timing Hash Table*
 - KEY: (std::string) Code section name
 - VALUE: (double) Total time
- *Timer Hash Table*
 - KEY: (std::string) Timer key
 - VALUE: (ChilliSource::PerformanceTimer) Timer object
- *Counting Hash Table*
 - KEY: (std::string) Code section name
 - VALUE: (unsigned int) Number of code section calls

Although the *Timing System* does suit our needs, it is not as efficient as Shiny. Figures 4.8 and 4.9 show the amount of overhead that timing the inner particle iteration loop takes. This is most likely due to the creation and deletion of a timer for every **StartTimer** and **StopTimer** function call. However, this trade-off does not negatively impact our studies as the drawbacks of Shiny and Visual Studio did. This is because the timing outputs that we receive all share the same overhead from the system, and thus the outputs should be able to be compared fairly.

²In C++ 11, the closest thing to a hash table is an unordered map from the std namespace.

timed section name	time in seconds	num section calls
ParticleUpdateTask	38.6354	307
ParticleIter_OuterLoop	37.613	307
ParticleIter_InnerLoop	10.3021	301360

Figure 4.8 An example of timing output by the *Timing System* based on the metadata from Figure 4.1.

timed section name	time in seconds	num section calls
ParticleUpdateTask	5.74985	1220
ParticleIter_OuterLoop	0.298633	1220

Figure 4.9 The same example as Figure 4.8, but this series of games did not have the inner loop timed.

Code Listing 13: Using the pseudocode from Code Listing 5, this shows how our custom *Timing System* could be used to instrument **ParticleUpdateTask**.

```

1 ParticleUpdateTask(copiedAtts)
2 {
3     // Statically call the timing system to start a timer with hash key
4     // "ParticleUpdateTask".
5     pUpdateTimerKey = TimingSystem::Start("ParticleUpdateTask");
6
7     // Use the timing system to keep track of the outer and inner loop
8     // times during particle iteration.
9     pOutIterTimerKey = TimingSystem::Start("ParticleIter_OuterLoop");
10    for (particle in copiedAtts.particleArray)
11    {
12        pInIterTimerKey = TimingSystem::Start("ParticleIter_InnerLoop");
13        if (particle.isActive)
14            particle.UpdateValues();
15        TimingSystem::Stop("ParticleIter_InnerLoop", pInIterTimerKey);
16    }
17    TimingSystem::Stop("ParticleIter_OuterLoop", pOutIterTimerKey);
18
19    // ... and so on
20
21    // Stop the timer with hash key "ParticleUpdateTask" and the outputted
22    // timer key
23    TimingSystem::Stop("ParticleUpdateTask", pUpdateTimerKey);
24 }
```

4.4 Output and its Evolution

From the examples that were shown above, we can see that the system output lends itself to a tabular structure. The raw output is, in fact, following a comma-separated-values format. Storing the data like this was simple, but storing the metadata proved to be more difficult.

At first, the metadata was stored all within the titles of the CSV (e.g. `metrics_maxRunNum = 5_particle-effects = 1 ... [timestamp].csv`). This worked well when there were only a couple of metadata values, but, as we saw above, there are now a great deal of metadata values. Predictably, the titles of the CSVs eventually became so unruly that Windows refused to open the files because the "file path was too long". This problem was solved by first bundling the metrics and timing CSVs into one timestamped directory, and then outputting another CSV file exclusively for metadata in the same directory.

Although the metadata problem was solved, bundles of CSVs were not a particularly tidy approach when it came to distribution. However, any other method of outputting the data in C++ would be far too complicated and out of scope for the purposes of our studies. Outputting the data in another format using python, however, would not be as complex. A python script was created to import the three bundled CSVs into what is called an HDF5 file. HDF5, i.e. Hierarchical Data Format 5, is "a unique open source technology suite for managing data collections of all sizes and complexity," [7], but it was truly designed to support large, complex datasets that could be used on every size and type of system. HDF5 is hierarchical, high-performance, portable, can be used in numerous languages, and it is self-describing. Admittedly, HDF5 may be overkill for our tens of CSVs, but its ability to efficiently bundle all of our output files with metadata built right in is still incredibly useful.

4.5 Initial Results

A variety of automated games were ran and their outputs were examined on Windows, iOS, and Android platforms. This was done in an effort to find an area within the *Particle Effect Component* class that needed optimization. In the beginning of our studies, the only available datasets were from the *Metrics System* and the Visual Studio CPU sampling profiles, as the timing instrumentation was still not complete. We found that a particular automated game in which the *total maximum particles* and *particles per emission* values were the same³ caused the game to visually lag, or "skip frames". Armed with only the

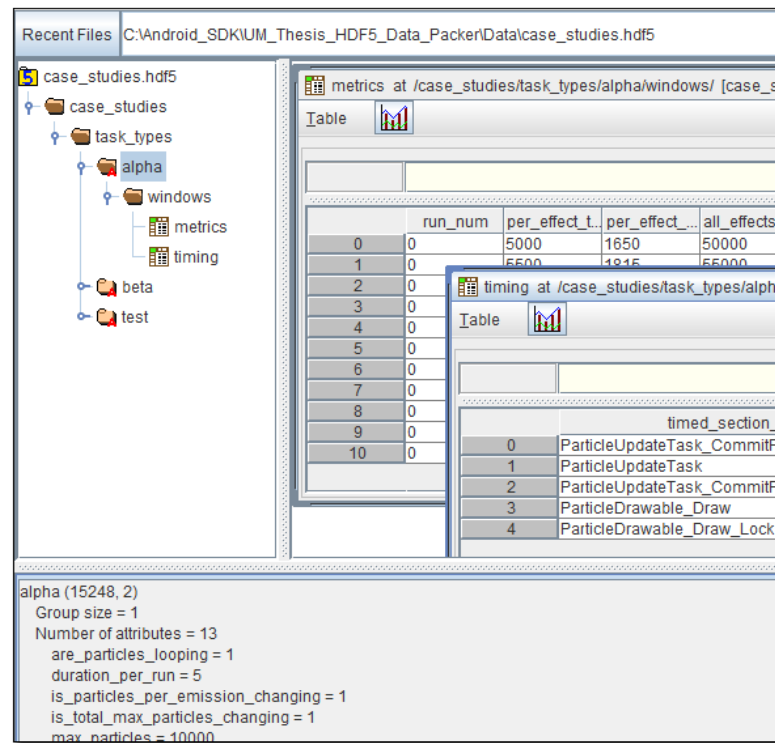


Figure 4.10: An example of an HDF5 file containing sample metrics, timing, and metadata information.

metrics information and CPU sampling (see Figure 4.11, all we knew is that the following *Particle Effect Component* methods had a considerable number of samples:

- **ParticleDrawable::DrawParticles**
- **ParticleUpdateTask**
- **ParticleAffector::AffectParticles**

Given that these three methods are ran repeatedly throughout the games, these results are unsurprising. At the time, it was tempting to pursue **ParticleUpdateTask** since it iterated over the entire particle array multiple times. We thought that, perhaps, creating a cache-friendly data structure that kept track of active particles would be a good initial optimization. However, as it shows in Figure 4.13, the functions that iterate over all of the particles are only taking $(1.0 + 1.7 + 12.0 + 0.6)/48.4 \approx 32\%$ of **ParticleUpdateTask**'s

³Specifically, *total maximum particles* and *particles per emission* were 10,000 in Figures 4.11, 4.12, and 4.13.

Level	Function Name	Inclusive Samples %	Exclusive Samples %
8	ChilliSource::Renderer::RenderSceneToTarget	66.57	0
9	ChilliSource::Renderer::Render	63.03	0
10	ChilliSource::ParticleEffectComponent::Render	60.84	0
11	ChilliSource::ParticleDrawable::Draw	60.7	0
12	ChilliSource::StaticBillboardParticleDrawable::DrawParticles	60.36	0
13	ChilliSource::StaticBillboardParticleDrawable::DrawWorldSpace	60.21	0.2
12	ChilliSource::TaskPool::ProcessTasks	29.02	0
13	ChilliSource::TaskPool::PerformTask	29.02	0
20	ChilliSource::'anonymous namespace'::ParticleUpdateTask	28.94	0.13
14	ChilliSource::'anonymous namespace'::BuildSpriteData	25.13	0.55
14	ChilliSource::DynamicSpriteBatch::Render	24.3	0.23
21	ChilliSource::ColourOverLifetimeParticleAffector::AffectParticles	14.02	0.32
15	ChilliSource::GenericVector3<float>::Rotate	11.39	0.52
16	ChilliSource::GenericVector3<float>::Rotate	10.81	1.12
15	ChilliSource::DynamicSpriteBatch::ForceRender	9.33	0
16	ChilliSource::DynamicSpriteBatch::BuildAndFlushBatch	9.26	0

Figure 4.11 A Visual Studio call tree sorted in the same way as Figure 4.7, but from the games based on the metadata in Figure 4.12.

attribute	value
are particles looping	TRUE
num particle effects	1
min particles	10000
max particles	10000
particles step	500
is total max particles changing	TRUE
is particles per emission changing	TRUE
particles per emission	0
total maximum particles	0
particles per emission step	500
total maximum particles step	500
total num runs	10
duration per run	5

timed section name	time in seconds	num section calls
ParticleDrawable Draw	41.114	692
ParticleDrawable Draw Lock	0.0222	692
ParticleUpdateTask	48.4279	652
ParticleUpdateTask BoundingShapes	0.616089	652
ParticleUpdateTask CommitParticleData	31.5472	652
ParticleUpdateTask CommitParticleData Lock	29.3924	662
ParticleUpdateTask ParticleEffectors	12.0305	652
ParticleUpdateTask ParticleEmission	1.71181	652
ParticleUpdateTask ParticleIteration	0.998673	652

Figure 4.13 Timing output based on the metadata in Figure 4.12.

Figure 4.12 Metadata of a series of games that demonstrated contention in **CommitParticleData**.

time. The **CommitParticleData** function, on the other hand, is taking $31.5/48.4 \approx 65\%$ of **ParticleUpdateTask**'s time, and 29.4 of those 31.5 seconds are spent waiting for a lock to release. Thus, it is clear that, in this case, **ParticleUpdateTask**'s bottleneck was

CommitParticleData.

These initial results not only demonstrate a possible place to optimize, but it also reinforces the importance of backing up possible optimizations with data in order to avoid optimizing into a vacuum. Without the timing data, we could have easily gone forward with optimizing particle iteration instead of focusing on the contention in **CommitParticleData**, and that would have been a waste of time.

CHAPTER 5 CHILLISOURCE PARTICLE EFFECT COMPONENT CASE STUDIES

5.1 Counting Contention

From our initial results, we found that **CommitParticleData** was experiencing a great deal of contention. Figure 4.13 showed that, of the 31.5 seconds that **CommitParticleData** took, 29.4 seconds was spent waiting for a lock to release. This tells us, then, that it only needed about 2 seconds to do any meaningful work and, if we could reduce this contention, we could speed up that function drastically. The first step in reducing this contention, however, is knowing for certain what is making the background thread in **CommitParticleData** wait.

The simplest way to ascertain what is causing the contention is to do some counting. In *Concurrent Particle Data*, we can keep track of an atomic *thread counter* which will

Function Name	Thread Counter > 1 After Increment (<i>thread is waiting for lock</i>)	Thread Counter >1 Before Decrement (<i>other thread is waiting for lock</i>)
StartUpdate	301	0
HasActiveParticles	0	0
GetAABB	0	0
GetBoundingSphere	0	0
Lock	1	0
Unlock	0	521
CommitParticleData	521	302

Figure 5.1: The results of "counting the contention" between the background and main threads in *Concurrent Particle Data*.

keep track of how many threads are contending for resources. This *thread counter* will be incremented right before a thread attempts to gain access to a lock, and it will be decremented right before it relinquishes access to that lock. The **IncrementCounter** and **DecrementCounter** methods will print a message when the thread counter is greater than one. During incrementation, a *thread counter* that is greater than one means that the current thread is waiting to retrieve the lock. During decrementation, a *thread counter* that is greater than one means that another thread is waiting for the current thread to release the lock. Since we have only scheduled one background thread, we know that the *thread counter* should never be greater than two.

This "contention counting" was done using the same metadata from Figure 4.12 in our initial results, and the data collected can be found in Figure 5.1. The functions in which either the background or main threads waited for a lock to release were **StartUpdate**¹ (301 times), **Lock** (1 time), and **CommitParticleData** (521 times). The functions in which the background or main thread had the other thread waiting for a lock to release were **Unlock** (521 times) and **CommitParticleData** (302 times). This tells us, then, that **StartUpdate** and **Lock** were waiting on **CommitParticleData** since $301 + 1 = 302$, and that **CommitParticleData** was waiting on **Unlock** since their counts were both 521.

With these results in mind, recall from Section 2.3.3 that the **Lock** function is used by the *Particle Drawable* when drawing particles in order to lock **ConcurrentParticleData** and secure access to the particle array. We know from our study of the system that the *Particle Drawable* is the only one who uses this **Lock** function, and so we know exactly where and how the contention is taking place. We also know that the *Particle Drawable* reads information from *Concurrent Particle Data*'s particle and new particle indices array. Thus, it is reasonable to say that it is unnecessary to lock both of those data structures at once and, instead, we could try using more than one mutex in **CommitParticleData** and

¹Although not a part of the pseudocode provided for the **ParticleEffectComponent::OnUpdate** method in earlier chapters, this simple function merely lets the caller know whether or not particle data has been committed since the last time it was called.

Draw and see if that reduces the contention.

5.2 Using More Than One Mutex

5.2.1 Code

The changes needed to add mutexes for the two data structures and remaining variables that are copied are fairly minimal. The *particlesMutex*, *newParticlesIndicesMutex*, and the *normalMutex* are added as member variables to *Concurrent Particle Data*. Each of these mutexes are locked when their respective variables need to be read or written to. When a mutex is locked within *Concurrent Particle Data*, it is done by directly accessing the mutex (see Code Listing 14). When a mutex is locked within the *Particle Drawable*, it is done by utilizing special locking functions² provided by *Concurrent Particle Data* (see Code Listing 15).

One of the potential issues with these changes is that, even though a particle is currently active, it may not be available for rendering by the time the **DrawParticles** function is called. To more concretely understand, imagine that the *Particle Drawable* is taking in new particle indices to activate within the static billboard to make it available for rendering. At the same time, **CommitParticleData** was copying new particle data and there happened to be newly emitted particle indices. Since the *Particle Drawable* had old information that did not include these newly emitted particles, these new particles will not be rendered because, even though they are marked as active, they are not activated in the static billboard. Although this sounds like a significant problem, we can argue that the inconsistent state cannot possibly be perceived by the user since a considerable number of updates happen within a short span of time. If we look at Figure 5.5, for example, the number of calls to the **Draw** function is about 800, and that is within approximately a $20 \text{ runs} * 5 \text{ seconds} / \text{run} = 100$ second timespan. This means that this function was called about 8 times per second, and

²In actuality, a single locking function is used. An enumerator is passed to the function specifying which mutex needs to be locked. Although this is better practice, it obfuscates the actual meaning of the code by making it more verbose.

it is highly doubtful that a person could notice the inconsistent state of the particle effect within $\frac{1}{8}$ th to $\frac{1}{4}$ th of a second. Thus, it is not something that we will worry about.

Code Listing 14: Pseudocode illustrating the changes needed to utilize more than one mutex in the **CommitParticleData** function.

```

1 ConcurrentParticleData::CommitParticleData(particleArray,
2                                           newParticleIndicesArray,
3                                           AABB, Sphere)
4 {
5     // Lock the particles mutex
6     using lock(this.particlesMutex)
7     {
8         // Copy over all particles to the member particle array
9         for(i = 0 to particleArray.size())
10        {
11            this.particleArray[i] = particleArray[i].copySubset();
12        }
13    }
14
15    // Lock the new particle indices mutex
16    using lock(this.newParticleIndicesMutex)
17    {
18        // Append the new indices to the member new indices array
19        this.newParticleIndices.append( newParticleIndices.copy() );
20    }
21
22    // Lock the normal (non-data structure) mutex
23    using lock(this.normalMutex)
24    {
25        // Copy the bounding information to the member variables
26        this.AABB = AABB.copy();
27        this.Sphere = Sphere.copy();
28    }
29 }

```

Code Listing 15: Pseudocode illustrating the changes needed to utilize more than one mutex in the **Draw** function.

```

1 Drawable::Draw(camera)
2 {
3     // Lock the particle new indices array
4     this.concurrentParticleData.LockNewParticleIndices();
5
6     // Activate the newly emitted particles in the static billboard
7     for(particleIndex in this.concurrentParticleData.GetNewParticleIndices())
8     {
9         ActivateParticleInBillboard(particleIndex);
10    }
11
12    // Unlock the particle new indices array
13    this.concurrentParticleData.UnlockParticleIndices();
14 }

```

```

15 // Lock the particles array
16 this.concurrentParticleData.LockParticles();
17
18 // Iterate through all of the particles and draw each one
19 DrawParticles(this.concurrentParticleData.GetParticleArray(), camera);
20
21 // Unlock the particles array
22 this.concurrentParticleData.UnlockParticles();
23 }

```

5.2.2 Results

attribute	1 PE, 0-50K	1 PE, 50K
are particles looping	TRUE	TRUE
num particle effects	1	1
min particles	0	50K
max particles	50K	50K
particles step	5K	5K
is total max particles changing	TRUE	TRUE
is particles per emission changing	TRUE	TRUE
particles per emission	0	0
total maximum particles	0	0
particles per emission step	{500, 5K}	{500, 5K}
total maximum particles step	5K	5K
total num runs	5	20
duration per run	5	5

Figure 5.2: The metadata used for the 8 series of automated games for the case studies.

The following four figures are the result of running a series of automated games in two scenarios:

1. 1 particle effect, particles range from 0 - 50K, 5K step, 5 5-second runs per effect.
2. 1 particle effect, 50K, 0 step, 20 5-second runs.

Both of these scenarios were ran when *particles per emission* (PPE) was 10% and 100% of *total maximum particles* (TMP). In addition, these four series of automated games were ran with and without the above changes to the source code. This totals to 8 series of automated games. Figure 5.2 shows the specific metadata values that were used. The first scenario is

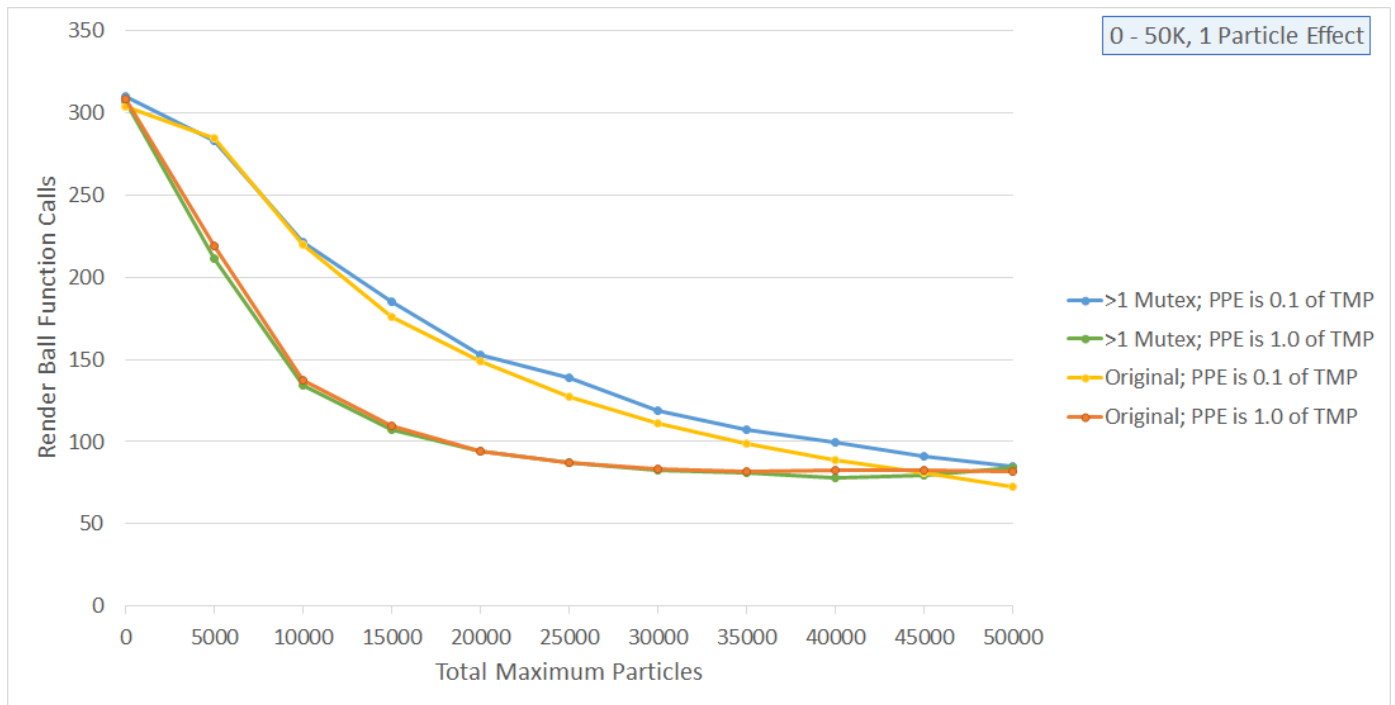


Figure 5.3 A graph showing *render ball function calls* over *total maximum particles* for a single particle effect.

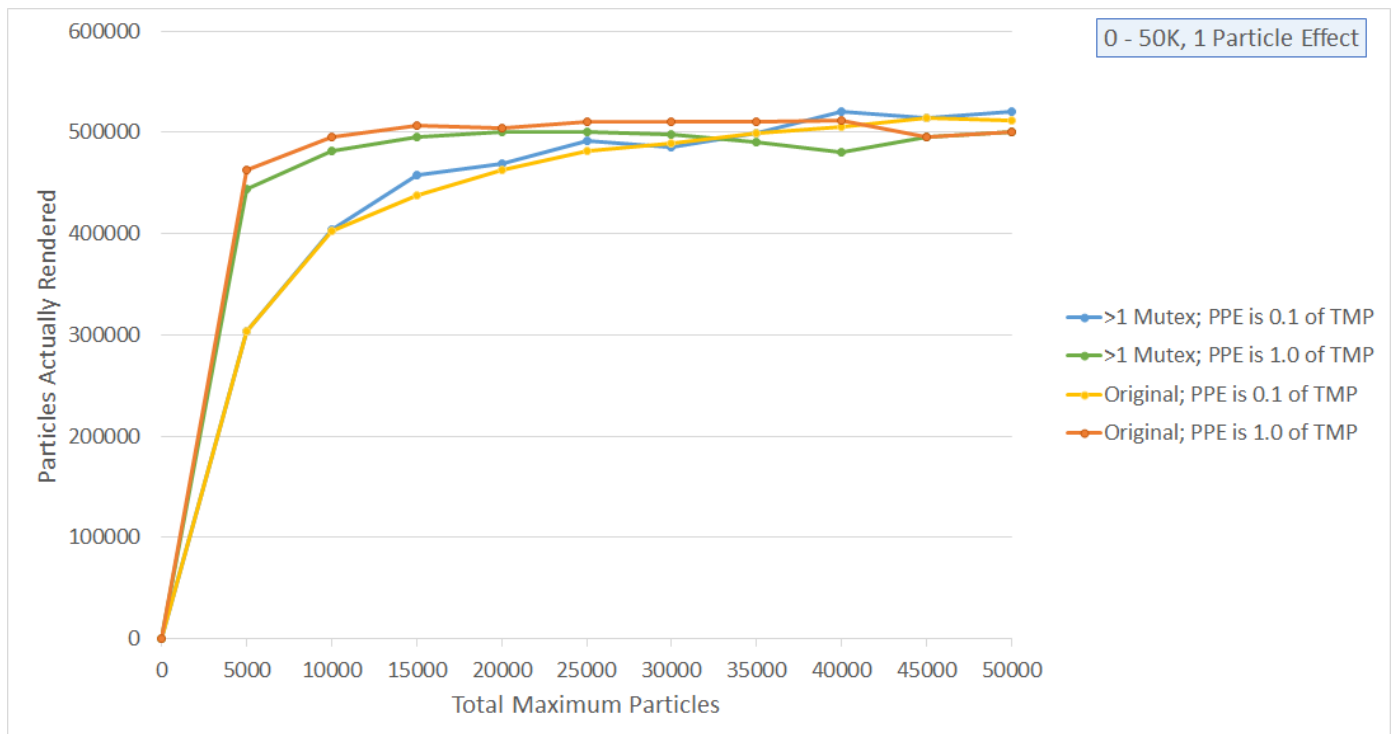


Figure 5.4 A graph showing *particles actually rendered* over *total maximum particles* for a single particle effect.

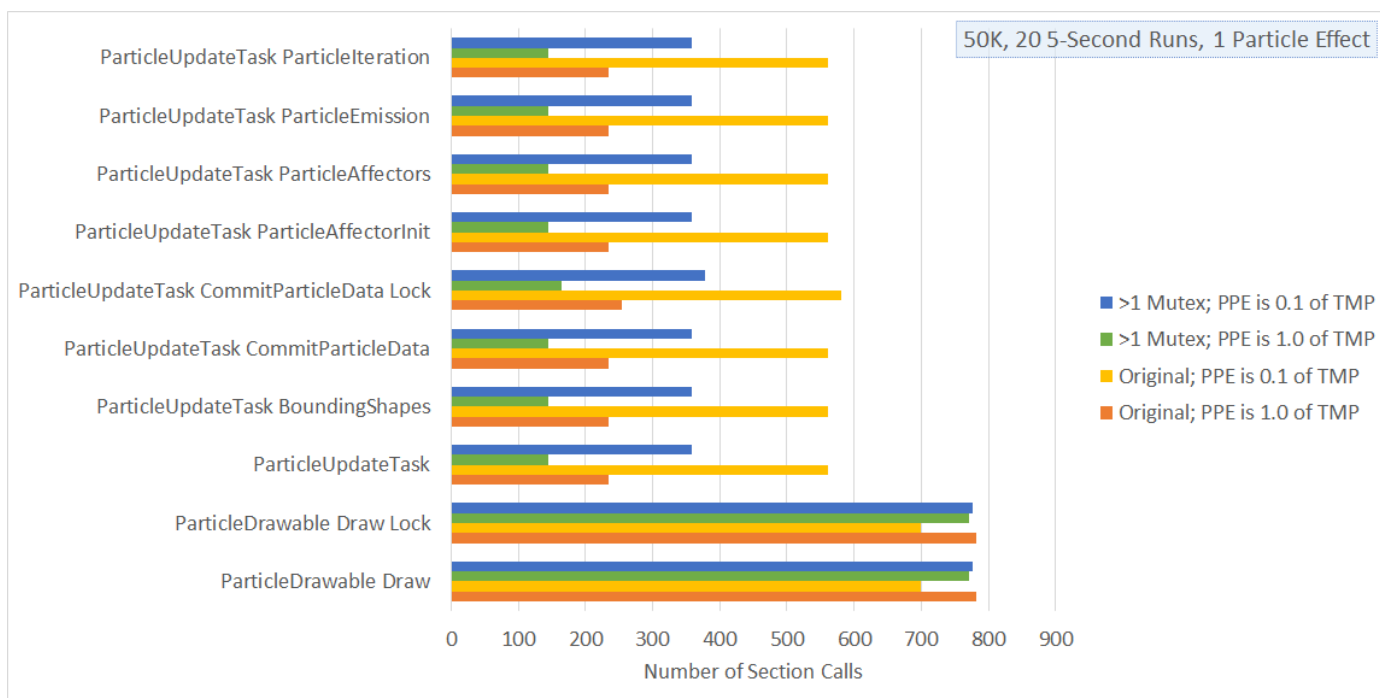


Figure 5.5 A graph showing the number of section calls for various sections of code for a single particle effect. Note that the "ParticleUpdateTask CommitParticleData Lock" section is timing the particle array lock for the "> 1 Mutex" data series.

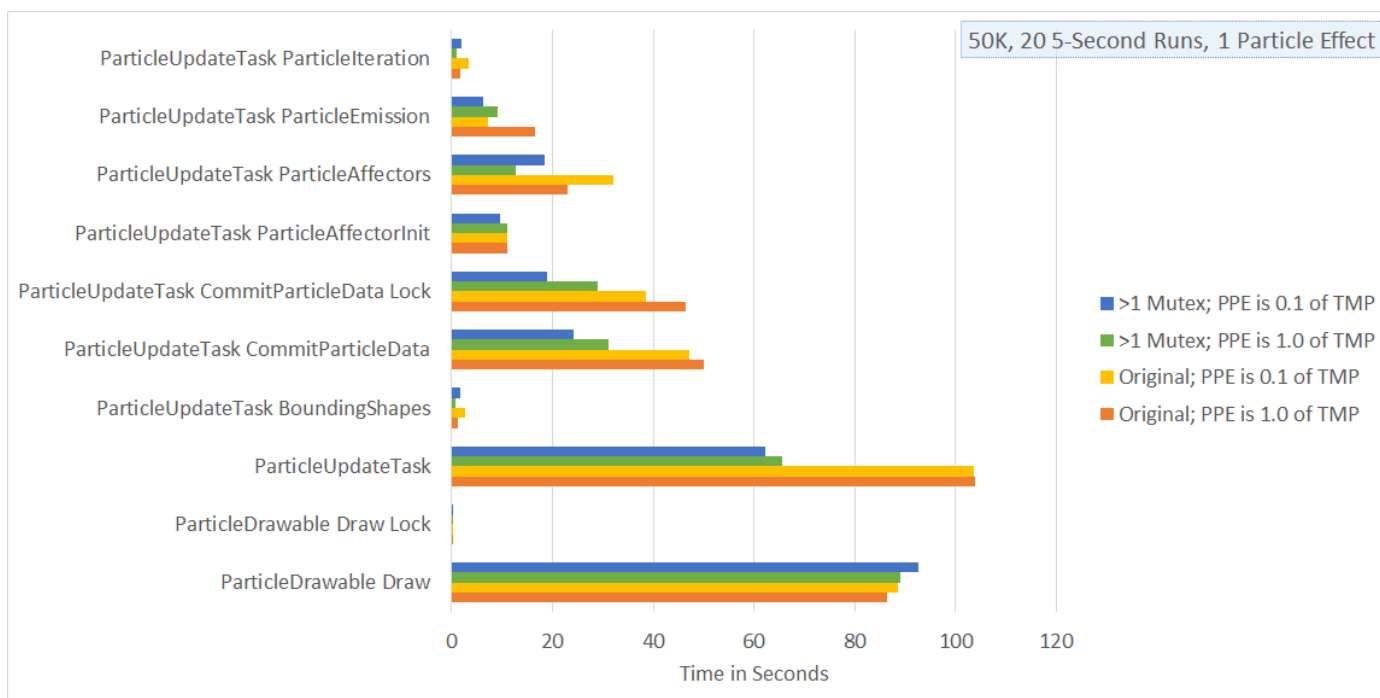


Figure 5.6 A graph showing the time spent in various sections of code for a single particle effect. Note that the "ParticleUpdateTask CommitParticleData Lock" section is timing the particle array lock for the "> 1 Mutex" data series.

shown in figures 5.3 and 5.4, and the second scenario is shown in figures 5.5 and 5.6.

When examining these results, the yellow "Original 0.1 TMP" series should be directly compared with the blue "> 1 Mutex 0.1 TMP" series and the orange "Original 1.0 TMP" series should be directly compared with the green "> 1 Mutex 1.0 TMP" series. This is because comparing the two series that "played the same game" will yield more accurate conclusions than cross-comparisons would.

5.2.3 Discussion

The figures that focus on the particle metrics *render ball function calls* and *particles actually rendered* do not exhibit a great deal of improvement from the original. For both of those figures, improvement would mean higher values. This is because, when more particles are rendered or the ball is rendered more often, the game appears to run more smoothly. Or, in other words, the "frames per second" improves.

The figures focusing on the particle timing, on the other hand, show considerable improvement. For 0.1 TMP, **CommitParticleData** went down from 47 seconds to 24. For 1.0 TMP, **ParticleUpdateTask** went down from 50 seconds to 31. This is approximately a 40% increase in speed!³ However, we can see that there is still a large amount of contention for the particle array. For 0.1 TMP, 19 of those 24 seconds was spent waiting for the particle array's mutex to unlock. For 1.0 TMP, 29 of those 31 seconds was spent waiting for the particle array's mutex to unlock. It is clear, then, that we should go further in our attempt to reduce contention for the particle array. Instead of adding more mutexes, we could try taking one away.

³See Figure 5.20 in Section 5.4 for a summary table of these timed sections.

5.3 Using A Lock Free Data Structure

5.3.1 Code

A lock free data structure could mitigate the contention for the particle array by getting rid of its mutex completely. Similar to our "multiple mutex" solution, this has the potential problem of the particle array being in an inconsistent state during rendering. However, an analogous argument can be made. Even though the particle array might have partially up to date information during any given render, the renderer is going to update again very shortly and the particle array should be up to date by then. We can know this because **CommitParticleData** is called frequently (see Figure 5.5) and, without waiting for locks to release, **CommitParticleData** runs fairly quickly. Therefore, the potentially inconsistent state of particles should not be noticeable to the end user.

To make the particle array a lock free data structure, we have to first identify which threads are reading and which ones are writing. We know that the main thread (the one that is drawing) is only reading the particles, and we also know that the background thread is writing to the particles. Without a mutex guarding the particle array, there is a possibility that the main thread reads a particle just as the background thread is writing to it. This may result in corrupted data. Thus, a solution in which the two threads can both simultaneously access the array but not the same particle is needed.

There are a variety of strategies that we could employ, but the simplest is to rely on an atomic instruction that is used to achieve synchronization. Specifically, we will use something that is called a *compare-and-swap* operation. A *compare-and-swap* is an atomic read-modify-write operation on a memory location that can only be implemented using special hardware instructions [8]. In one atomic action, it compares the value of a variable with what is expected and, once the variable has that expected value, it swaps that expected value with a desired value. Once the atomic operation is attempted, it generally returns a boolean indicating whether or not the operation was successful. A *compare-and-swap* operation is typically wrapped within a while loop since it may return false if

a concurrent or spurious write occurred. The latter may occur if the value of the variable is actually the expected value but the operation fails anyway. This could happen if we utilized the weak version of *atomic_compare_exchange* [9], which is a ***compare-and-swap*** operation defined in the **std::atomic** library in C++11. One could utilize the strong version of *atomic_compare_exchange*, instead, in order to avoid spurious writes, but the checks needed within the strong version are expensive. Thus, the weak version can yield better performance if the operation is wrapped within a loop. Due to the infrequency of spurious writes and the small number of threads, the weak version would be the best option in our case. Given that this operation is supported on all of the platforms that ChilliSource supports, the weak version of *atomic_compare_exchange* will be used.

In order to take advantage of the *atomic_compare_exchange* function, we must use an atomic variable. An atomic boolean was incorporated into *Concurrent Particle Data*'s particle struct, as shown in Code Listing 16. This atomic boolean, *isContended*, serves as the variable that the ***compare-and-swap*** operation examines. It indicates whether or not another thread is using this particle struct. When a thread wants access to the particle struct, it calls the *atomic_compare_exchange* function. It will then compare *isContended* with "false" and, if it is false, will swap *isContended* with "true". The normal particle operations will take place, and then *isContended* will be set back to false. Code listings 17 and 18 illustrate the needed changes in the **CommitParticleData** and **Draw** functions.

Code Listing 16: The struct that is used to within the particle array in *Concurrent Particle Data* during the Lock Free case study.

```

1  struct Particle final
2  {
3      mutable std::atomic<bool> m_isContended = { false };
4      bool m_isActive = false;
5      Vector3 m_position;
6      Vector2 m_scale = Vector2::k_zero;
7      f32 m_rotation = 0.0f;
8      Colour m_colour = Colour::k_white;
9  };

```

Code Listing 17: Pseudocode illustrating the changes needed to make the *particles* array a lock free data structure in the **CommitParticleData** function.

```

1 ConcurrentParticleData::CommitParticleData(particleArray,
2                                           newParticleIndicesArray,
3                                           AABB, Sphere)
4 {
5     // Copy over all particles to the member particle array
6     for(i = 0 to particleArray.size())
7     {
8         // Retrieve the particle, but we don't have ownership of it yet
9         currentParticle = particleArray[i];
10        gotOwnershipOfParticle = false;
11
12        // Keep spinning if *isContended* is true.
13        // Set it to true once it is false.
14        while(gotOwnershipOfParticle is false)
15        {
16            gotOwnershipOfParticle = currParticle.isContended
17                                   .CompareExchange(false, true);
18        }
19
20        // Copy the particle now that we have ownership of it
21        this.particleArray[i] = currentParticle.copySubset();
22
23        // Release the particle
24        currentParticle.isContended = false;
25    }
26
27    // Lock the mutex for the remainder of the function
28    using lock(this.mutex)
29    {
30        // Append the new indices to the member new indices array
31        this.newParticleIndices.append( newParticleIndices.copy() );
32
33        // Copy the bounding information to the member variables
34        this.AABB = AABB.copy();
35        this.Sphere = Sphere.copy();
36    }
37 }

```

Code Listing 18: Pseudocode illustrating the changes needed to make the *particles* array a lock free data structure in the **Draw** function.

```

1 Drawable::Draw(camera)
2 {
3     // Lock the concurrent particle data object
4     this.concurrentParticleData.Lock();
5
6     // Activate the newly emitted particles in the static billboard
7     for(particleIndex in this.concurrentParticleData.GetNewParticleIndices())
8     {
9         ActivateParticleInBillboard(particleIndex);

```

```

10     }
11
12     // Unlock the concurrent particle data object
13     this.concurrentParticleData.Unlock();
14
15     // Iterate through all of the particles and draw each one
16     DrawParticles(this.concurrentParticleData.GetParticleArray(), camera);
17 }
18
19 StaticBillboardDrawable::DrawParticles(particleArray, camera)
20 {
21     // Copy over all particles to the member particle array
22     for(i = 0 to particleArray.size())
23     {
24         // Retrieve the particle, but we don't have ownership of it yet
25         currentParticle = particleArray[i];
26         gotOwnershipOfParticle = false;
27
28         // Keep spinning if *isContended* is true.
29         // Set it to true once it is false.
30         while(gotOwnershipOfParticle is false)
31         {
32             gotOwnershipOfParticle = currParticle.isContended
33                                     .CompareExchange(false, true);
34         }
35
36         // Render the particle if it is active and is activated in the billboard
37         if(currentParticle.isActive and currentParticle is in particleBillboard)
38         {
39             Render(currentParticle);
40         }
41
42         // Release the particle
43         currentParticle.isContended = false;
44     }
45 }

```

5.3.2 Results

Just as in the first case study, the lock free case study produced the following four figures running a series of automated games in two scenarios:

1. 1 particle effect, particles range from 0 - 50K, 5K step, 5 5-second runs per effect.
2. 1 particle effect, 50K, 0 step, 20 5-second runs.

Again, both of these scenarios were ran with PPE values that were 10% and 100% of TMP. These four series of automated games were ran with and without the above changes

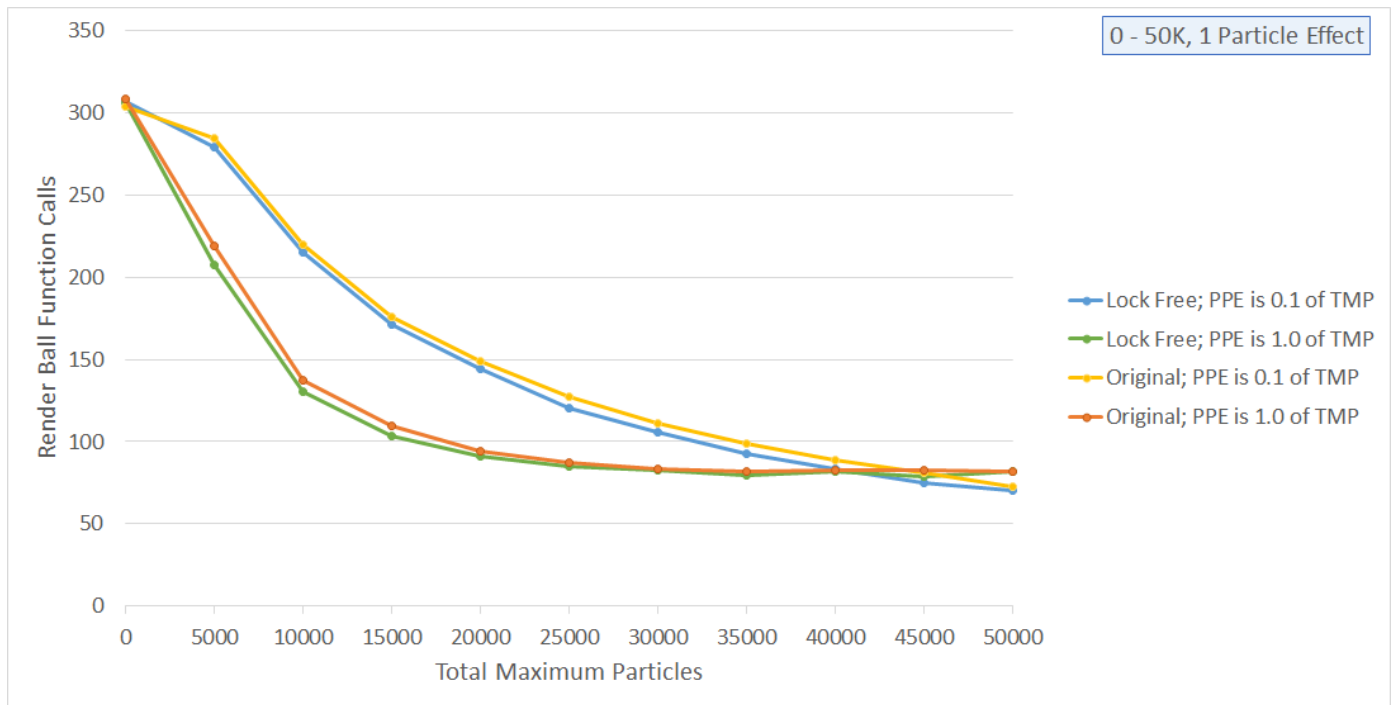


Figure 5.7 A graph showing *render ball function calls* over *total maximum particles* for a single particle effect.

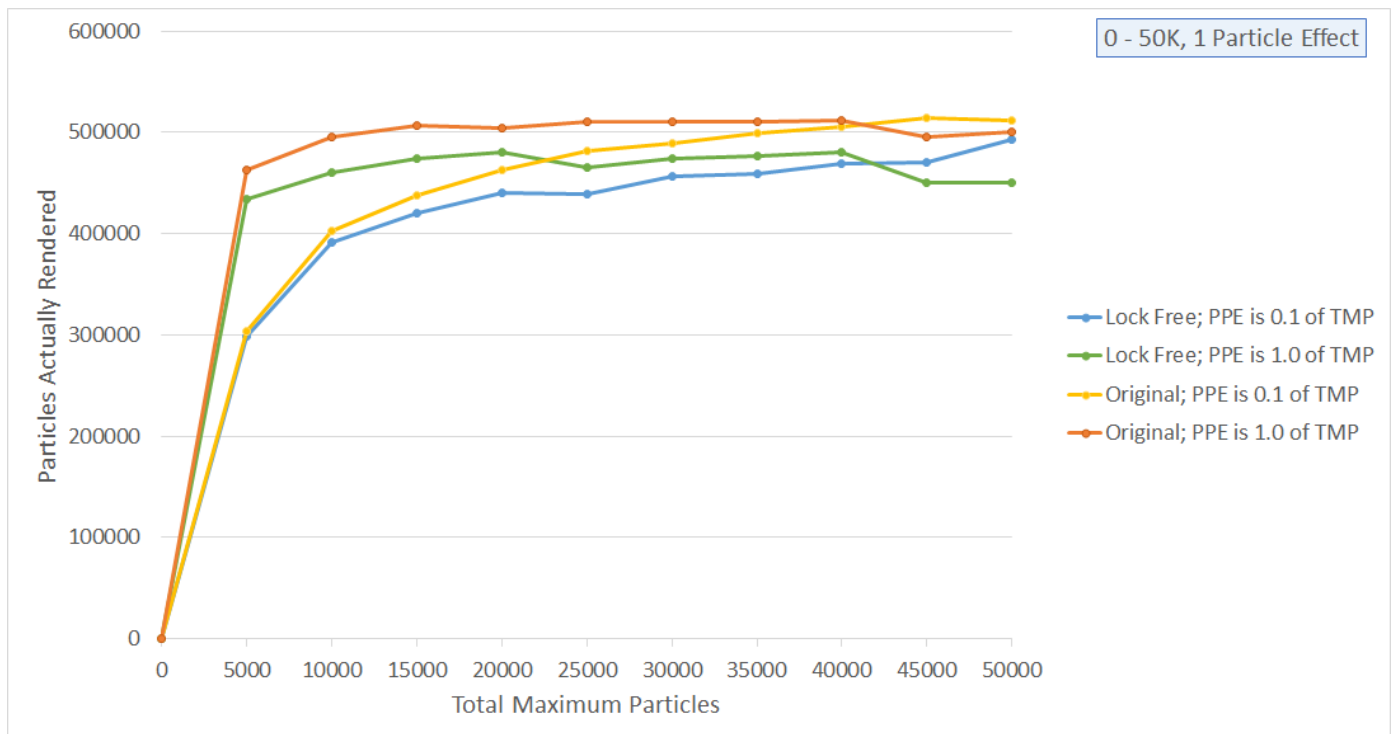


Figure 5.8 A graph showing *particles actually rendered* over *total maximum particles* for a single particle effect.

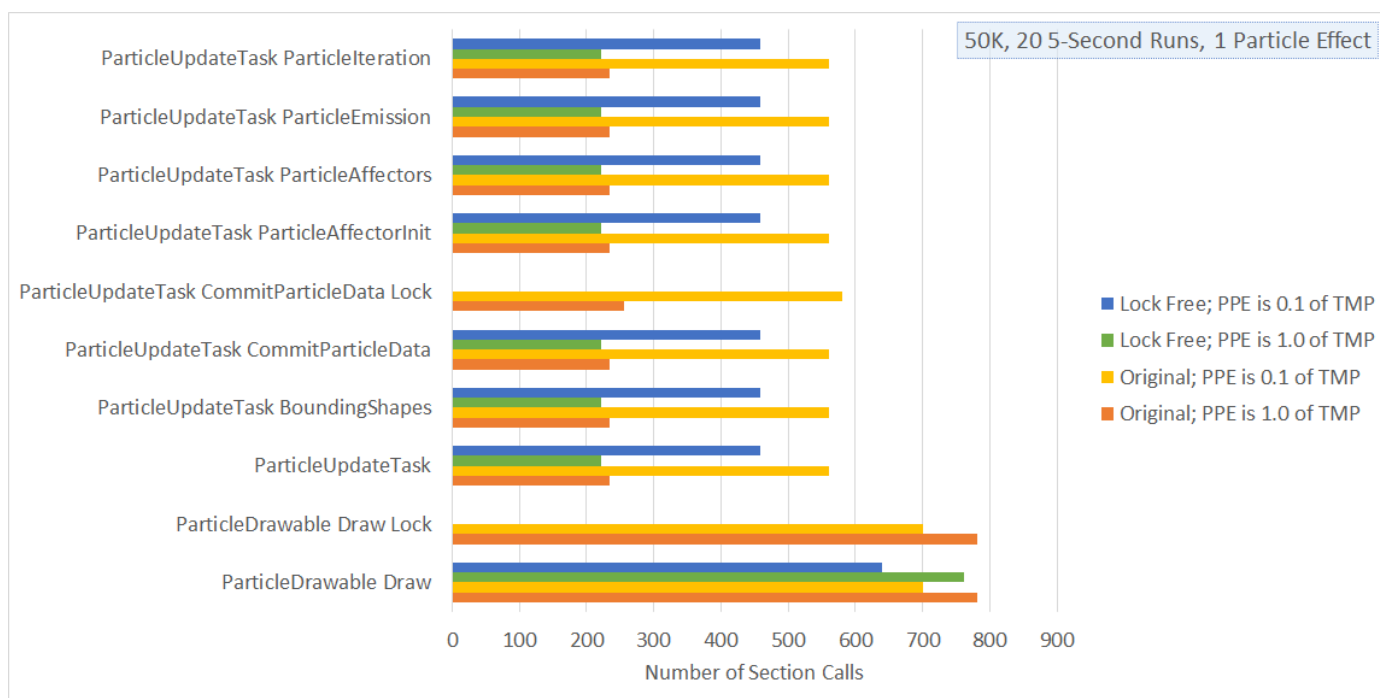


Figure 5.9 A graph showing the number of section calls for various sections of code for a single particle effect.

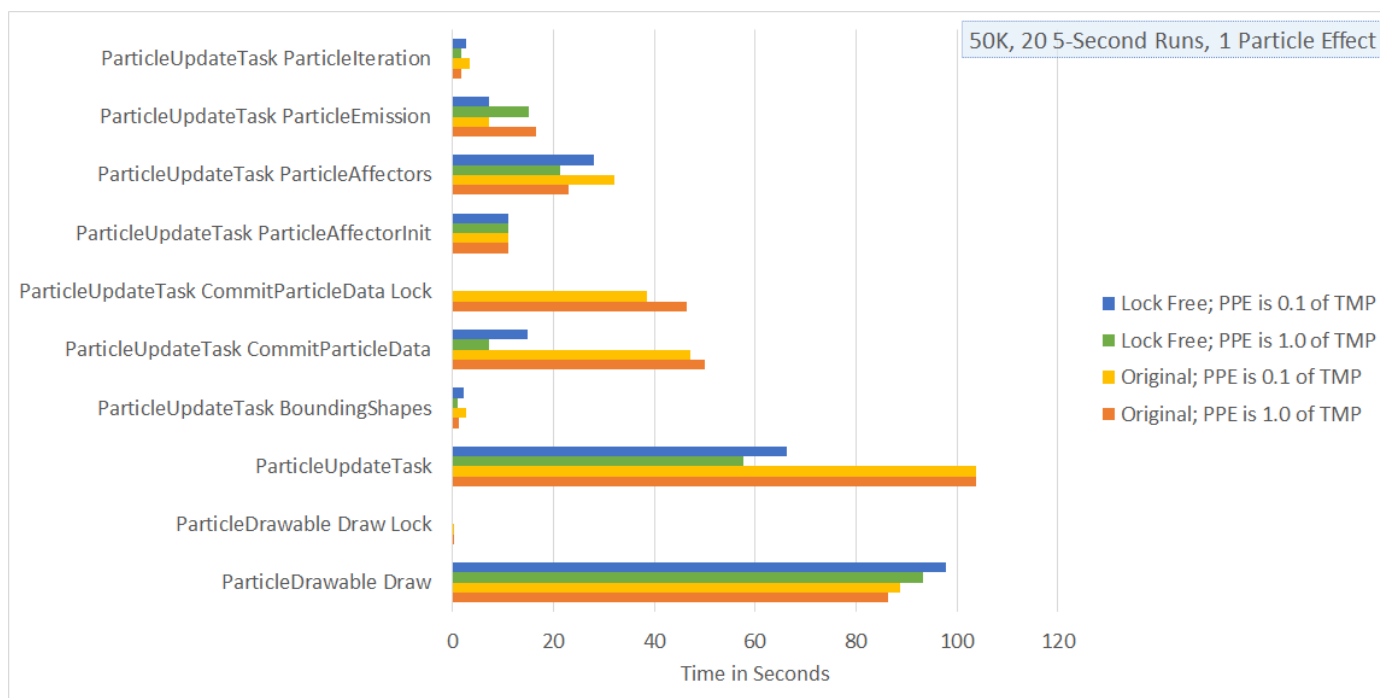


Figure 5.10 A graph showing the time spent in various sections of code for a single particle effect.

to the source code, and this totals to 8 series of automated games. The first scenario is shown in figures 5.7 and 5.8, and the second scenario is shown in figures 5.9 and 5.10. We can refer to Figure 5.2 for the specific metadata values used.

As before, we should compare the two series that "played the same game" in order to ensure that we draw the correct conclusions.

5.3.3 Discussion

The *particles actually rendered* and *render ball function calls* figures, similar to the first case study, do not show significant improvement from the original. However, the Lock Free data series in the *particles actually rendered* figure seem to have slightly lower numbers than the original, but it is not a significant enough difference to say with absolute certainty that the Lock Free code results in fewer particles being rendered.

As predicted, the timing metric figures show that **CommitParticleData** experienced a healthy performance boost. For 0.1 TMP, the function went down from 47 seconds to 15. For 1.0 TMP, it went down from 50 seconds to 7. This is, respectively, about a 68% and 86% boost!⁴We see a larger boost in the latter scenario due to the fact that there are more active particles earlier in the game(s), and so we should expect to see more contention since the draw function needs to render more particles. This will block **CommitParticleData** for longer, and minimizing that increased wait time will result in better run times.

Although these are excellent initial results for the the Lock Free case study, there are two problems:

- The particle metric figures did not improve.
- The timing metric are only favorable for a single particle effect.

The first problem is an issue because these optimizations should have aided in rendering more particles since updating them takes less time. As for the second problem, we will come to understand in the next section that the two case studies behave quite differently when the games are ran with multiple particle effects instead of just one.

⁴See Figure 5.20 in Section 5.4 for a summary table of these timed sections.

5.4 Unexpected Results with Multiple Particle Effects

attribute	10 PE, 0-5K	10 PE, 5K
are particles looping	TRUE	TRUE
num particle effects	10	10
min particles	0	5K
max particles	5K	5K
particles step	500	500
is total max particles changing	TRUE	TRUE
is particles per emission changing	TRUE	TRUE
particles per emission	0	0
total maximum particles	0	0
particles per emission step	{50, 500}	{50, 500}
total maximum particles step	500	500
total num runs	5	20
duration per run	5	5

Figure 5.11: The metadata used for the 8 series of automated games for the unexpected results from the case studies.

In order to exhibit the unexpected results for the Multiple Mutex and Lock Free case studies, we will run 8 automated games per case study as we did before. This time, however, we will distribute the particles across 10 particle effects. For example, the scenario that had a particle range of 0 to 50K will now have one of 0 to 5K per particle effect. For clarification, please refer to Figure 5.11.

5.4.1 Using More Than One Mutex Results

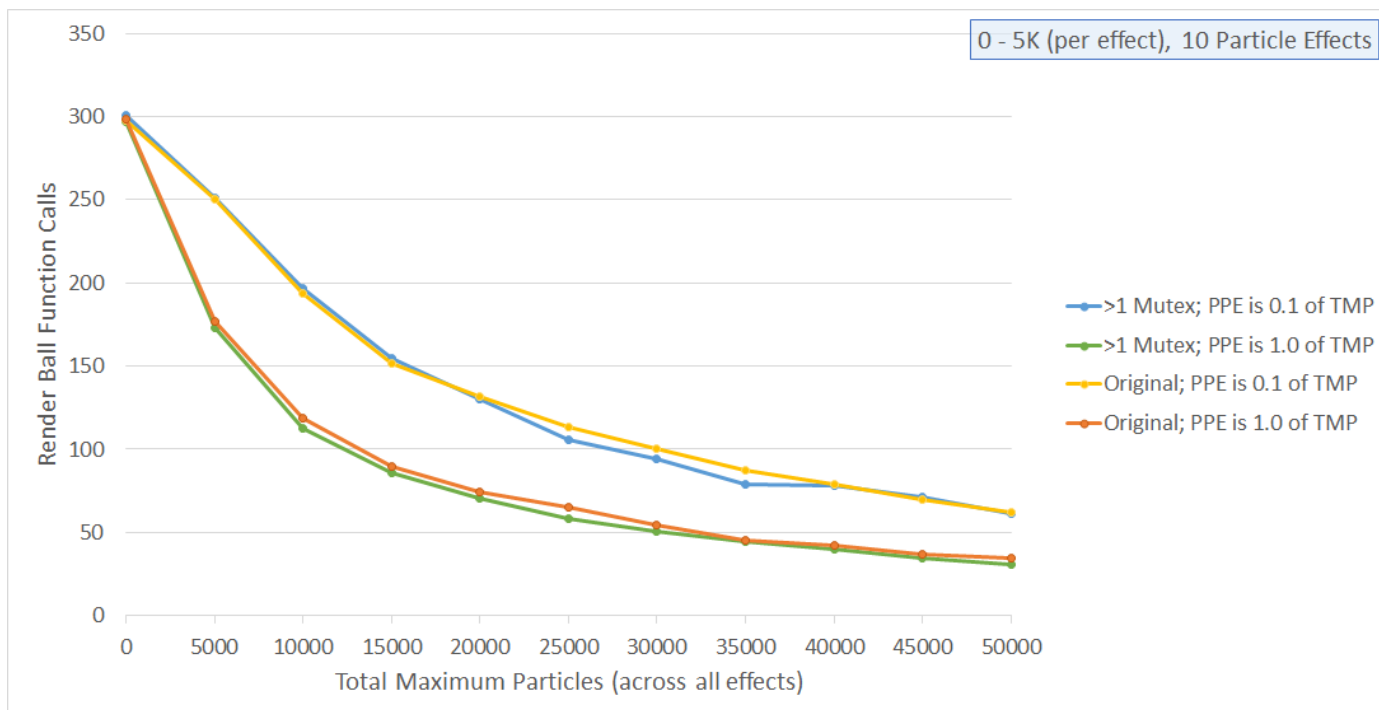


Figure 5.12 A graph showing *render ball function calls* over *total maximum particles* for ten particle effects.

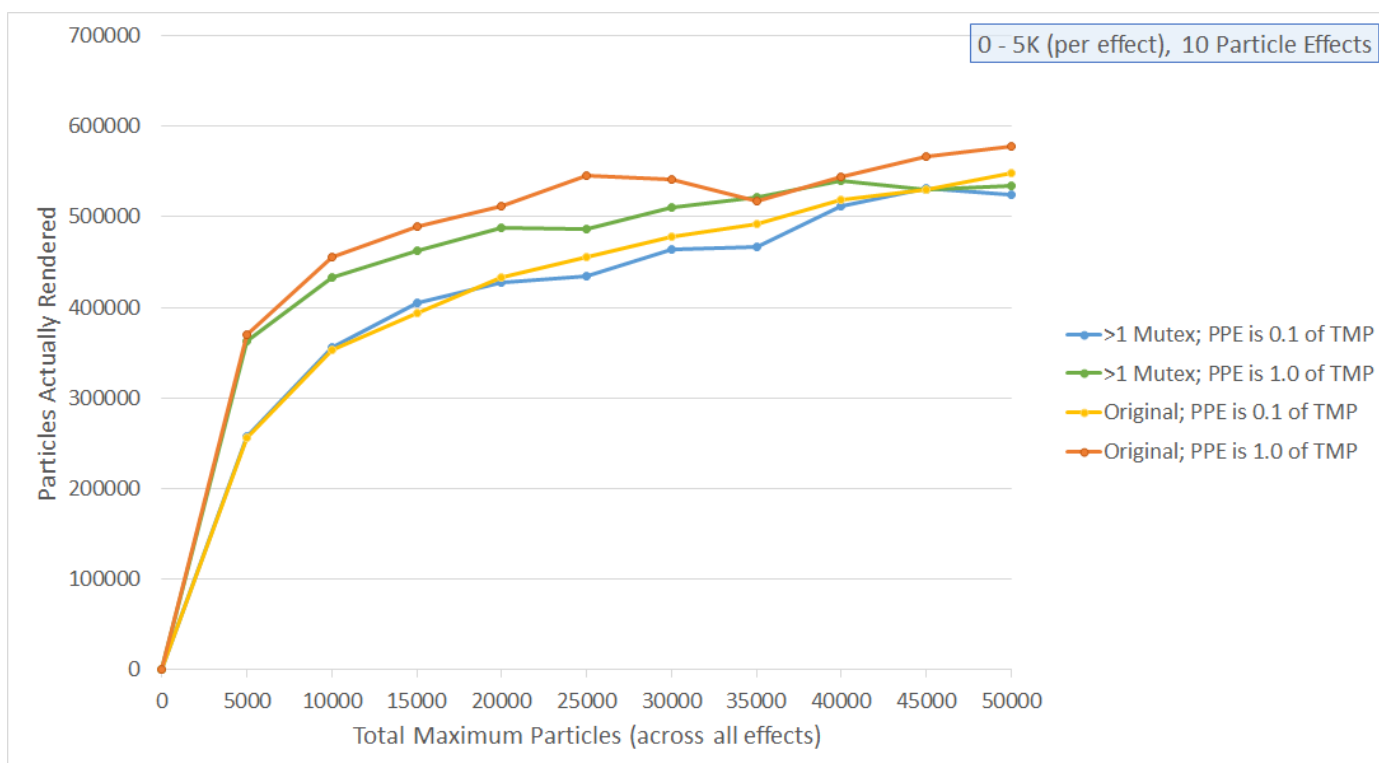


Figure 5.13 A graph showing *particles actually rendered* over *total maximum particles* for ten particle effects.

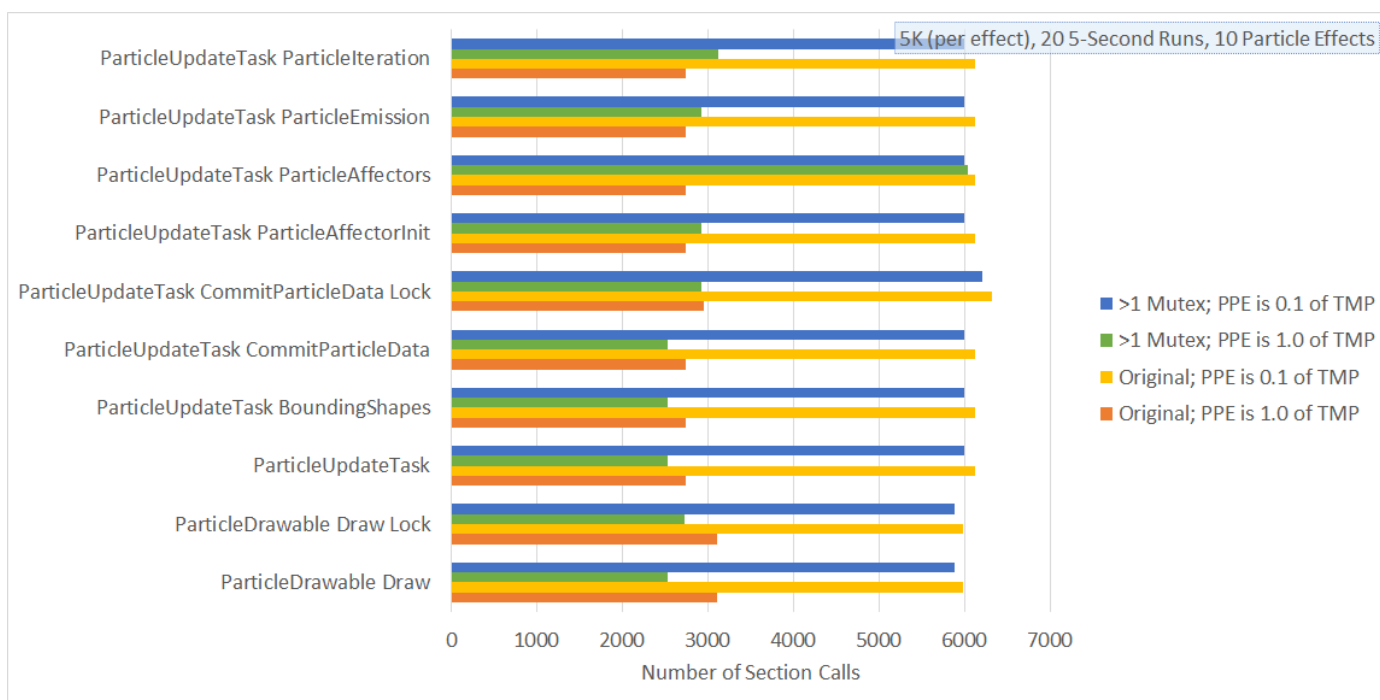


Figure 5.14 A graph showing the number of section calls for various sections of code for ten particle effects.

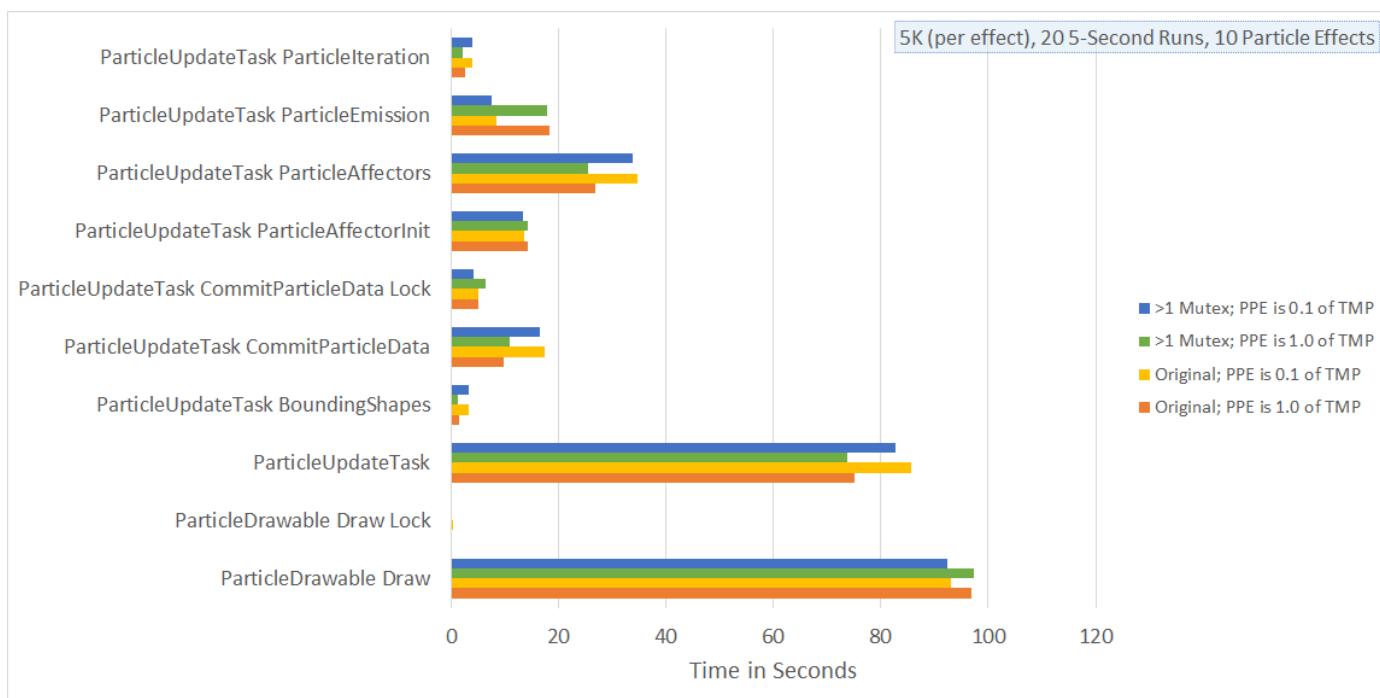


Figure 5.15 A graph showing the time spent in various sections of code for ten particle effects.

5.4.2 Lock Free Results

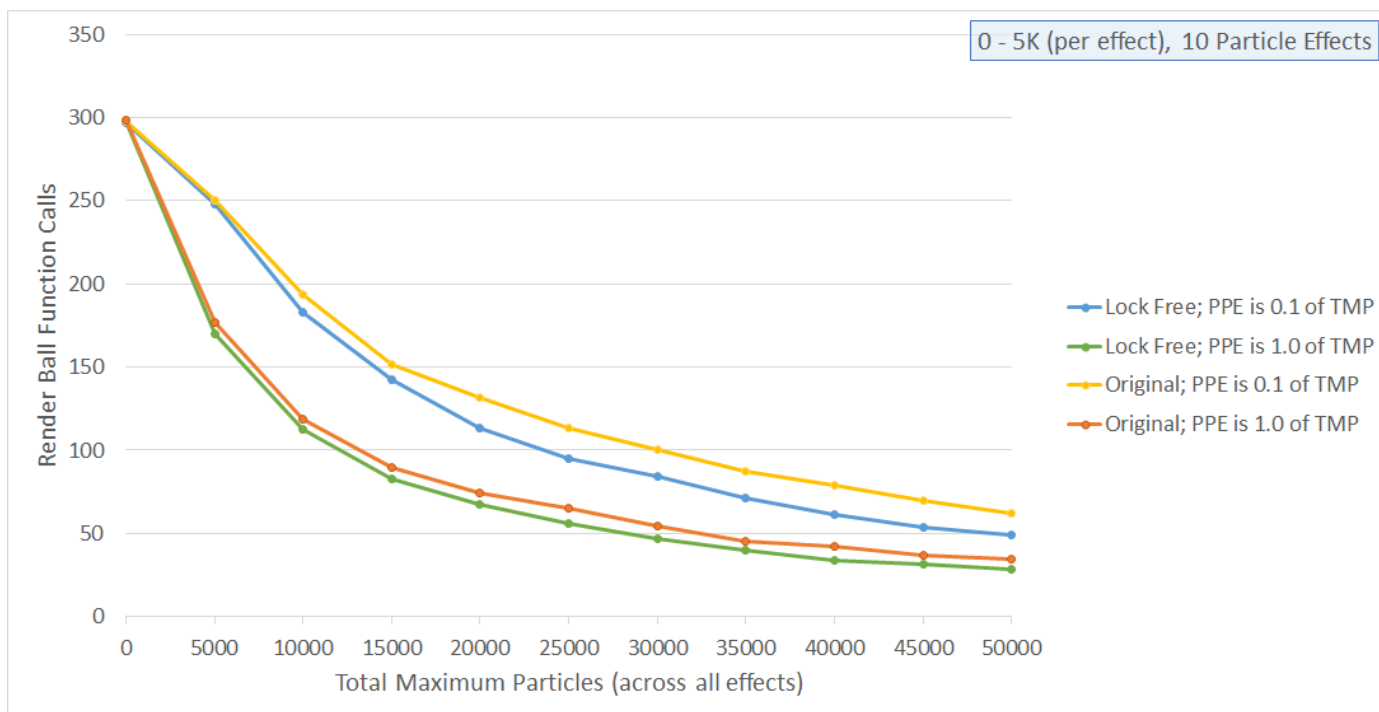


Figure 5.16 A graph showing *render ball function calls* over *total maximum particles* for ten particle effects.

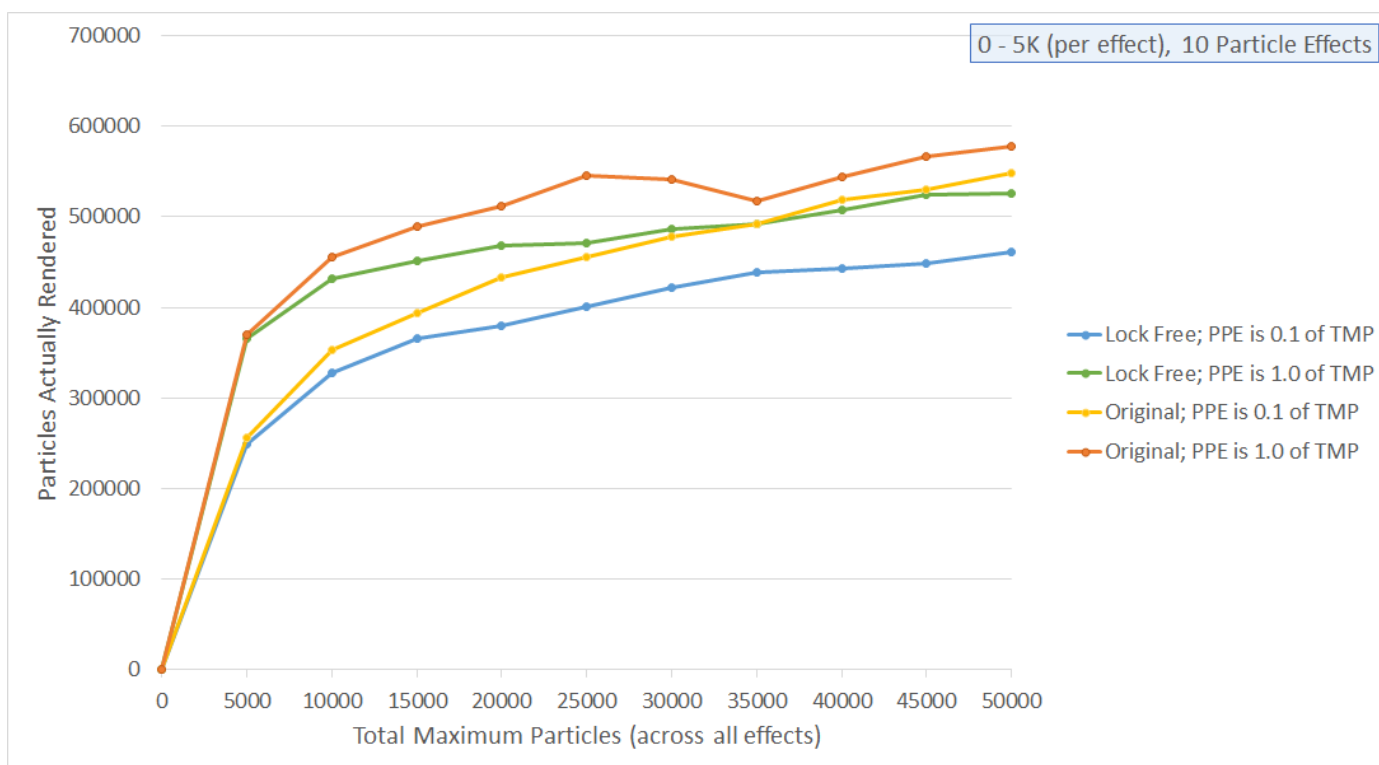


Figure 5.17 A graph showing *particles actually rendered* over *total maximum particles* for ten particle effects.

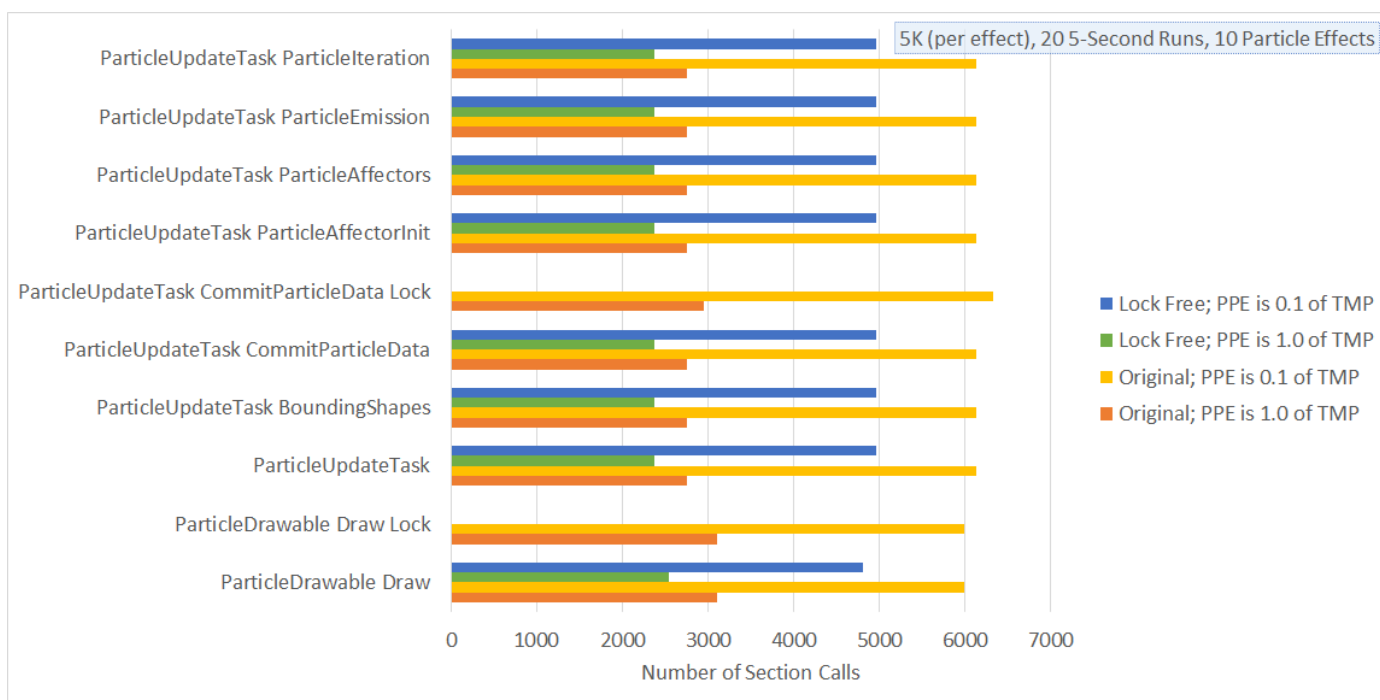


Figure 5.18 A graph showing the number of section calls for various sections of code for ten particle effects.

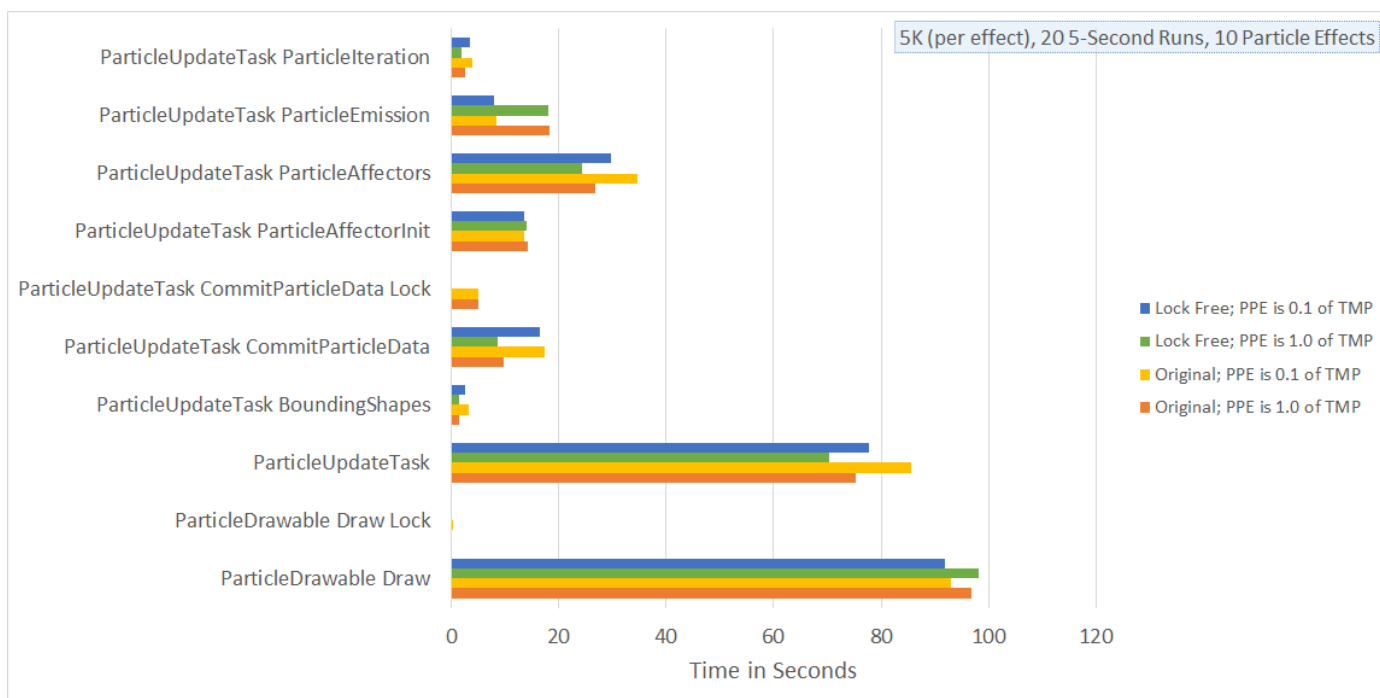


Figure 5.19 A graph showing the time spent in various sections of code for ten particle effects.

5.4.3 Discussion

The most interesting (and, perhaps, frustrating) part of these results is that all of the data series that "played the same game" look essentially the same. The ball is rendered nearly as many times for the four scenarios for both case studies, and the *particles actually rendered* are also similar for both as well. However, just like in the Lock Free case study results, there are slightly less particles rendered than in the original. To try to understand what happened, let us examine figures 5.20 and 5.21. In particular, let us focus on the "1.0 TMP" scenario with the Original and Lock Free implementations.

Within the 1.0 TMP Original results for a single particle effect, about 50 of the 104⁵ seconds spent in the function **ParticleUpdateTask** are dedicated to calling **CommitParticleData**. Further, about 46 of these 50 seconds in **CommitParticleData** are spent waiting for a lock to release. Within the 1.0 TMP Lock Free results for a single particle effect, about 7 of the 58 seconds spent in the function **ParticleUpdateTask** are spent calling **CommitParticleData**.

Within the 1.0 TMP Original results for ten particle effects, about 9 of the 75 seconds spent in the function **ParticleUpdateTask** are dedicated to calling **CommitParticleData**. Further, about 5 of these 9 seconds in **CommitParticleData** are spent waiting for a lock to release. Within the 1.0 TMP Lock Free results for a single particle effect, about 9 of the 75 seconds spent in the function **ParticleUpdateTask** are spent calling **CommitParticleData**.

For a single effect, we see that the Lock Free implementation does an excellent job in reducing contention for a large data structure between a background thread writing to it and a main thread reading from it. For ten effects, the Lock Free implementation does little to reduce the contention because there was no contention to begin with. This can be observed by the fact that the results from the Original implementation with ten effects show that, compared with a single effect, an insignificant amount of time is spent waiting

⁵It may not make sense why there are greater than 100 seconds here since we ran 20 runs for 5 seconds, but it is mostly due to the timer starting only when the ball is activated after the entire scene has been created.

Lock Free, 0.1 TMP, 1 effect w/ 50K each, 20 5-second runs			Lock Free, 1.0 TMP, 1 effect w/ 50K each, 20 5-second runs		
timed section name	time in seconds	num section calls	timed section name	time in seconds	num section calls
ParticleDrawable Draw	97.91	640	ParticleDrawable Draw	93.38	762
ParticleDrawable Draw Lock	0.00	0	ParticleDrawable Draw Lock	0.00	0
ParticleUpdateTask	66.25	458	ParticleUpdateTask	57.55	222
ParticleUpdateTask CommitParticleData	14.75	458	ParticleUpdateTask CommitParticleData	7.13	222
ParticleUpdateTask CommitParticleData Lock	0.00	0	ParticleUpdateTask CommitParticleData Lock	0.00	0
Original, 0.1 TMP, 1 effect w/ 50K each, 20 5-second runs			Original, 1.0 TMP, 1 effect w/ 50K each, 20 5-second runs		
timed section name	time in seconds	num section calls	timed section name	time in seconds	num section calls
ParticleDrawable Draw	88.69	700	ParticleDrawable Draw	86.35	781
ParticleDrawable Draw Lock	0.05	700	ParticleDrawable Draw Lock	0.10	781
ParticleUpdateTask	103.70	561	ParticleUpdateTask	103.89	235
ParticleUpdateTask CommitParticleData	47.18	561	ParticleUpdateTask CommitParticleData	50.08	235
ParticleUpdateTask CommitParticleData Lock	38.59	581	ParticleUpdateTask CommitParticleData Lock	46.37	255
>1 Mutex, 0.1 TMP, 1 effect w/ 50K each, 20 5-second runs			>1 Mutex, 1.0 TMP, 1 effect w/ 50K each, 20 5-second runs		
timed section name	time in seconds	num section calls	timed section name	time in seconds	num section calls
ParticleDrawable Draw	92.61	776	ParticleDrawable Draw	89.14	771
ParticleDrawable Draw Lock	0.03	776	ParticleDrawable Draw Lock	0.03	771
ParticleUpdateTask	62.36	358	ParticleUpdateTask	65.69	145
ParticleUpdateTask CommitParticleData	24.31	358	ParticleUpdateTask CommitParticleData	31.10	145
ParticleUpdateTask CommitParticleData Lock	18.89	378	ParticleUpdateTask CommitParticleData Lock	29.05	165

Figure 5.20 A summary of the timed sections for the unexpected results for the Lock Free and > 1 Mutex case studies for a single particle effect.

Lock Free, 0.1 TMP, 10 effects w/ 5K each, 20 5-second runs			Lock Free, 1.0 TMP, 10 effects w/ 5K each, 20 5-second runs		
timed section name	time in seconds	num section calls	timed section name	time in seconds	num section calls
ParticleDrawable Draw	91.96	4810	ParticleDrawable Draw	98.20	2550
ParticleDrawable Draw Lock	0.00	0	ParticleDrawable Draw Lock	0.00	0
ParticleUpdateTask	77.69	4963	ParticleUpdateTask	70.32	2381
ParticleUpdateTask CommitParticleData	16.48	4963	ParticleUpdateTask CommitParticleData	8.69	2381
ParticleUpdateTask CommitParticleData Lock	0.00	0	ParticleUpdateTask CommitParticleData Lock	0.00	0
Original, 0.1 TMP, 10 effects w/ 5K each, 20 5-second runs			Original, 1.0 TMP, 10 effects w/ 5K each, 20 5-second runs		
timed section name	time in seconds	num section calls	timed section name	time in seconds	num section calls
ParticleDrawable Draw	92.98	5990	ParticleDrawable Draw	96.90	3107
ParticleDrawable Draw Lock	0.27	5990	ParticleDrawable Draw Lock	0.12	3107
ParticleUpdateTask	85.61	6128	ParticleUpdateTask	75.21	2748
ParticleUpdateTask CommitParticleData	17.48	6128	ParticleUpdateTask CommitParticleData	9.83	2748
ParticleUpdateTask CommitParticleData Lock	5.00	6328	ParticleUpdateTask CommitParticleData Lock	5.02	2948
>1 Mutex, 0.1 TMP, 10 effects w/ 5K each, 20 5-second runs			>1 Mutex, 1.0 TMP, 10 effects w/ 5K each, 20 5-second runs		
timed section name	time in seconds	num section calls	timed section name	time in seconds	num section calls
ParticleDrawable Draw	92.40	5880	ParticleDrawable Draw	97.33	2920
ParticleDrawable Draw Lock	0.21	5880	ParticleDrawable Draw Lock	0.11	2920
ParticleUpdateTask	82.74	6005	ParticleUpdateTask	73.85	2535
ParticleUpdateTask CommitParticleData	16.43	6005	ParticleUpdateTask CommitParticleData	10.99	2535
ParticleUpdateTask CommitParticleData Lock	4.28	6205	ParticleUpdateTask CommitParticleData Lock	6.47	2735

Figure 5.21 A summary of the timed sections for the unexpected results for the Lock Free and > 1 Mutex case studies for ten particle effects.

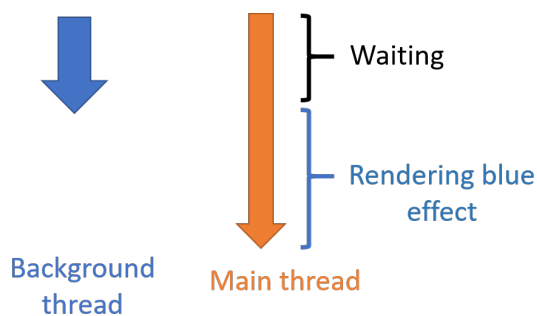


Figure 5.22 The lifetime of a single particle effect in relation to its scheduled background thread and the main thread.

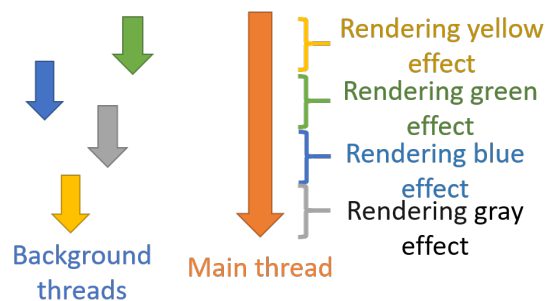


Figure 5.23 The lifetime of a many particle effects in relation to their scheduled background threads and the main thread.

for a lock to release.

This behavior can be better understood if we consider the lifetime of particle effects in two scenarios: one with many effects, and one with a single effect. The one with many particle effects would have its particles distributed across many *Particle Effect Components* and, thus, many *Particle Drawables*. The *Renderer* running on the main thread, then, will render each *Particle Effect Component* separately. When the main thread requests to lock any given effect's particle array mutex, it has a reduced chance of waiting for it. This is because each *Particle Effect Component* has its own background thread and each one is only responsible for a fraction of the particles. The one with a single particle effect, on the other hand, has one *Particle Effect Component* with a large array of particles that it has to manage on its own. It can only schedule one background thread at a time and is responsible for updating every single particle. When the *Renderer* running on the main thread requests to lock the effect's particle array mutex, it has a greater chance of waiting for it.

The unexpected results of these case studies demonstrated the differences between managing contention for a single, large data structure and for many pieces or "shards" of a fragmented data structure. The former is likely to have contention if there is writing involved between two threads, and a solution that utilizes "lock free" atomic read-modify-write operations can significantly reduce contention. The latter, however, is unlikely to

have contention between two threads, and so the developer can get away with naively using mutexes to synchronize their data.

CHAPTER 6 CHILLISOURCE PARTICLE EFFECT COMPONENT OBSERVATIONS

6.1 Overview

The case studies that were performed focused primarily on understanding the contention between the main thread that rendered particles and the background thread that updated them. This chapter will explore various other aspects of *Particle Effect Components*. Topics that will be examined include the importance of the *ConcurrentParticleData* object, the relationship between ChilliSource's *Task Scheduler* and *Particle Effect Components*, and the perplexing scenarios in which particle effects do not visually emit particles.

6.2 Copying Data with ConcurrentParticleData

6.2.1 Overview

As discussed in prior chapters, the *ConcurrentParticleData* object contains copies of primitives, objects, and data structures needed by the *Particle Drawable* to render the particles. It also handles all of the locking necessary in order to synchronize the background and main threads. This section will address what occurs when *ConcurrentParticleData* is not used and the member variables within *Particle Effect Component* are instead directly used by the *Particle Drawable*.

6.2.2 Code

There are a considerable number of changes that are needed to remove *ConcurrentParticleData* from the *Particle Effect Component* and *Particle Drawable* classes:

- Modify the *Particle Drawable* to use pointers from the *Particle Effect Component* and remove the *ConcurrentParticleData* member variable.
- Add variables that *ConcurrentParticleData* was keeping track of.
- Add functions that *ConcurrentParticleData* provided. (Which primarily include accessor and locking/unlocking functions)
- Modify **ParticleUpdateTask** to update member variables from the *Particle Effect Component* .
- Modify the *Particle Effect Component* and *Particle Drawable* to utilize locks and mutexes in order to ensure thread safety.

The majority of the above changes are described in Code Listings 19 and 20, but it is also necessary to expand on some points.

In addition to keeping track of information needed for rendering, *ConcurrentParticleData* has two booleans that the *Particle Effect Component* used for scheduling updates: *hasActiveParticles* and *isUpdating*. These booleans describe whether or not there are any particles active in the effect and if **ParticleUpdateTask** is running, respectively. If *hasActiveParticles* is false when the component is updating on the main thread, then the component stops¹. If *isUpdating* is false when the component is updating on the main thread, then the component will schedule a **ParticleUpdateTask**. In order to replace this functionality from *ConcurrentParticleData*, these booleans are added to *Particle Effect Component* as atomic booleans that are passed by reference to the **ParticleUpdateTask** function.²

Since the *Particle Effect Component* uses the bounding information objects on the main thread and the *Drawable* does not, two mutexes will be created for the bounding information and the rest of the rendering data.

As shown by Code Listing 19, the **ParticleUpdateTask** is locked by a member variable mutex. It is also passes references to new particle indices (i.e. *copiedAtts.newParticleIndices*)

¹The particle effect will only completely stop if *looping* is false as well.

²These booleans are not shown in the pseudocode for simplicity, but their behavior is simple; *hasActiveParticles* will switch to false if it did not update any particles in **ParticleUpdateTask**, and *isUpdating* will switch to false at the end of **ParticleUpdateTask**.

and to the bounding information (i.e. *copiedAtts.AABB* and *copiedAtts.Sphere*).

Code Listing 19: Pseudocode describing the important changes needed to remove the *ConcurrentParticleData* instance from the *Particle Effect Component*.

```

1 ParticleEffectComponent::OnUpdate()
2 {
3     // Get a struct containing copies (and pointers) to pass to the task
4     copiedAtts = CopyUpdateAttributes();
5
6     // Schedule a task to update the particle in the background
7     ApplicationTaskScheduler.ScheduleTask
8     ({
9         // Lock the mutex for the whole task
10        using lock(this.mutex)
11        {
12            ParticleUpdateTask(copiedAtts);
13        }
14    });
15 }
16
17 ParticleUpdateTask(copiedAtts)
18 {
19     // Update particles if they are active
20     for (particle in copiedAtts.particleArray)
21     {
22         if (particle.isActive)
23             particle.UpdateValues();
24     }
25
26     // Apply affectors
27     for (affector in copiedAtts.particleAffectors)
28     {
29         affector.AffectParticles();
30     }
31
32     // Try to emit
33     newParticleIndices = copiedAtts.particleEmitter.EmitParticles();
34
35     // Append the new indices to the member new indices array
36     copiedAtts.newParticleIndices.append( newParticleIndices.copy() );
37
38     // Update member bounding shapes; lock the bounding shapes mutex
39     using lock(copiedAtts.boundingMutex)
40     {
41         copiedAtts.AABB, copiedAtts.Sphere = CalculateBoundingShapes();
42     }
43 }

```

Code Listing 20 shows that the *Particle Drawable* did not change significantly. The most notable change is that the **Draw** function is now locked by *Particle Effect Component*'s

mutex within the **Render** function. Additionally, the particles array is not passed to the **DrawParticles** function. Instead, the array is passed into the drawable when it is created by the *Particle Effect Component*.

Code Listing 20: Pseudocode describing the important changes needed to remove the *ConcurrentParticle-Data* instance from the *Particle Drawable*.

```

1 ParticleEffectComponent::Render(camera)
2 {
3     // Use the Particle Drawable to draw particles to the screen
4     if(this.playbackState is playing or this.playbackState is stopping)
5     {
6         // Lock the member mutex to keep particles and new particle indices safe
7         using lock(this.mutex)
8         {
9             this.drawable.Draw(camera, this.TakeNewParticleIndices());
10        }
11    }
12 }
13
14 Drawable::Draw(camera, newParticleIndices)
15 {
16     // Activate the newly emitted particles in the static billboard
17     for(particleIndex in newParticleIndices)
18     {
19         ActivateParticleInBillboard(particleIndex);
20     }
21
22     // Iterate through all of the particles and draw each one
23     // The particles array is a member variable of the drawable
24     DrawParticles(camera);
25 }

```

6.2.3 Results

The metrics that are presented are the result of running a series of automated games in four scenarios:

1. 1 particle effect, particles range from 0 - 50K, 5K step, 5 5-second runs per game.
2. 1 particle effect, 50K, 0 step, 20 5-second runs.
3. 10 particle effects, particles range from 0 - 5K, 500 step, 5 5-second runs per game.
4. 10 particle effects, 5K, 0 step, 20 5-second runs.

All of these scenarios were ran with PPE values that were 10% and 100% of TMP. These eight series of automated games were ran with and without the above changes to the source

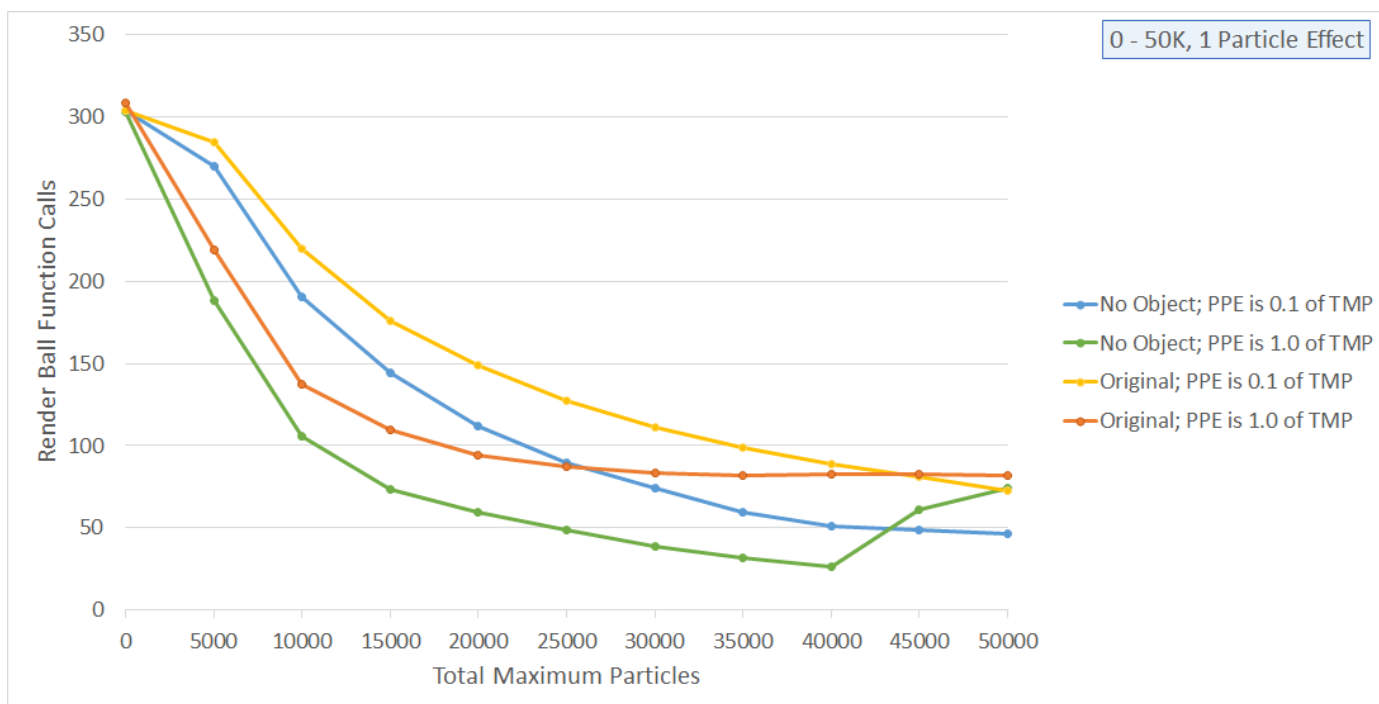


Figure 6.1 A graph showing *render ball function calls* over *total maximum particles* for a single particle effect.

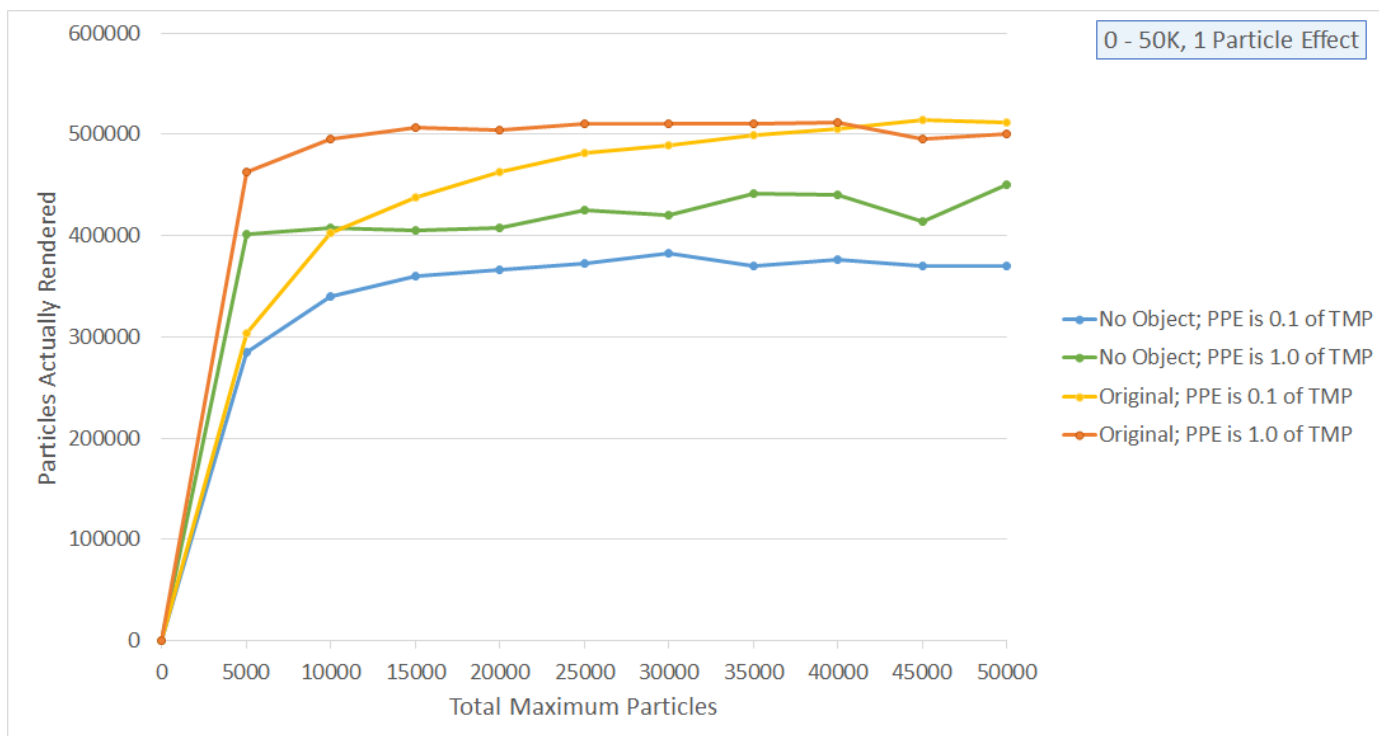


Figure 6.2 A graph showing *particles actually rendered* over *total maximum particles* for a single particle effect.

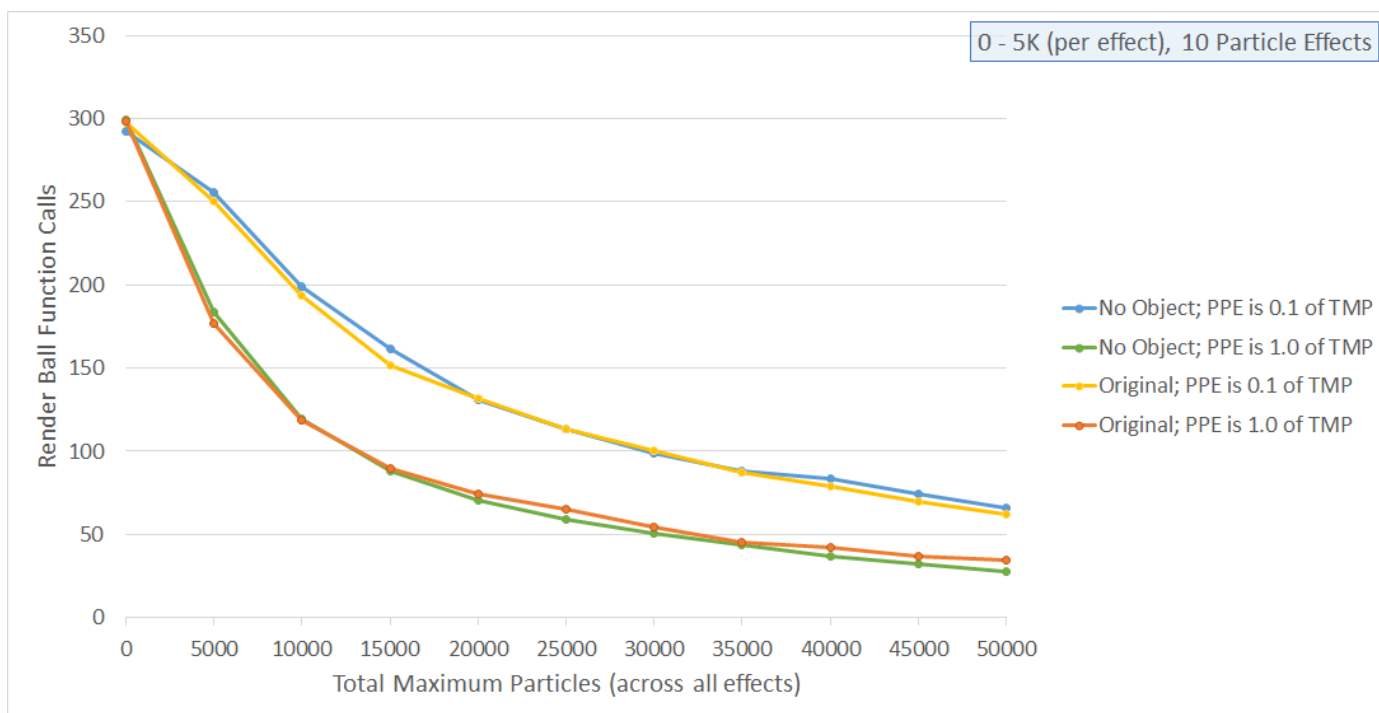


Figure 6.3 A graph showing *render ball function calls* over *total maximum particles* for ten particle effects.

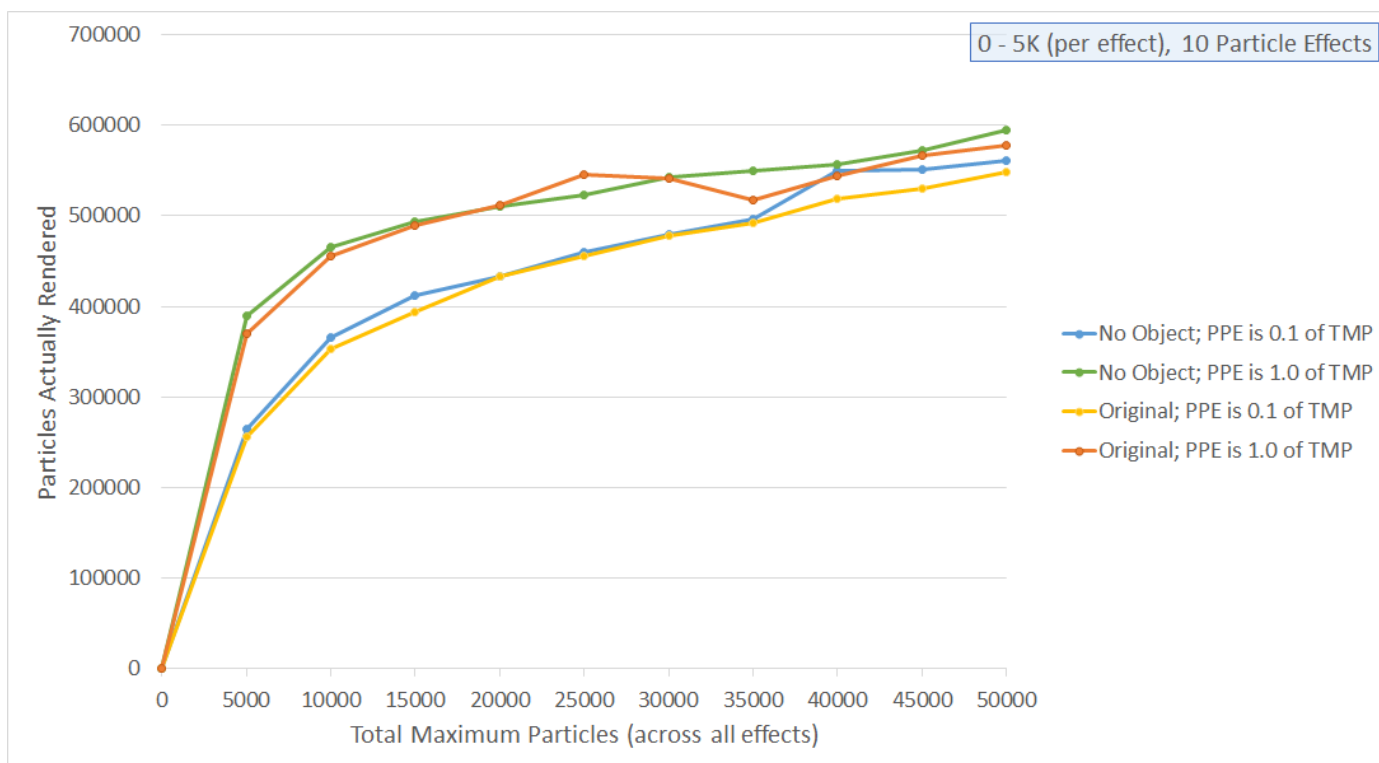


Figure 6.4 A graph showing *particles actually rendered* over *total maximum particles* for ten particle effects.

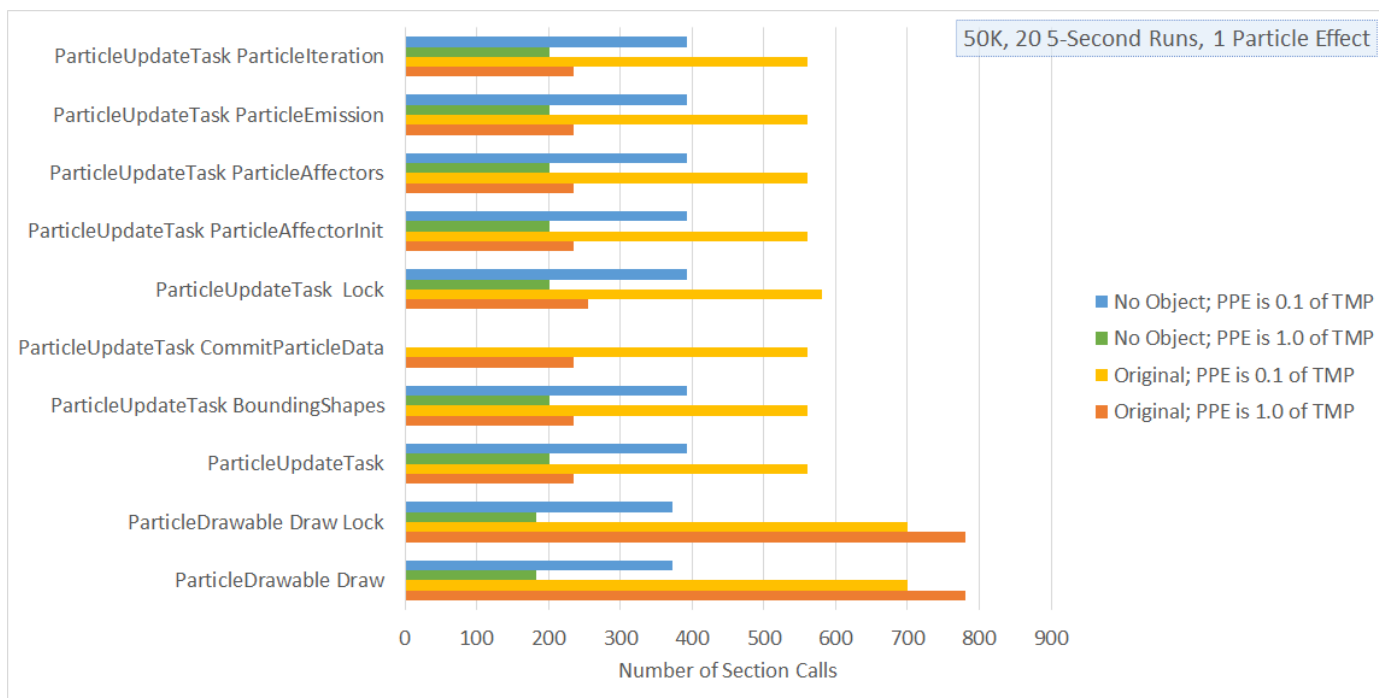


Figure 6.5 A graph showing the number of section calls for various sections of code for a single particle effect.

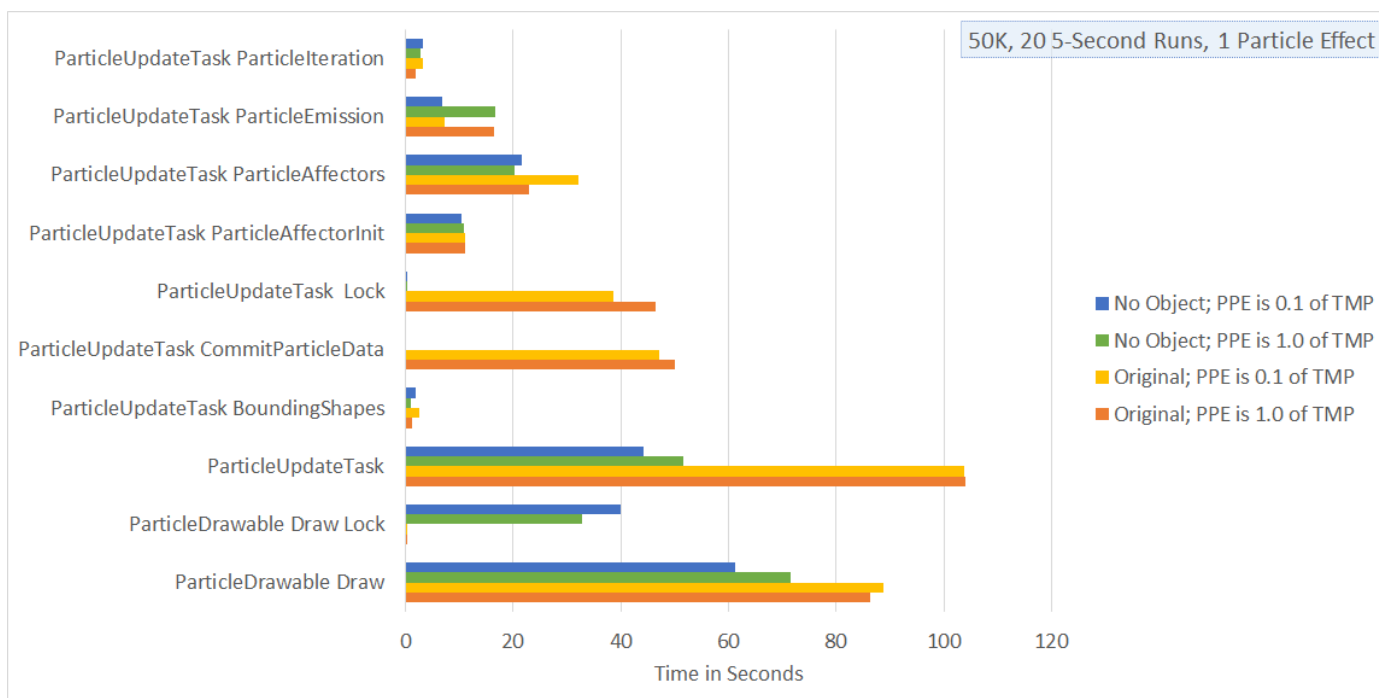


Figure 6.6 A graph showing the time spent in various sections of code for a single particle effect.

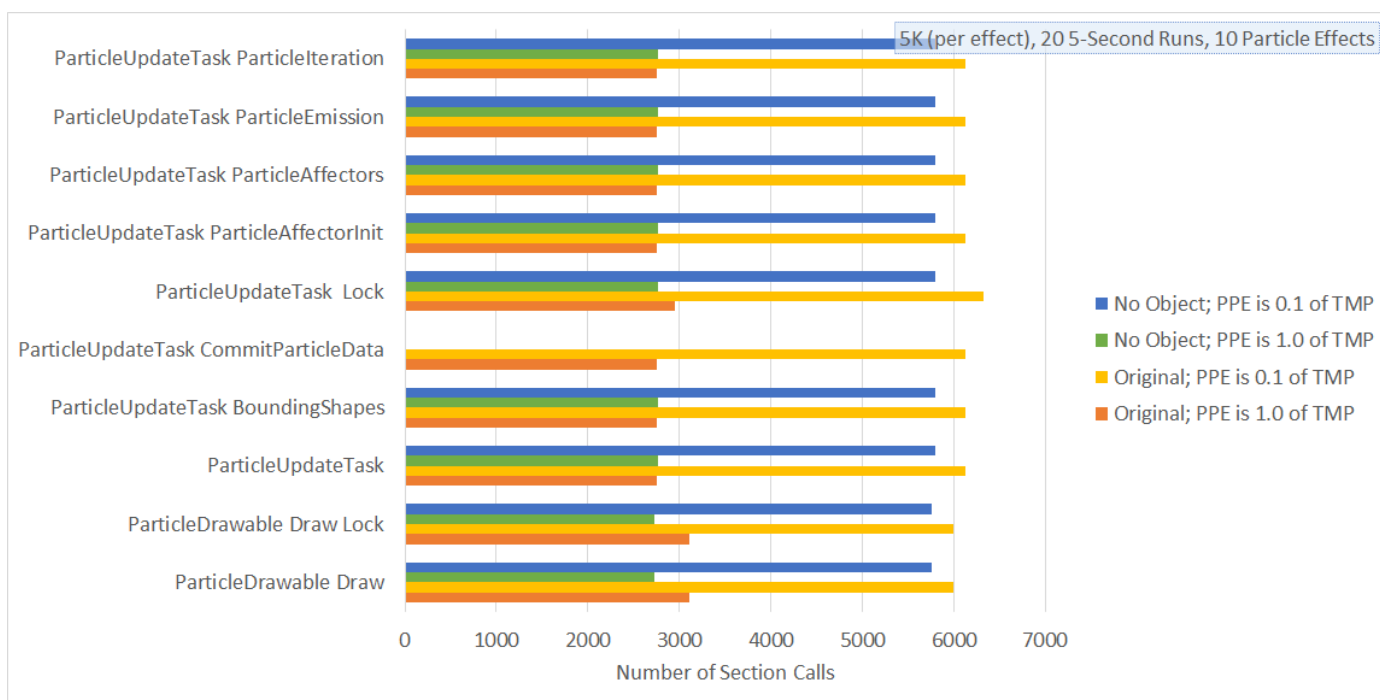


Figure 6.7 A graph showing the number of section calls for various sections of code for ten particle effects.

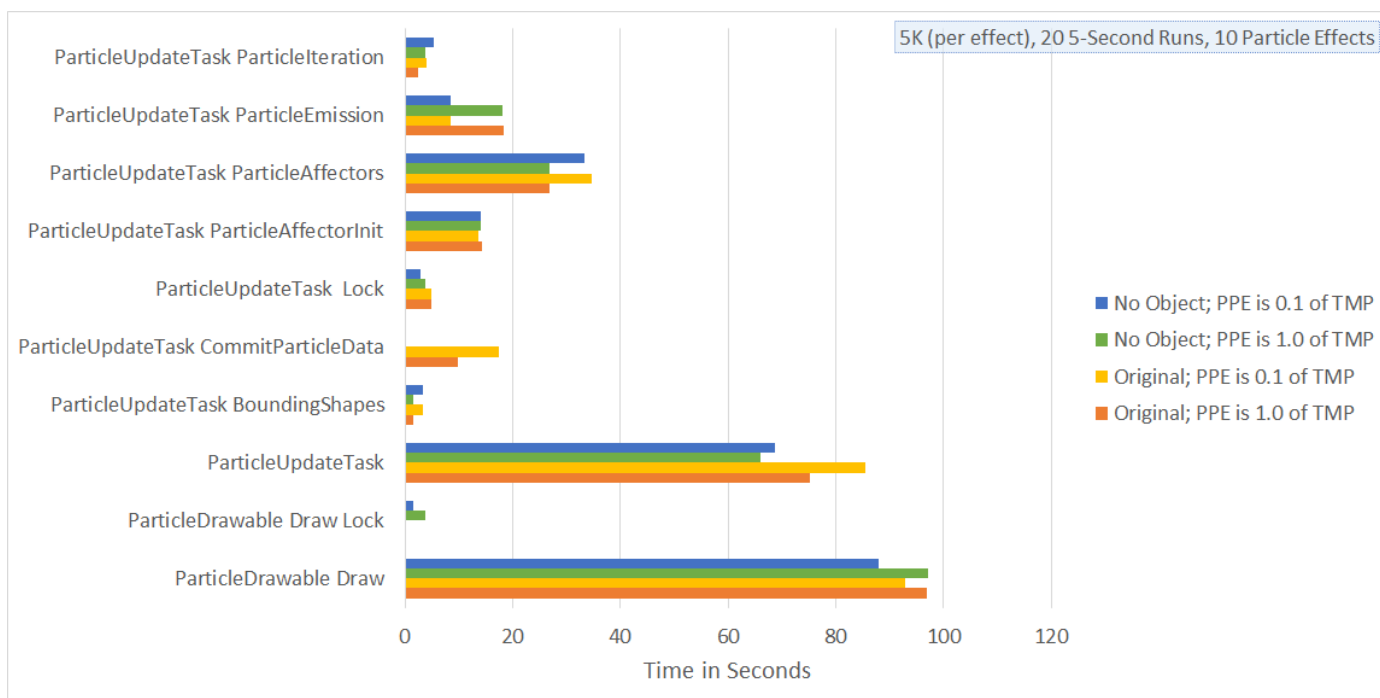


Figure 6.8 A graph showing the time spent in various sections of code for ten particle effects.

code, and this totals to 16 series of automated games.

6.2.4 Discussion

The *particles actually rendered* and *render ball function calls* charts for 1 particle effect show something interesting. The "No Object" data series renders the ball fewer times than the original, and it also renders its particles fewer times. An examination of the timing metrics for 1 particle effect shows why this is occurring. Instead of the background thread waiting for the mutex to release in **ParticleUpdateTask**, the contention has moved to the *Particle Drawable* in the **Draw** function. This increase in wait-time disallows the renderer to render as many particles and, as a result, it also takes away rendering time from everything else (including the ball).

The charts for 10 particle effects, however, do not demonstrate this. Both the original and "No Object" data series show nearly identical results. This is because there is less overall contention when the particles are distributed across effects, as discovered in the previous chapter. Although the benefits of using a *ConcurrentParticleData* object is not as significant when using many particle effects, it is appealing from a software engineering perspective. Encapsulating the thread synchronization to a single class is clearer and more maintainable, and that will be useful if the developer desires to refactor the *ParticleEffectComponent*.

6.3 Multithreading and Task Scheduling

6.3.1 The ChilliSource Task Scheduler

As mentioned earlier in the paper, **ParticleUpdateTask** is scheduled as a background thread by ChilliSource's *Task Scheduler*. The *Task Scheduler* can schedule a variety of task types, and this section will delve into the different outcomes when **ParticleUpdateTask** is scheduled as a *small task*, a *game logic task*, and if it is not scheduled at all and runs on the main thread.

Code Listing 21: Pseudocode showing how ChilliSource's *Task Scheduler* schedules *small tasks*, *game logic tasks*, and *main thread tasks*.

```

1 TaskScheduler::ScheduleTask(taskType, task)
2 {
3     switch(taskType)
4     {
5         // Add a task to the small thread task pool
6         case TaskType::small:
7             this.smallTaskPool.AddTask(task); break;
8
9         // Add a task to the main thread task pool
10        case TaskType::mainThread:
11            this.mainThreadTaskPool.AddTask(task); break;
12
13        // Schedule the task as a small task, but be sure to notify when
14        // all game logic tasks have been completed
15        case TaskType::gameLogic
16            ScheduleTask(TaskType::small,
17            {
18                task();
19                if(--this.gameLogicTaskCount is 0)
20                    this.gameLogicCondition.notify();
21            });
22
23            this.gameLogicTaskCount++;
24
25            break;
26
27        // File and large tasks can also be scheduled, but they are outside
28        // of this paper's scope
29    }
30 }

```

A *small task* is considered to be a task with a short execution time such that it can be processed within a single frame. If a *small task* runs too long, then the task will have a significantly negative impact on the overall performance of the program. A *game logic task* is a special kind of small task that is guaranteed to be executed prior *main thread tasks*. It is easy to see, then, that long running *game logic tasks* will drastically affect the program's performance. *Main thread tasks* are also small tasks that are executed on the main thread after *game logic tasks*. All of these tasks are added to *Task Pools*, which are a collection of tasks that are performed by one of the worker threads owned by the pool. These pools will continue to perform tasks until they are deallocated and, if there are no tasks to perform, they will sleep until another task is added.

Code Listing 22: Pseudocode showing how *game logic tasks* force the main thread to wait until they are finished before allowing the main thread to continue.

```

1 TaskScheduler::ExecuteMainThreadTasks()
2 {
3     // Use a lock, mutex, and condition variable to wait
4     // until we are notified to continue.
5     using lock(this.gameLogicMutex) as lock
6     {
7         while(this.gameLogicTaskCount is not 0)
8             this.gameLogicCondition.wait(lock);
9     }
10
11     this.mainThreadTaskPool.PerformTasks();
12 }

```

6.3.2 Code

Scheduling the different types of tasks is fairly straightforward; the task type is simply passed into the **ScheduleTask** method. This was not included in previous code listings in order to simplify them, but the code listings below illustrate the different ways the **ParticleUpdateTask** function was scheduled for this study. It should be noted that, by default, the *Particle Effect Component* scheduled **ParticleUpdateTask** as a *small task*. Thus, during this study, the "Small Task" data series is analogous to the "Original" data series in previous sections.

6.3.2.1 Small Task Type

Code Listing 23: Pseudocode showing how **ParticleUpdateTask** was scheduled as a *small task*.

```

1 ParticleEffectComponent::OnUpdate()
2 {
3     // Get a struct containing copies (and pointers) to pass to the task
4     copiedAtts = CopyUpdateAttributes();
5
6     // Schedule a small task to update the particle in the background
7     ApplicationTaskScheduler.ScheduleTask(TaskType::small,
8     {
9         ParticleUpdateTask(copiedAtts);
10    });
11 }

```

6.3.2.2 Game Logic Task Type

Code Listing 24: Pseudocode showing how **ParticleUpdateTask** was scheduled as a *game logic task*.

```

1 ParticleEffectComponent::OnUpdate()
2 {
3     // Get a struct containing copies (and pointers) to pass to the task
4     copiedAtts = CopyUpdateAttributes();
5
6     // Schedule a game logic task to update the particle in the background
7     ApplicationTaskScheduler.ScheduleTask(TaskType::gameLogic,
8     {
9         ParticleUpdateTask(copiedAtts);
10    });
11 }

```

6.3.2.3 No Task Scheduled (Not Threaded)

The sequential version of **ParticleUpdateTask** is performed in a similar fashion to the other tasks in that a struct is passed to it and the *ConcurrentParticleData* object is still used. Although these things are not strictly needed, they are included primarily to "level the playing field" so that the results from all three versions are easier to compare.

Code Listing 25: Pseudocode showing how **ParticleUpdateTask** was not scheduled and was simply ran on the main thread.

```

1 ParticleEffectComponent::OnUpdate()
2 {
3     // Get a struct containing copies (and pointers) to pass to the task
4     copiedAtts = CopyUpdateAttributes();
5
6     // Do not schedule a task
7     ParticleUpdateTask(copiedAtts);
8 }

```

6.3.3 Results

The following four scenarios were used when generating metrics for the small task type, the game logic task type, and the series without a task scheduled:

1. 1 particle effect, particles range from 0 - 50K, 5K step, 5 5-second runs per game.
2. 1 particle effect, 50K, 0 step, 20 5-second runs.
3. 10 particle effects, particles range from 0 - 5K, 500 step, 5 5-second runs per game.
4. 10 particle effects, 5K, 0 step, 20 5-second runs.

Similar to prior sections, each of these four scenarios were ran with PPE values that were 10% and 100% of TMP. Thus, this totals to 24 series of games that were executed in order

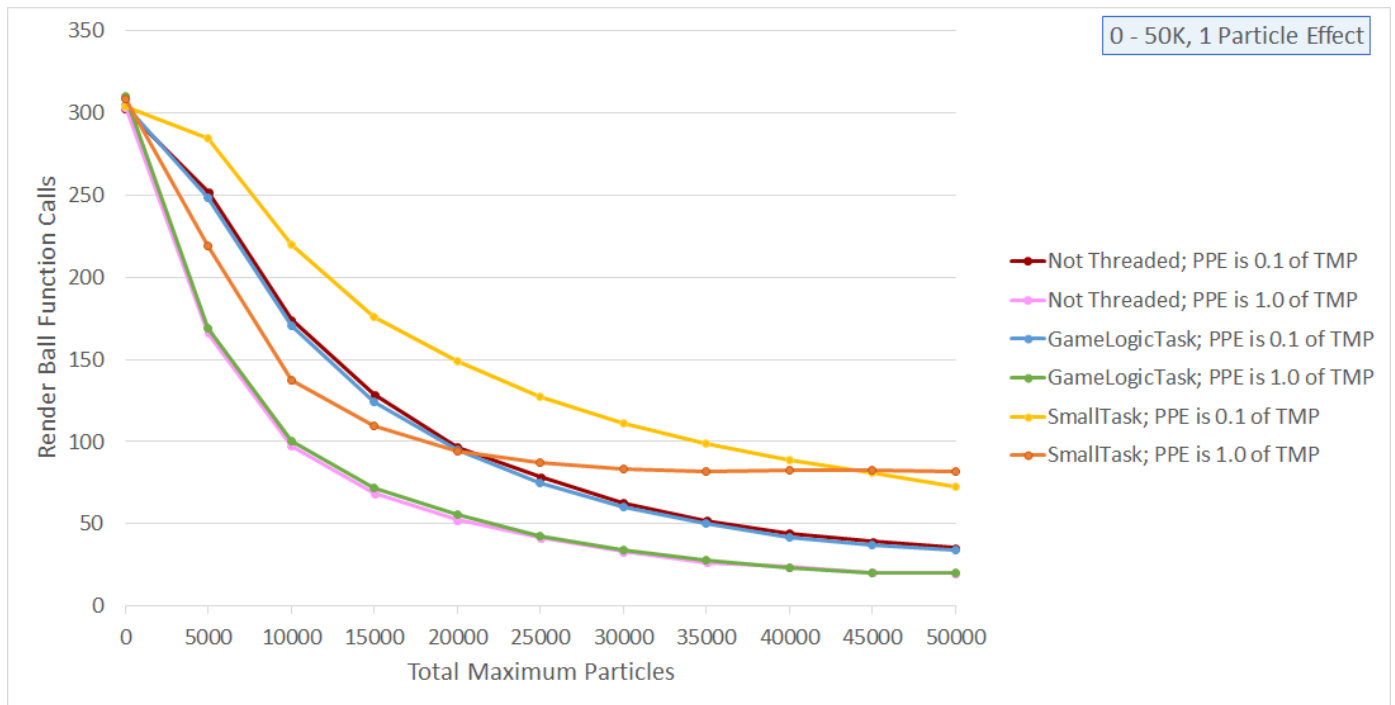


Figure 6.9 A graph showing *render ball function calls* over *total maximum particles* for a single particle effect.

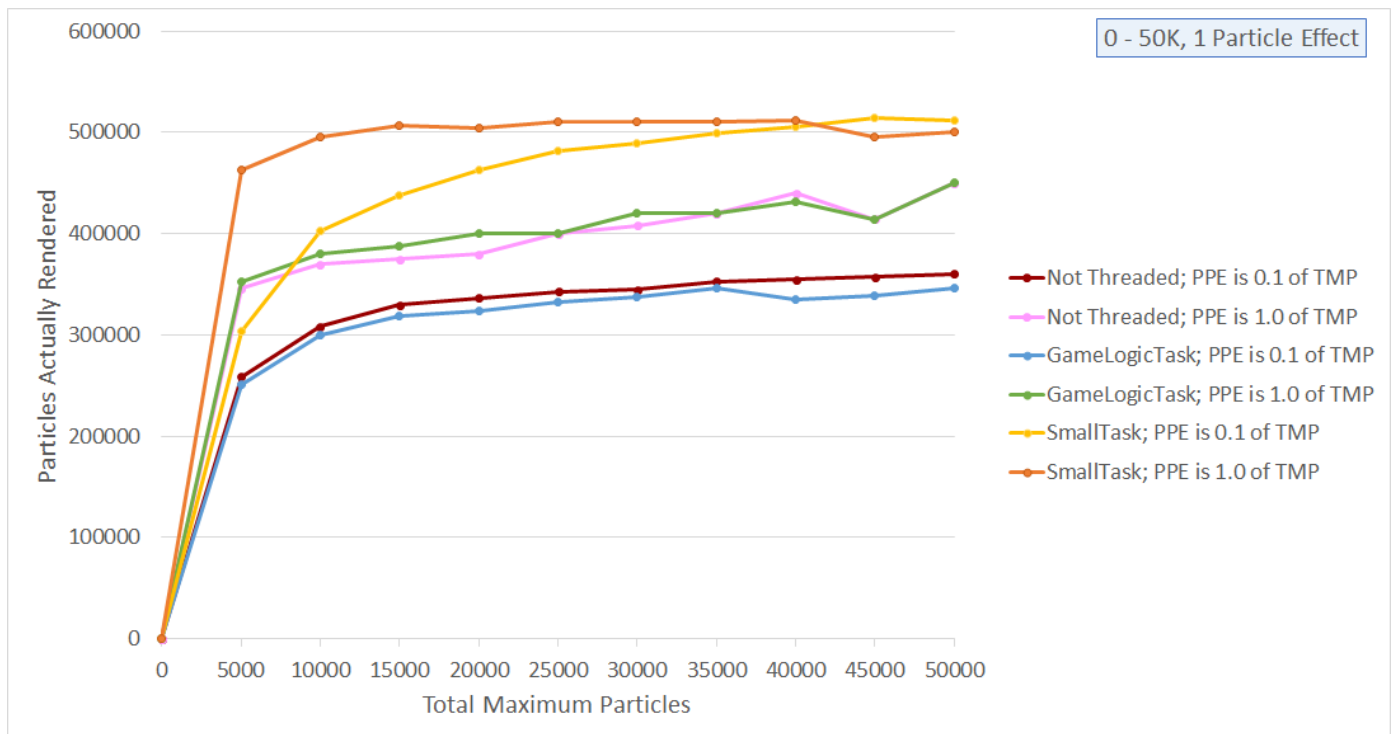


Figure 6.10 A graph showing *particles actually rendered* over *total maximum particles* for a single particle effect.

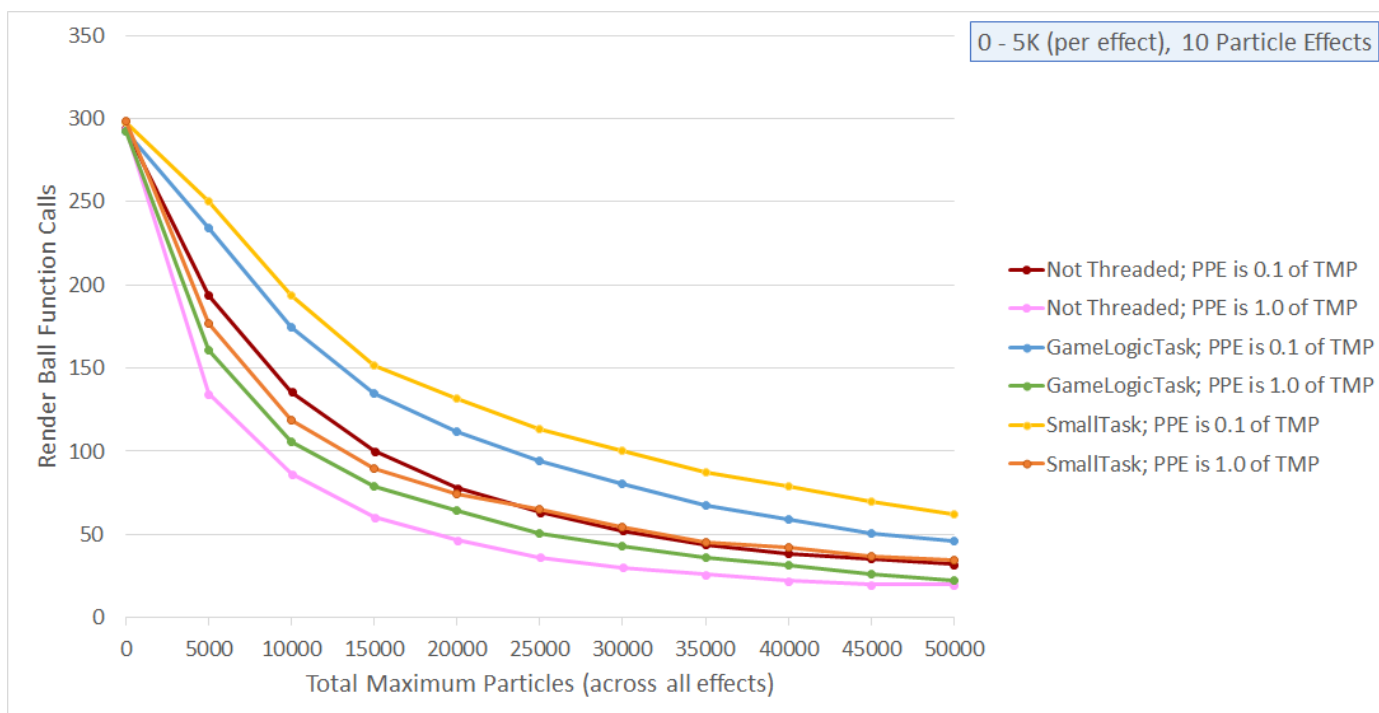


Figure 6.11 A graph showing *render ball function calls* over *total maximum particles* for ten particle effects.

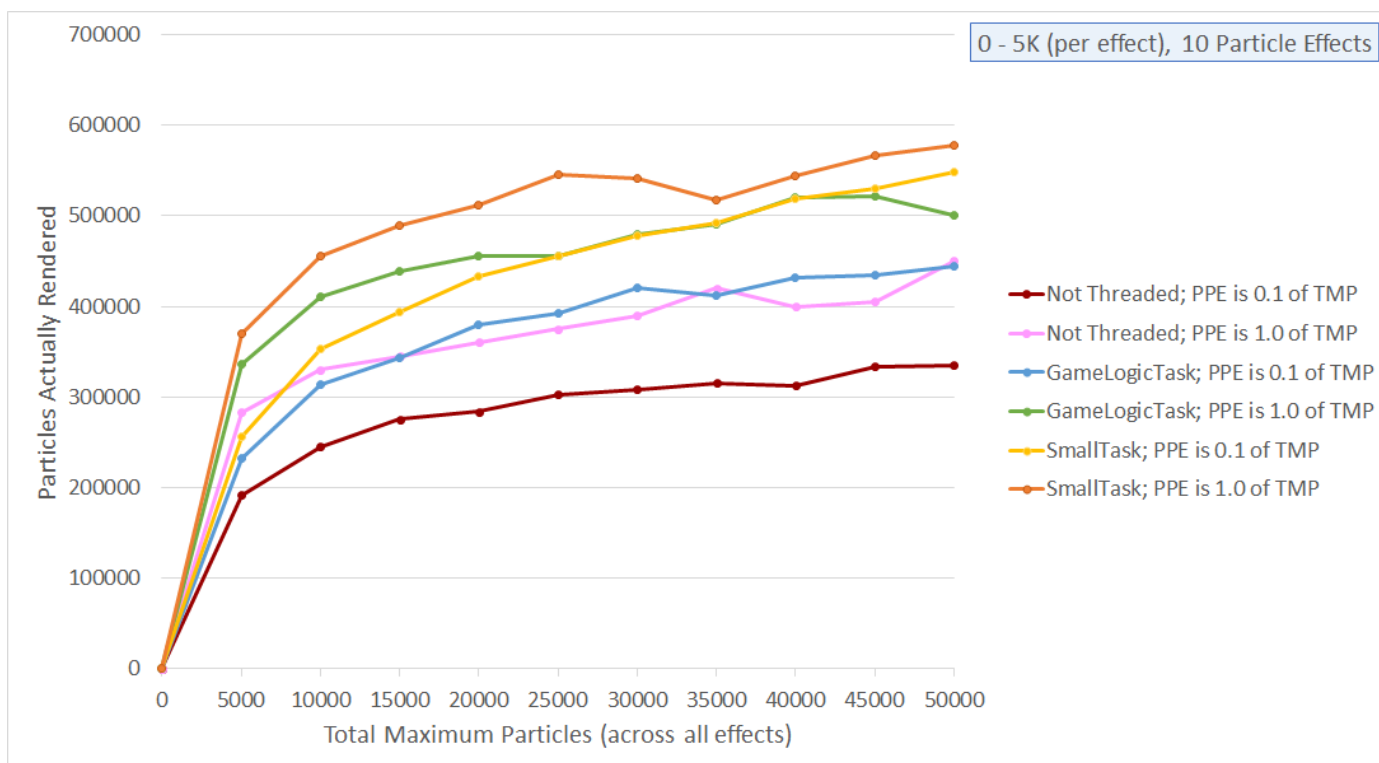


Figure 6.12 A graph showing *particles actually rendered* over *total maximum particles* for ten particle effects.

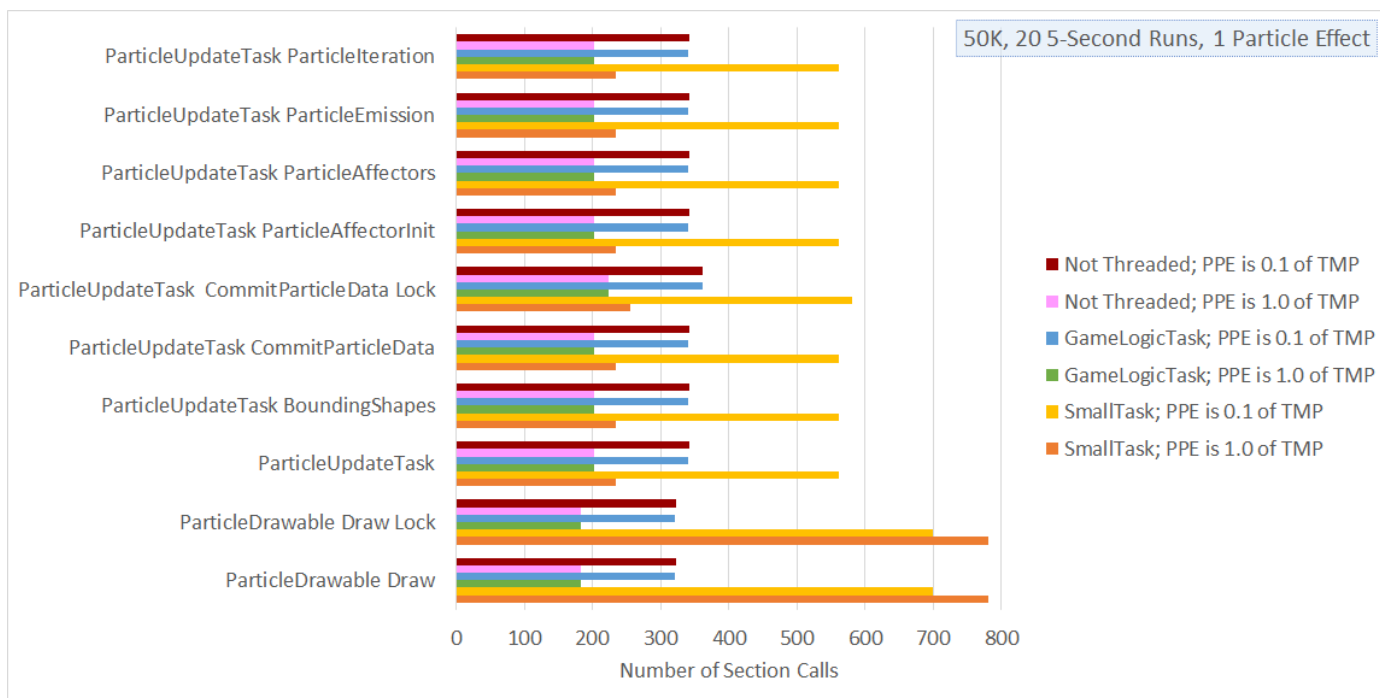


Figure 6.13 A graph showing the number of section calls for various sections of code for a single particle effect.

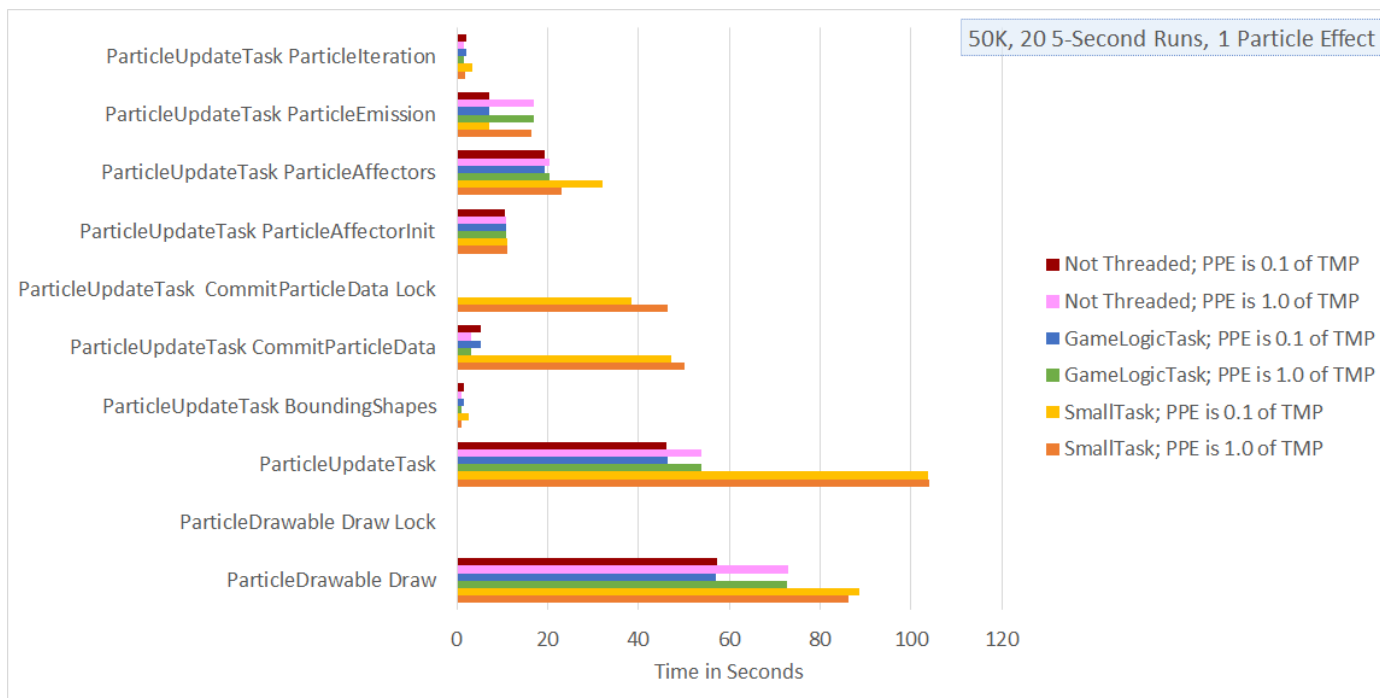


Figure 6.14 A graph showing the time spent in various sections of code for a single particle effect.

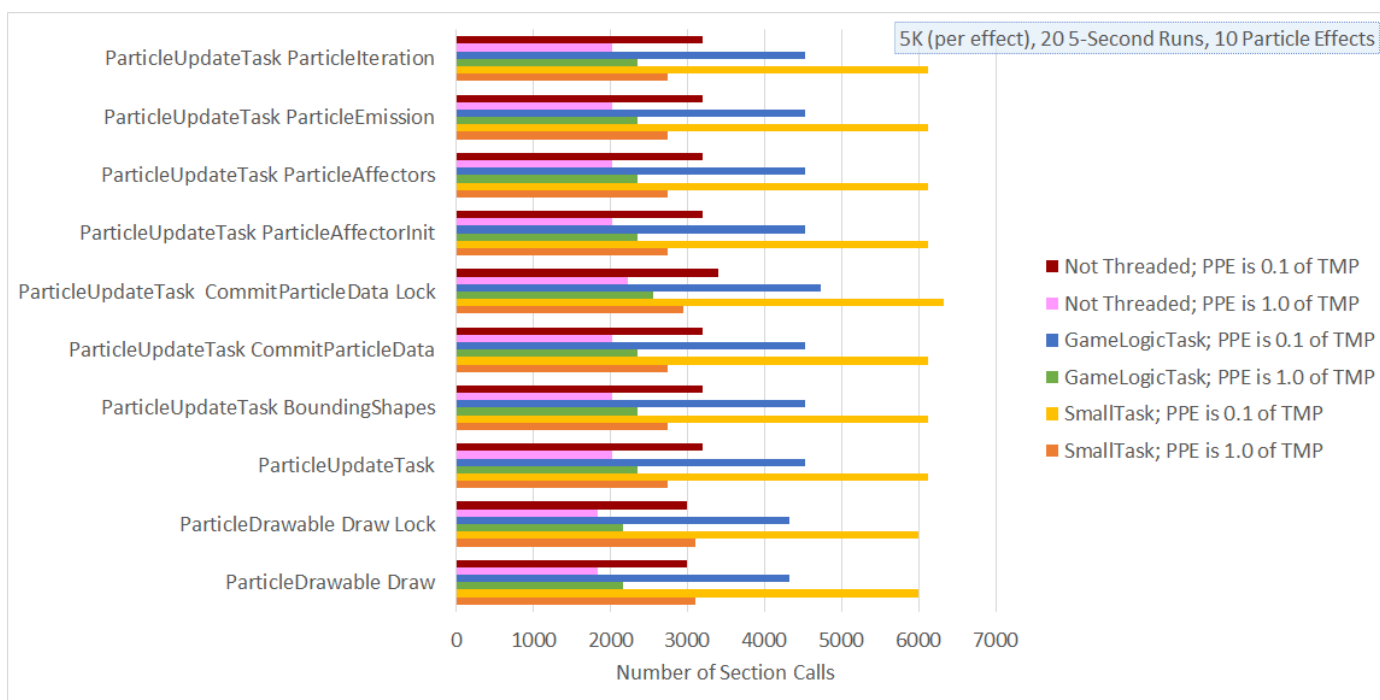


Figure 6.15 A graph showing the number of section calls for various sections of code for ten particle effects.

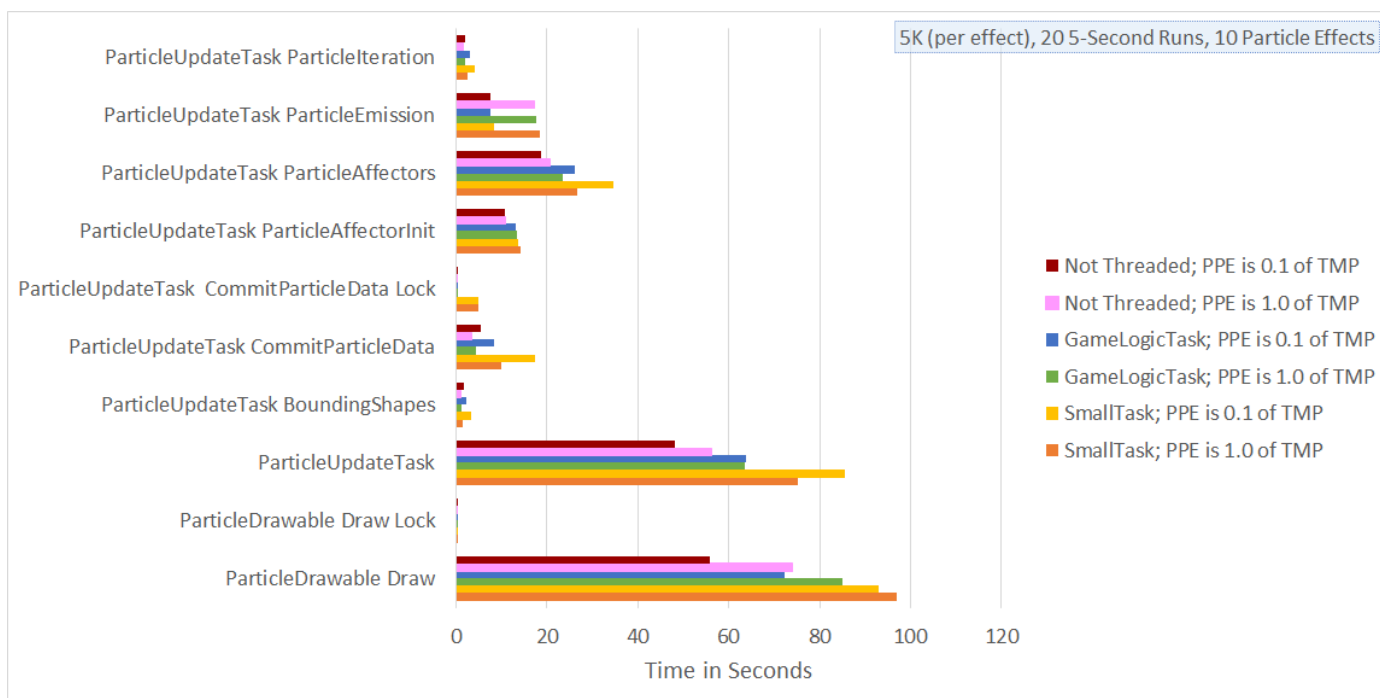


Figure 6.16 A graph showing the time spent in various sections of code for ten particle effects.

to generate these figures.

6.3.4 Discussion

For a single particle effect, the Not Threaded and GameLogicTask data series share nearly identical values. They render the same amount of particles, they render the ball just as many times, and they spend the same amount of time in various parts of the code. This shows us that, at least in the case of a single effect, the main thread waits about the same amount of time for the *game logic task* as if it was not even threaded.

For multiple particle effects, on the other hand, the overall performance of the Not Threaded data series is worse than the GameLogicTask data series. Although the timing metrics between the two data series are not too different, the particle metrics show that the Not Threaded data series rendered less particles and the ball was not rendered as many times. These results reveal that being able to process multiple effects at the same time before executing the main thread tasks proved to be more efficient than processing them sequentially.

The results of this particular study demonstrate that scheduling background tasks may not necessarily be faster than its single-threaded implementation. Although it seems obvious, it shows that being able to process many small tasks at once is more efficient than processing a single large task. As a software engineer, it is important to understand the lesson that this teaches. In general, a multi-threaded program often requires a considerable amount of overhead and is challenging to debug. Forcing a multi-threaded solution on a problem that does not easily lend itself to doing many small tasks at once may not yield optimal results. Thus, it is critical to structure multi-threaded programs such that it leverages these strengths.

6.4 Particles Failing to Visually Emit

6.4.1 Emission Failure

Particle Effect Components sometimes failed to visually emit particles during automated games. When this behavior was observed, particle metrics "particles actually rendered" and "particles actually emitted" were added to the instrumentation systems in order to numerically quantify (and, perhaps, understand) the failure. Two main variants of this failure emerged:

1. Particles were slow to emit, and it may look like it never emitted if the game's duration was too short
2. Particles never emitted even if the game's duration was lengthened significantly

The first variant usually exhibits a low "render ball function call" metric, i.e. a low frames per second, while the second variant does not seem to suffer from a performance hit in the slightest. The first kind is, in a way, expected for particle effects with a large number of particles simply because the renderer is slowed by the sheer number of iterations it is forced to make every update cycle. The second kind, on the other hand, is puzzling because it is not expected behavior.

6.4.2 Emission Failure Source

In order to demonstrate and understand this particle failure, metrics were generated using the same four scenarios as we used in the past:

1. 1 particle effect, particles range from 0 - 50K, 5K step, 5 5-second runs per game.
2. 1 particle effect, 50K, 0 step, 20 5-second runs.
3. 10 particle effects, particles range from 0 - 5K, 500 step, 5 5-second runs per game.
4. 10 particle effects, 5K, 0 step, 20 5-second runs.

Again, all of these scenarios were ran with PPE values that were 10% and 100% of TMP. The two data series, Non-Looping and Looping, are *Particle Effect Components* that have *isLooping* set to false and true, respectively.

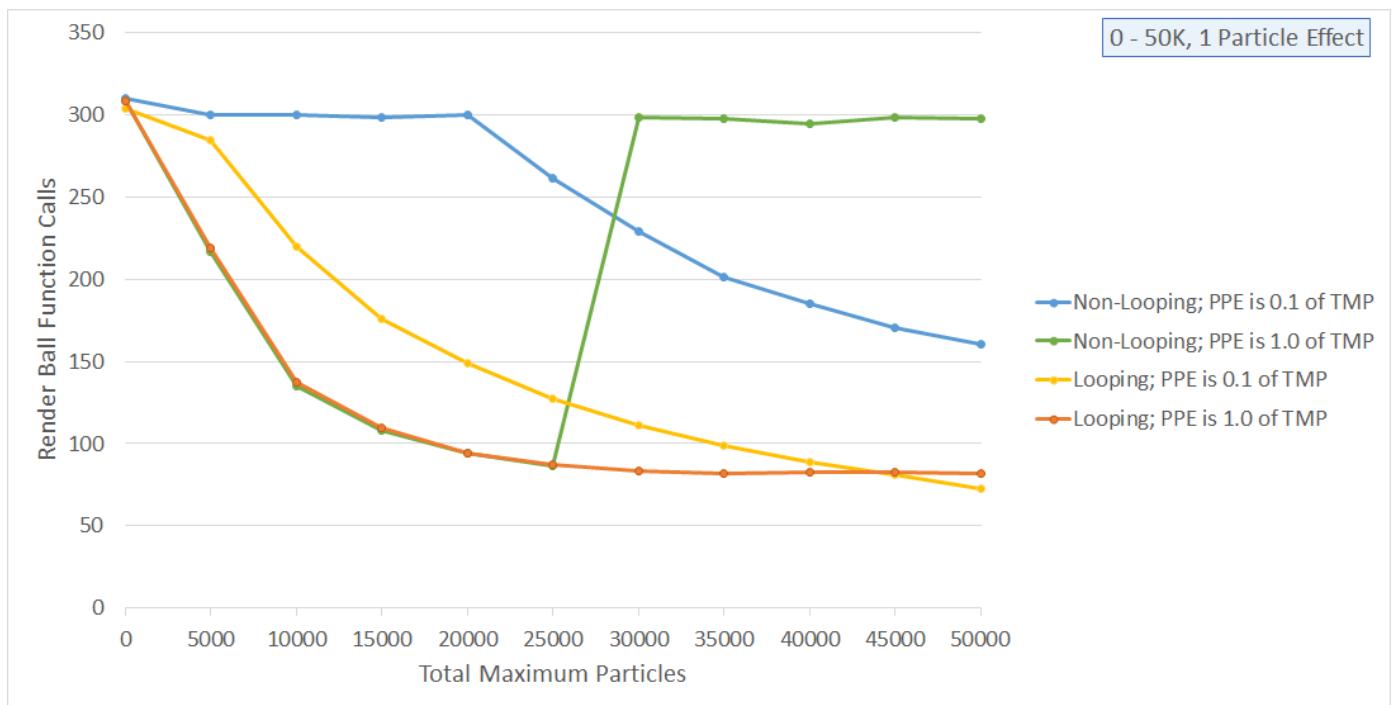


Figure 6.17 A graph showing *render ball function calls* over *total maximum particles* for a single particle effect.

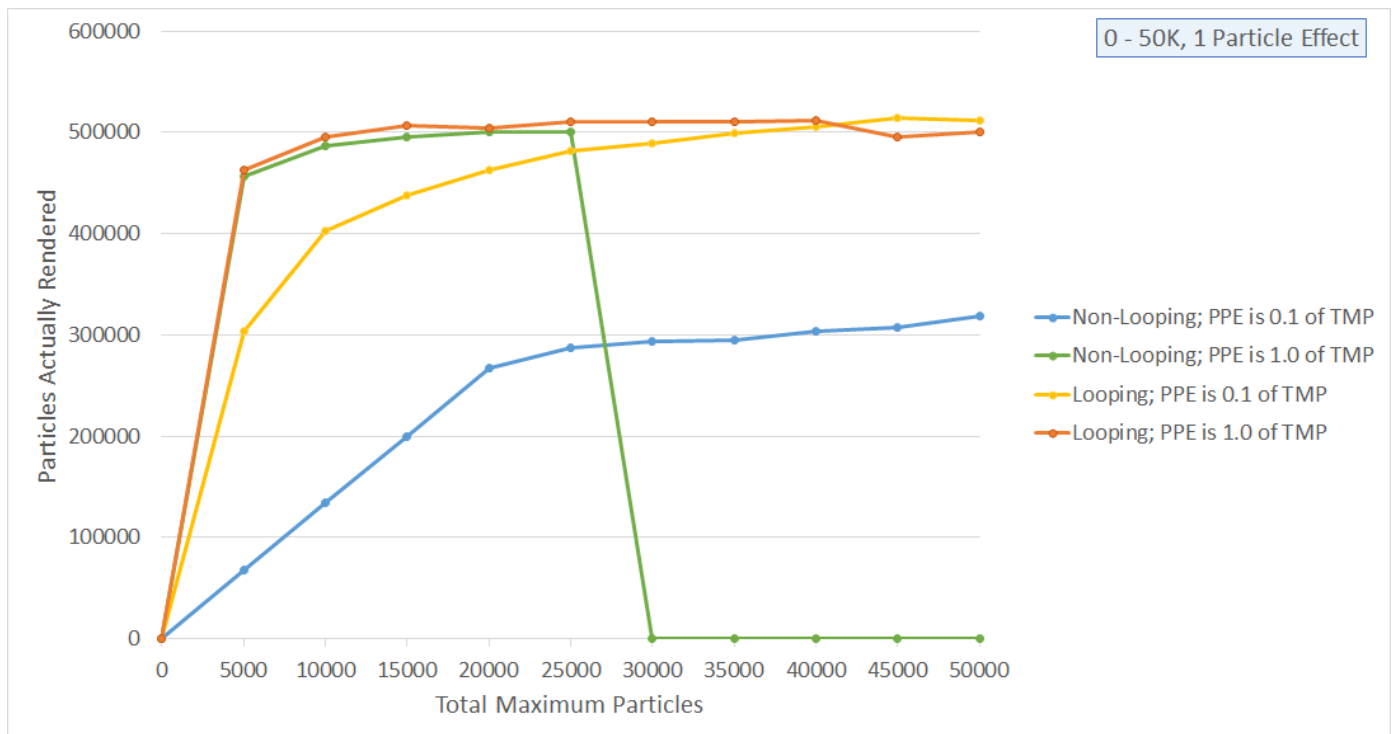


Figure 6.18 A graph showing *particles actually rendered* over *total maximum particles* for a single particle effect.

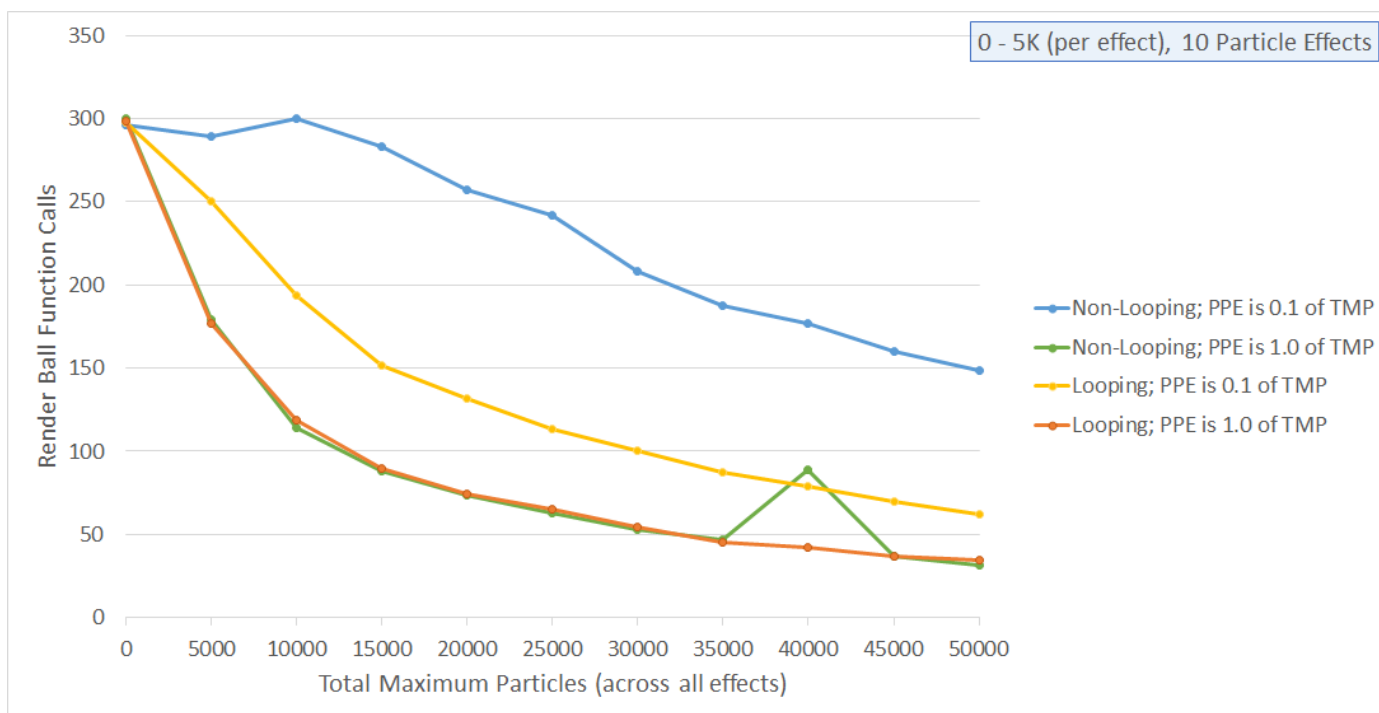


Figure 6.19 A graph showing *render ball function calls* over *total maximum particles* for a single particle effect.

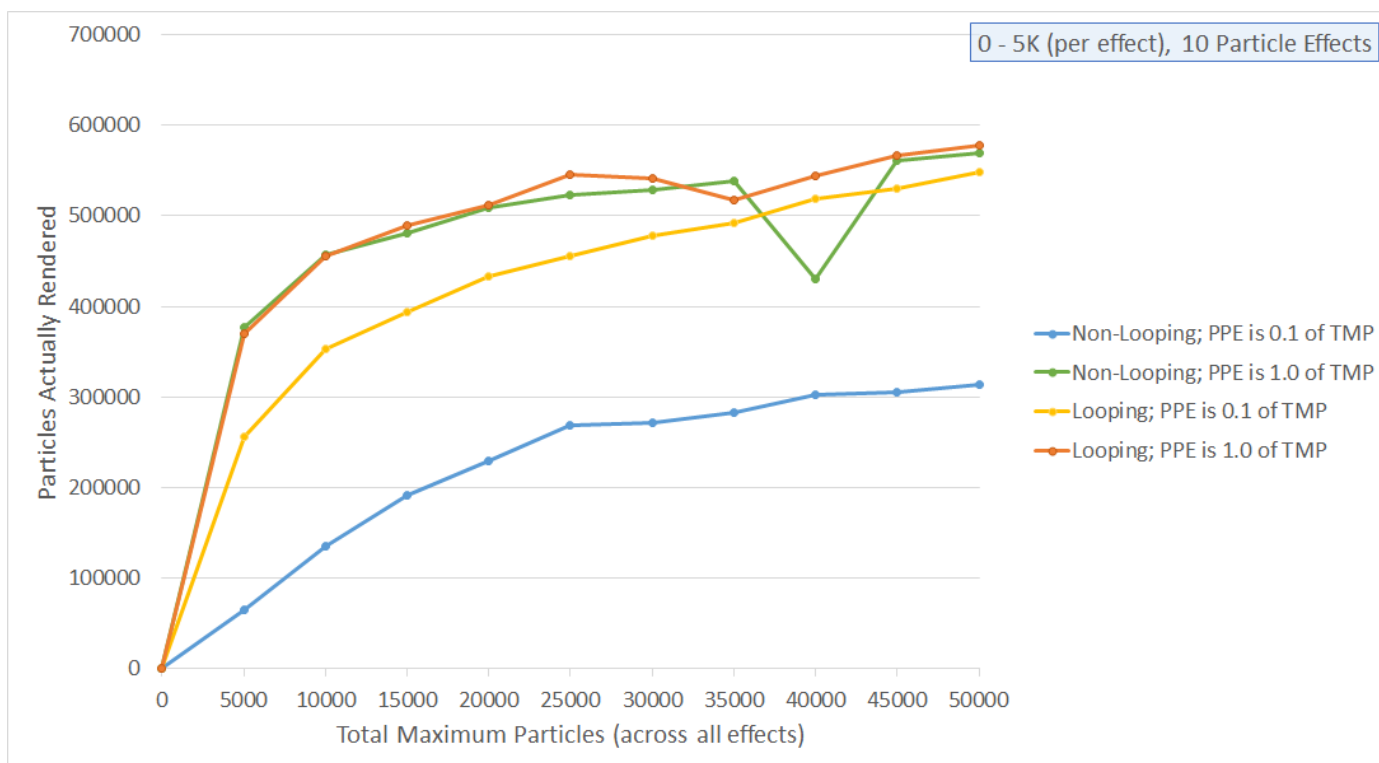


Figure 6.20 A graph showing *particles actually rendered* over *total maximum particles* for a single particle effect.

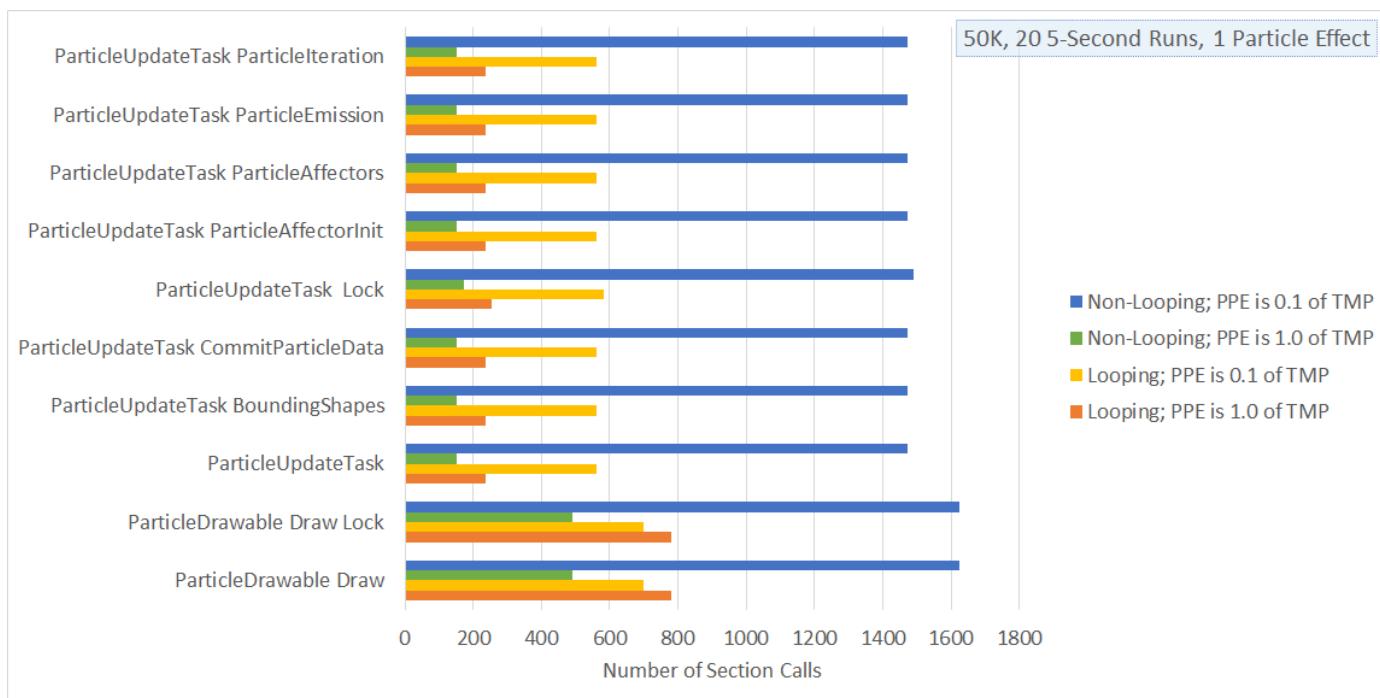


Figure 6.21 A graph showing the number of section calls for various sections of code for a single particle effect.

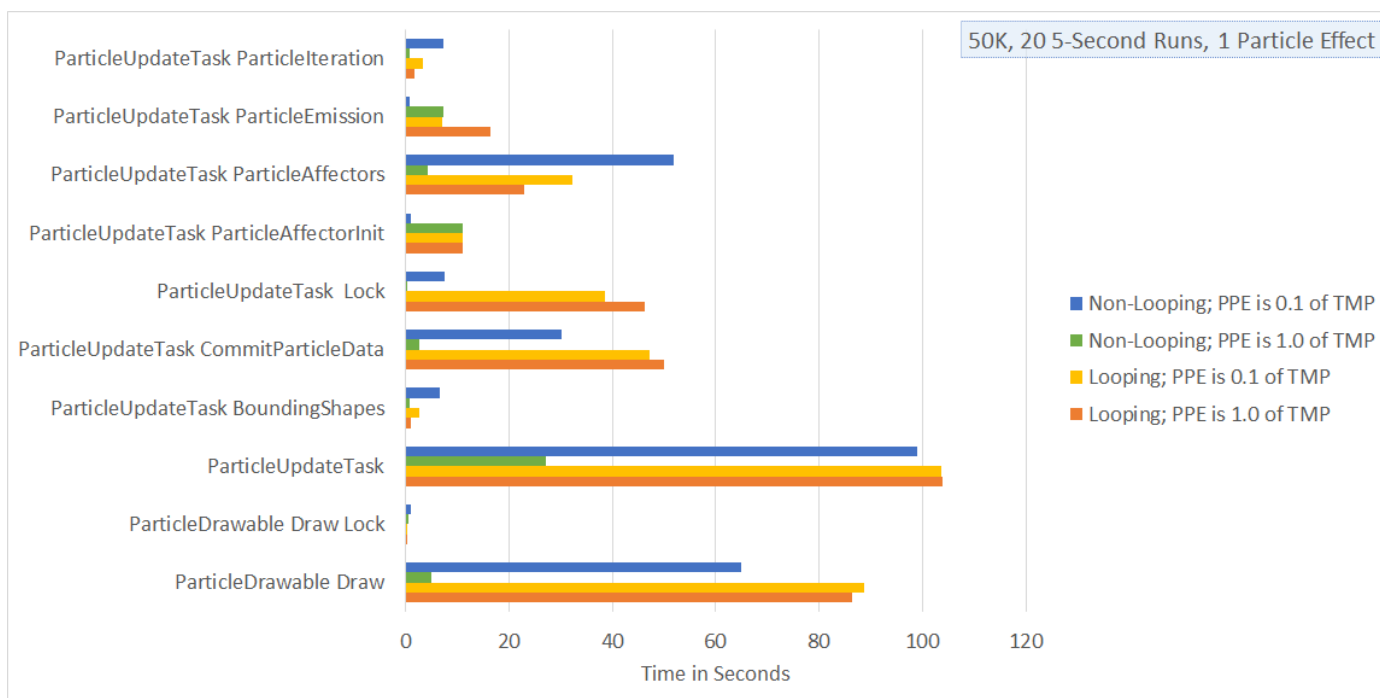


Figure 6.22 A graph showing the time spent in various sections of code for a single particle effect.

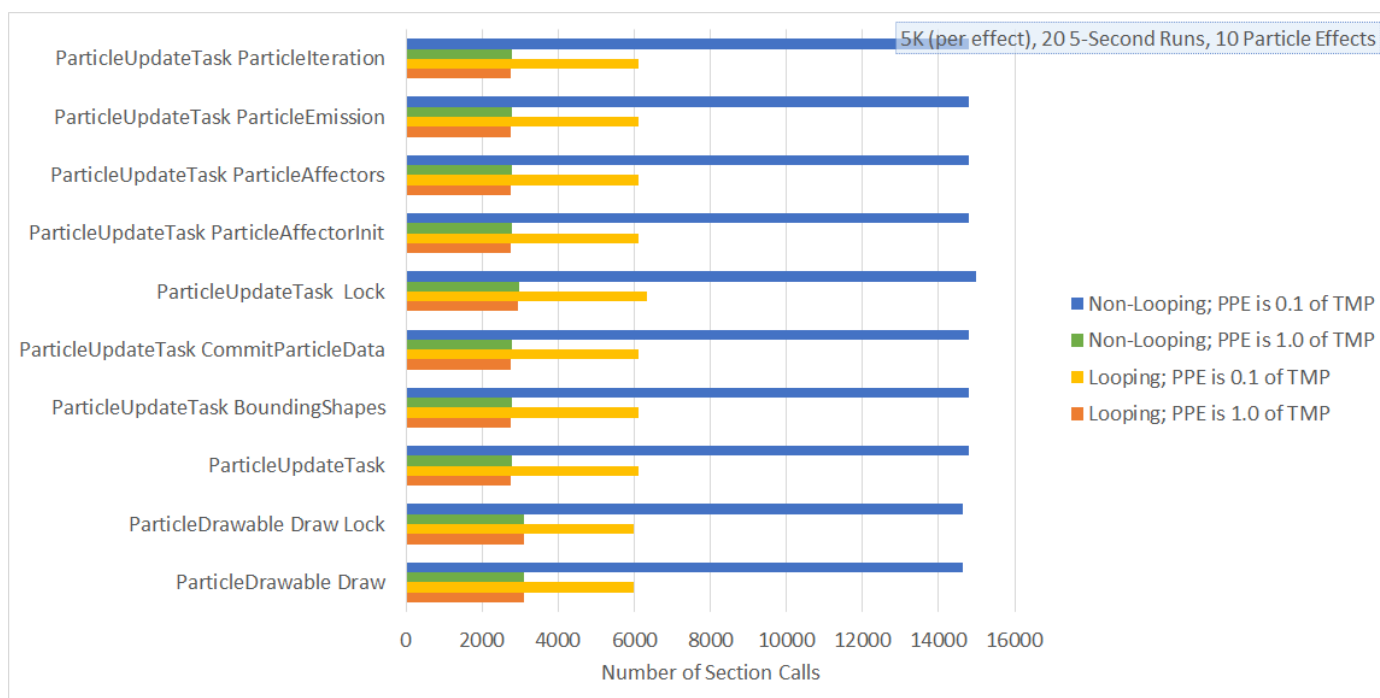


Figure 6.23 A graph showing the number of section calls for various sections of code for ten particle effects.

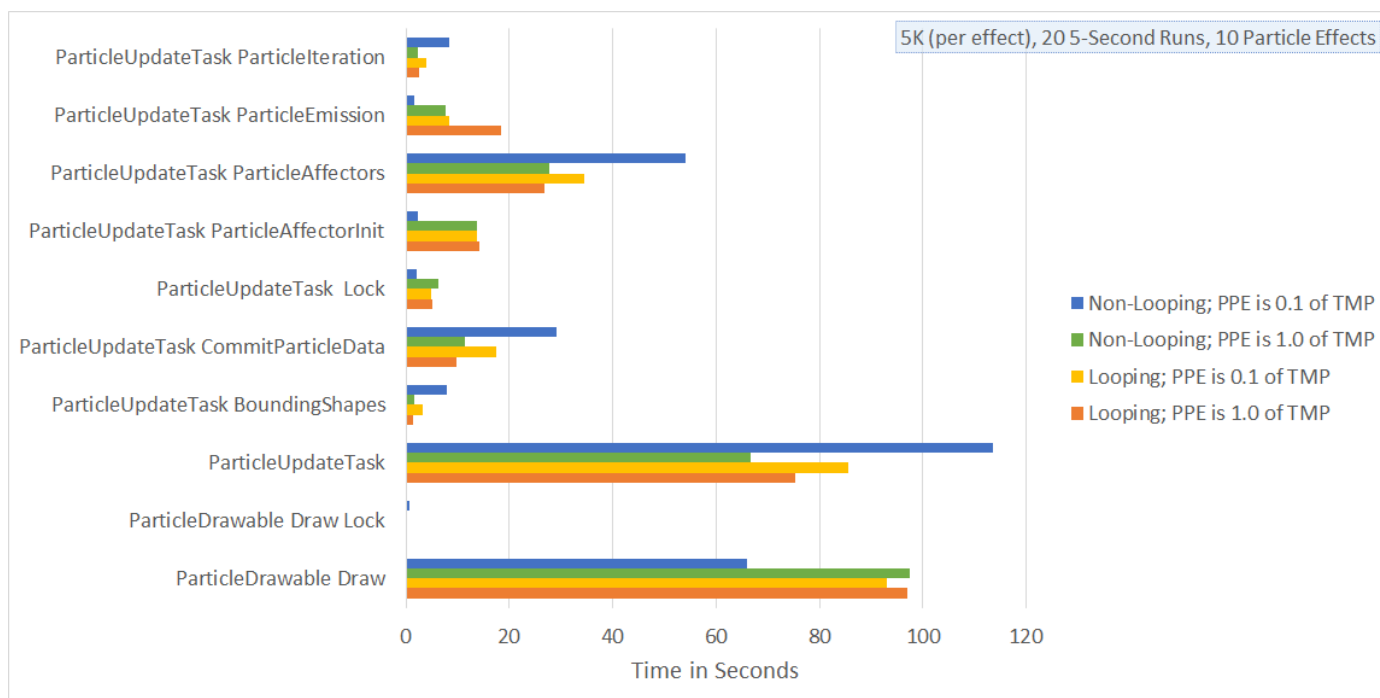


Figure 6.24 A graph showing the time spent in various sections of code for ten particle effects.

From these results, we can clearly see that the "Non-Looping; PPE is 1.0 of TMP" data series for a single particle effect demonstrates the catastrophic emission failure. At about 30,000 total maximum particles, zero particles are rendered and, as a result, the *render ball function calls* skyrocket since the renderer is not focusing on rendering particles. What is also interesting about this series is the extremely short amount of time that is spent in **ParticleUpdateTask**. We should expect it to take up most if not all of the $5seconds/run * 20runs = 100seconds$ like the other data series. Instead, it only spends about 27 seconds in **ParticleUpdateTask**.

The other data series do not experience the same catastrophic failure; the only one that comes close is the "Non-Looping; PPE is 1.0 of TMP" data series for multiple particle effects. For this data series, we see the *particles actually rendered* dip and the *render ball function calls* rise at 40,000 total maximum particles. Thus, this suggests that **non-looping** particle effects with a large number of particles per emission are at the heart of the issue. However, we still do not exactly know why this occurs.

6.4.3 Emission Failure Solution

Discovering why this failure occurred was a multi-step process. The automated game was set to run with a single effect with PPE at 1.0 TMP and a particle value of at least 30,000. Since this seemed to be a rendering issue, breakpoints were set in the **Render** function within the *Particle Effect Component* class. Interestingly enough, the **Render** function was never called during the game. To further investigate, breakpoints were placed within the renderer itself when it iterated over *Render Components* and, specifically, when it iterated over a *Particle Effect Component*. It turned out that the effect was being "culled", or ignored, because the bounding information associated with the effect was set to its initial value: *INTEGER_MAX*. Since the actual, non-default value of its bounding information is set inside of **ParticleUpdateTask**, this suggests that **ParticleUpdateTask** was not being called anymore. Placing breakpoints within the **Update** method in the *Particle Effect*

Component revealed that it calls the **Stop** function and ceased to schedule **ParticleUpdateTask**. This was called because the effect met two conditions:

- The effect's duration elapsed
- There were no active particles

These conditions were met because **ParticleUpdateTask** had not updated or activated any particles within the effect's duration time, which was 1 second. Thus, it can be inferred that a non-looping particle effect coupled with a small duration and a large number of particles results in particle failure. Increasing the duration of a such a particle effect should fix the problem.

The following figures present metrics that compare the emission failure from above with the solution to that failure. The metrics from this "Solution" data series are the result of running the same scenarios as above with a non-looping particle effect, but this non-looping particle effect has an increased duration time of 120 seconds. An examination of these results will show that this solution does indeed solve the problem. Particles are rendered from 0 to 50,000 total maximum particles, and a healthy amount of time is being spent within **ParticleUpdateTask**, as expected.

This study has shown that, when using a non-looping particle effect with a large number of particles, a large duration value should be set as well. Although this may seem like *ChilliSource* should disallow this from happening, it is also true that this resulted from a misconfiguration of a particle effect. A game developer should not expect that a duration of a single second is reasonable for a particle effect with at least 30,000 particles. However, it would still be useful to at least have a warning printed to the console that informs the developer that a "failing" or "broken" particle effect may be due to a small duration time.

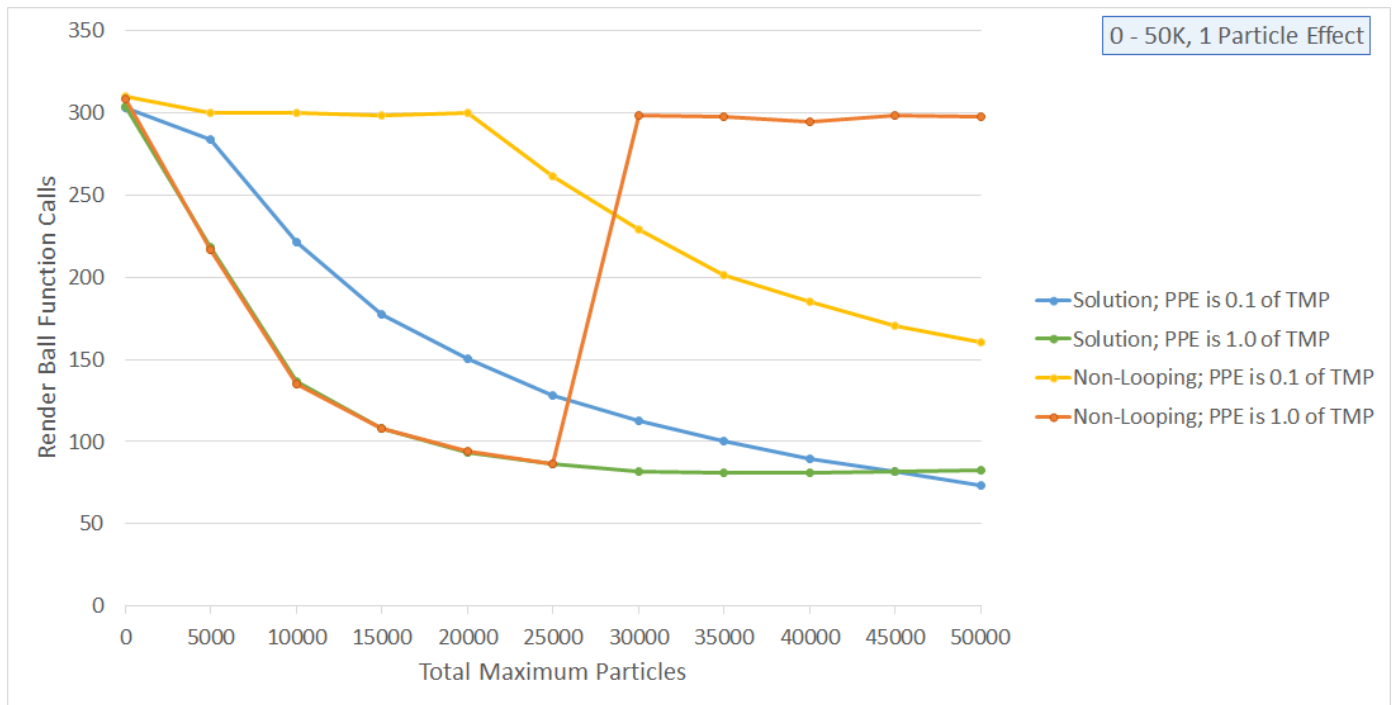


Figure 6.25 A graph showing *render ball function calls* over *total maximum particles* for a single particle effect.

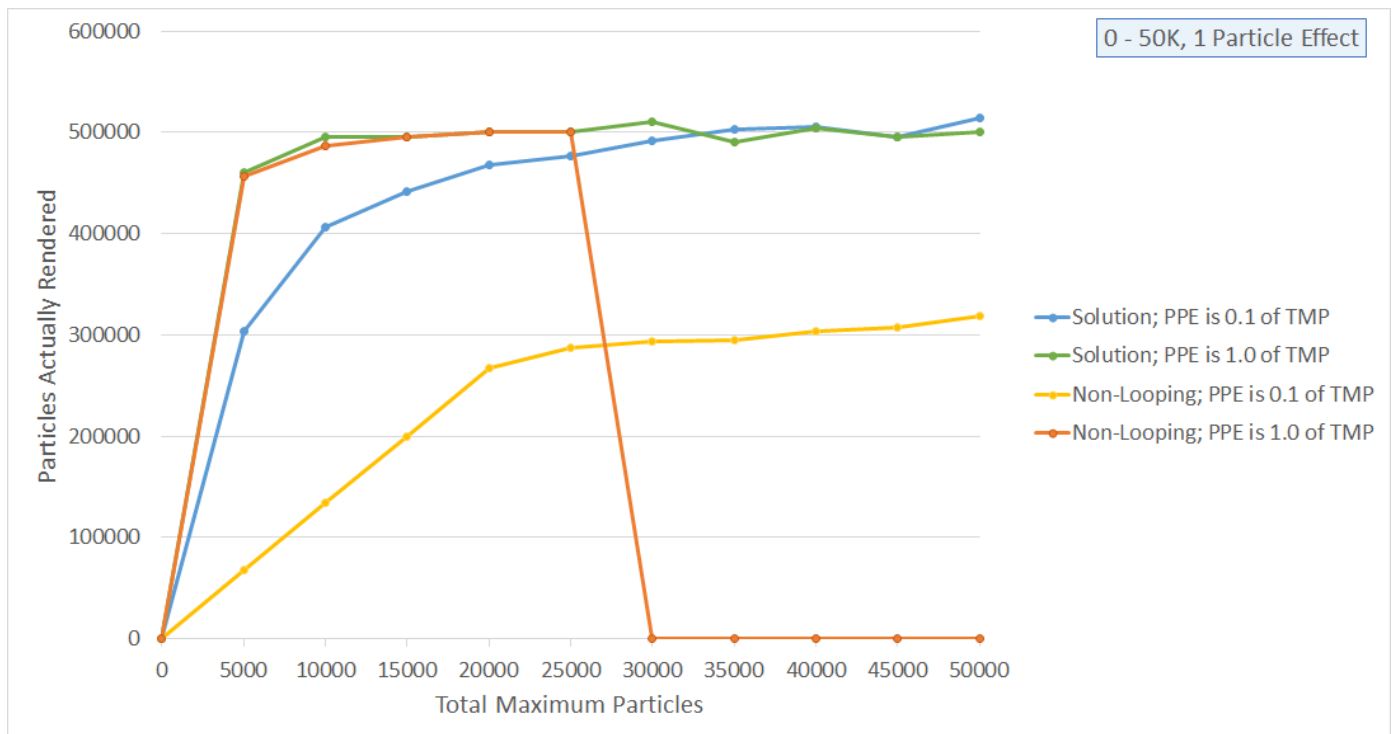


Figure 6.26 A graph showing *particles actually rendered* over *total maximum particles* for a single particle effect.

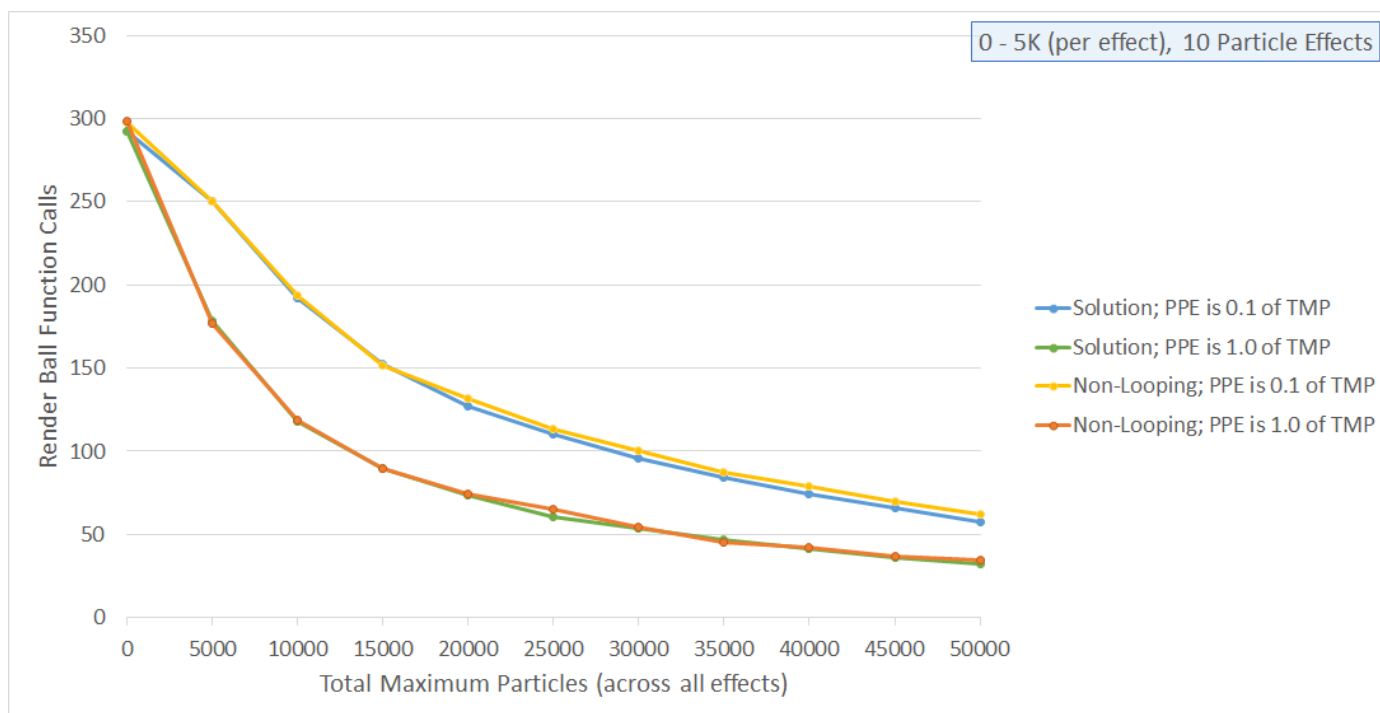


Figure 6.27 A graph showing *render ball function calls* over *total maximum particles* for a single particle effect.

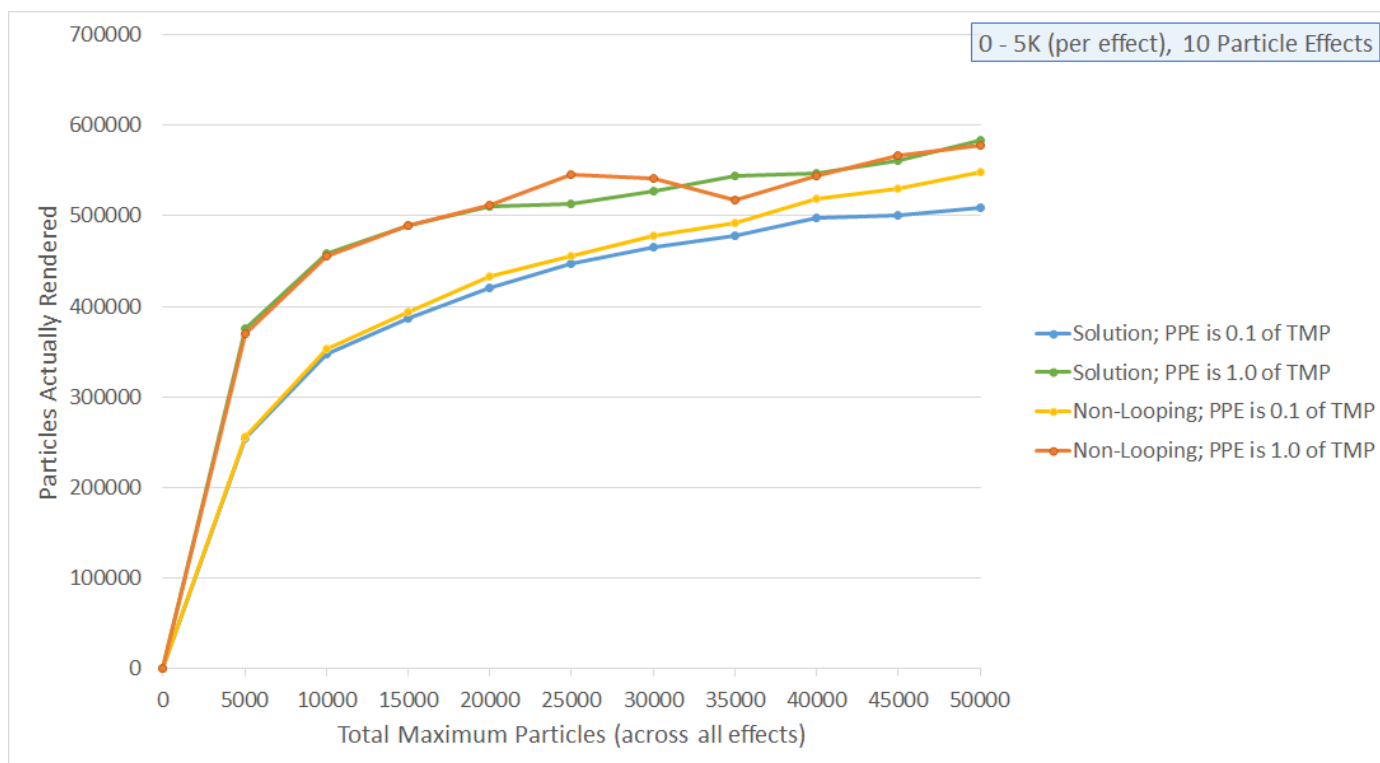


Figure 6.28 A graph showing *particles actually rendered* over *total maximum particles* for a single particle effect.

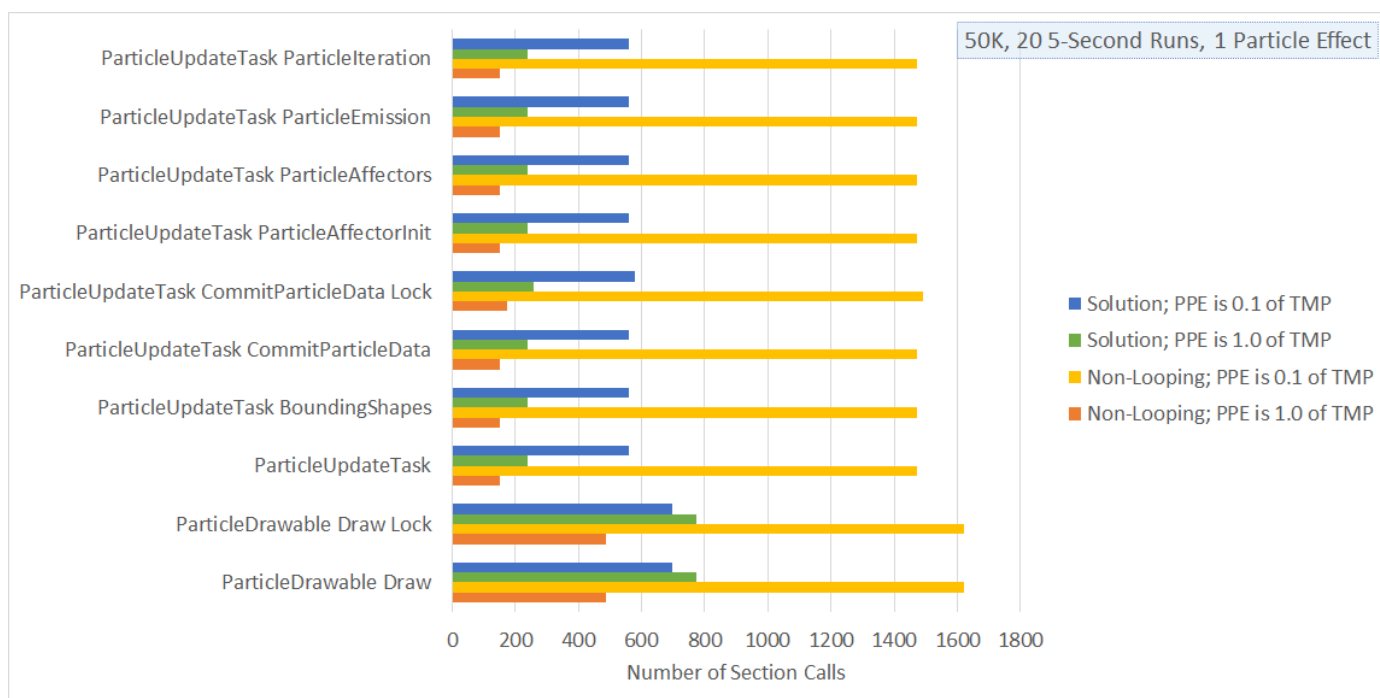


Figure 6.29 A graph showing the number of section calls for various sections of code for a single particle effect.

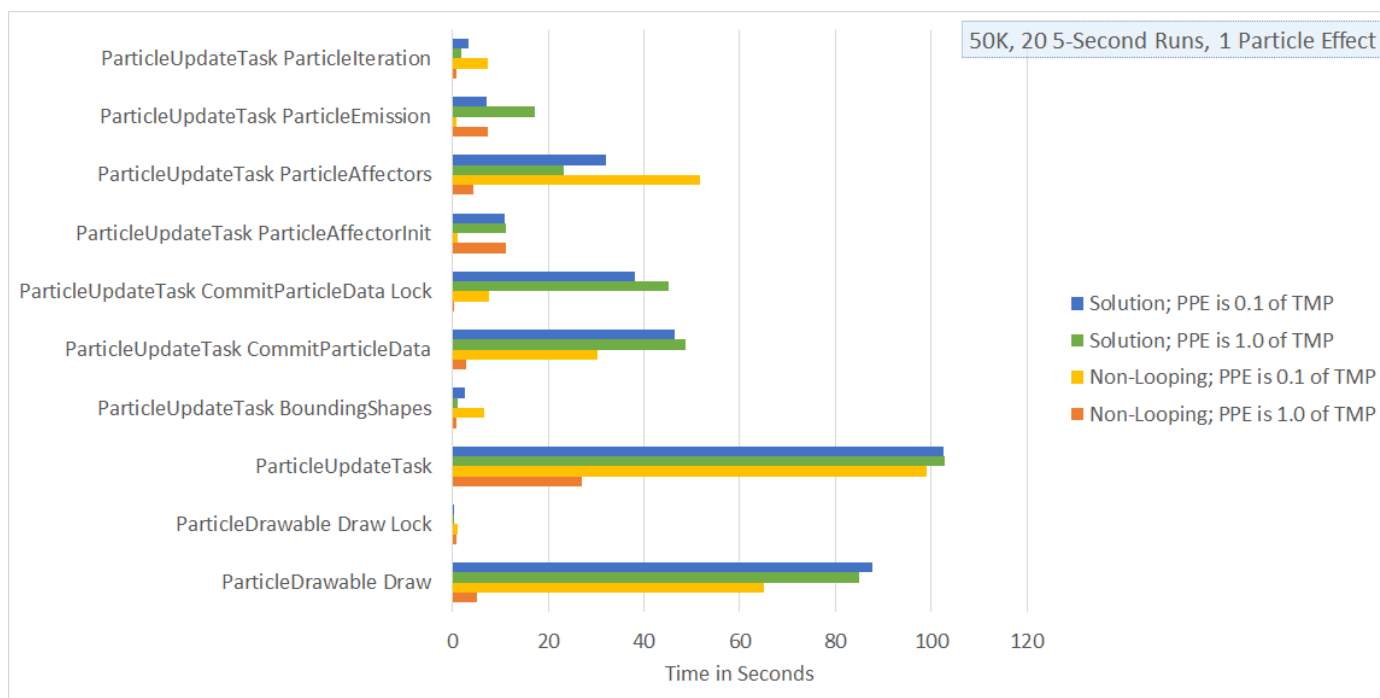


Figure 6.30 A graph showing the time spent in various sections of code for a single particle effect.

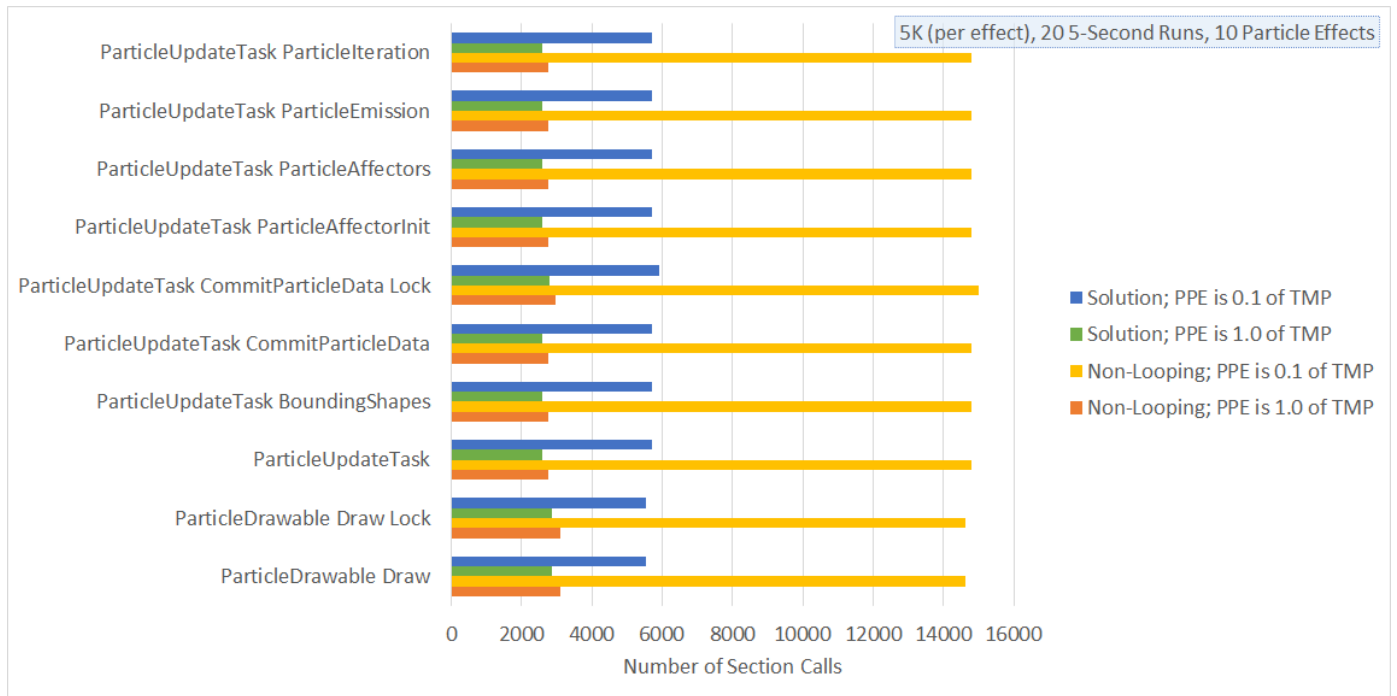


Figure 6.31 A graph showing the number of section calls for various sections of code for ten particle effects.

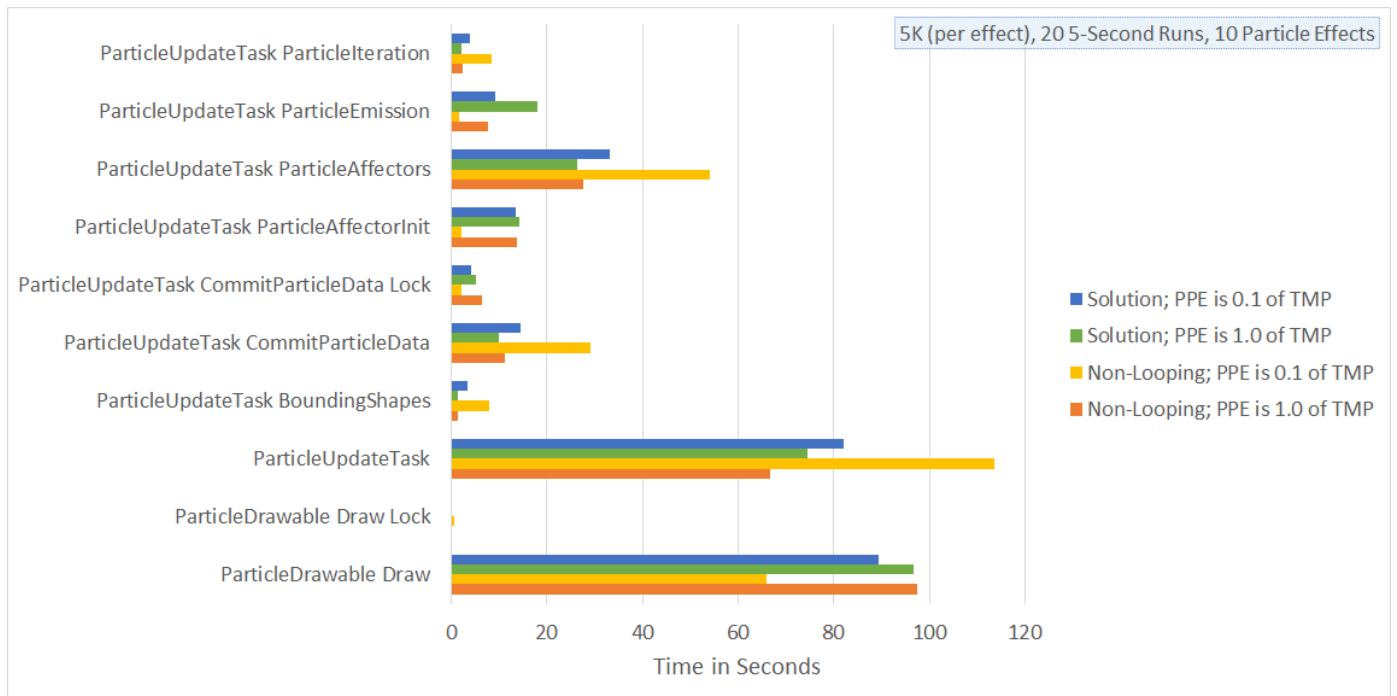


Figure 6.32 A graph showing the time spent in various sections of code for ten particle effects.

BIBLIOGRAPHY

- [1] W. T. Reeves, “Particle systems – a technique for modeling a class of fuzzy objects,” in *Proceedings of the 10th Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH '83. New York, NY, USA: ACM, 1983, pp. 359–375. [Online]. Available: <http://doi.acm.org/10.1145/800059.801167>
- [2] ChilliWorks. (2014) Features. [Online]. Available: <http://www.chillisourceengine.com/features>
- [3] ——. (2014) Life cycle events. [Online]. Available: <http://www.chillisourceengine.com/life-cycle-events>
- [4] ChilliWorks and A. Gross. (2016) Chillisource automated cspong game. [Online]. Available: <https://bitbucket.org/angelahnicole/um-thesis-cspong-benchmarking/src/c00b947411a7e04e00d464108dee6235e3d692c1/?at=automation>
- [5] ChilliWorks. (2016) Chillisource sample projects. [Online]. Available: <https://github.com/ChilliWorks/CSSamples>
- [6] A. Abedi and D. Love. (2011) Package of the shiny profiler by aidin abedi. [Online]. Available: <https://github.com/dlove24/Shiny>
- [7] HDFGroup. (2016) High level introduction to hdf5. [Online]. Available: <https://support.hdfgroup.org/HDF5/Tutor/HDF5Intro.pdf>
- [8] S. Mullender and R. Cox, “Semaphores in plan 9,” in *3rd International Workshop on Plan*, vol. 9, 2008, pp. 53–61.

- [9] cppreference. (2015) Atomic compare exchange. [Online]. Available: http://en.cppreference.com/w/cpp/atomic/atomic_compare_exchange
- [10] ChilliWorks. (2014) Basic structure. [Online]. Available: <http://www.chillisourceengine.com/basic-structure>
- [11] Halixi72. (2007) Particle emitter. [Online]. Available: https://en.wikipedia.org/wiki/File:Particle_Emitter.jpg
- [12] ——. (2007) Strand emitter. [Online]. Available: https://en.wikipedia.org/wiki/File:Strand_Emitter.jpg
- [13] ChilliWorks and A. Gross. (2016) Chillisource v1.6.0 code repository. [Online]. Available: <https://bitbucket.org/Chilli-Ian/chillisource-ag/src/781988bb9a8ee518c4ea6545ac0e5c55238ec562?at=temp/particleMetrics>